ADAPTOFLUX: A TYPED GRAPH REPRESENTATION SUBSTRATE FOR STATEFUL SYMBOLIC PROGRAM SYNTHESIS

Anonymous authors
Paper under double-blind review

ABSTRACT

AdaptoFlux is a representation substrate for composing typed numerical operations and stateful actions in executable graphs. A metadata-driven Method Pool supplies typed interfaces, while a Generalized Data Flow Graph (GDFG) represents data reuse and per-step computation. External state connects successive graph executions. We make the execution contract explicit: a persistent program order and resource-based control dependencies determine conflicting state accesses while retaining parallelism between independent operations. The representation supports both discrete structure search and parameter optimization. In a controlled stateful synthesis experiment with identical operator implementations, fine-grained, role-merged, and flat type declarations achieve strict success in 29/30, 15/30, and 1/30 runs. A paired translation audit matches implicit and explicit state execution across 70,670 environment steps. Fixed-depth execution measurements cover up to 1,024 operation nodes and 2,176 edges. A shared-feature action controller supplies a concrete example: learned action scores feed an exclusive router, and its adapted GDFG matches a frozen controller for 20,480 steps. Further experiments examine operator-pool expansion and sustained execution in a continual-learning application. Together, these results characterize how typed composition, state ordering, and optimization interact in one executable representation.

1 INTRODUCTION

Symbolic program synthesis searches over discrete executable structures. Its representation determines which operations can be composed, where intermediate results can be reused, and how a program interacts with mutable state. These choices matter for control tasks: a controller must combine numerical computation with observations, action selection, and environment updates, often over many successive steps. AdaptoFlux makes these interfaces explicit through a typed Method Pool and Generalized Data Flow Graphs (GDFGs).

Consider an action controller that reads an observation o_t and an internal state h_t, computes shared features z_t, scores each available action, and executes the action selected by an exclusive router. One feature can contribute to several action scores without being recomputed. The state-update branch commits h_{t+1} after reading h_t, and the same stored graph runs at the next step. This structure combines typed dataflow, learned numerical parameters, and stateful effects in a single program (Figure 1).

The representation separates three decisions. *Type signatures* define admissible data connections. *Execution contracts* define how nodes access shared resources. *Search procedures* select structures and parameters. This distinction permits several optimizers to use the same graph interface and prevents a search preference from being mistaken for a runtime guarantee. In particular, a data DAG gives a partial order: two independent data branches can still write the same external state. We therefore represent their selected state-access order explicitly.

The paper makes four contributions. First, it specifies a common registration and execution interface for typed operations, reusable subgraphs, and actions. Second, it formalizes stateful execution using a fixed program order and additional control dependencies, implemented consistently across training, inference, and model persistence. Third, controlled experiments quantify the effect of semantic

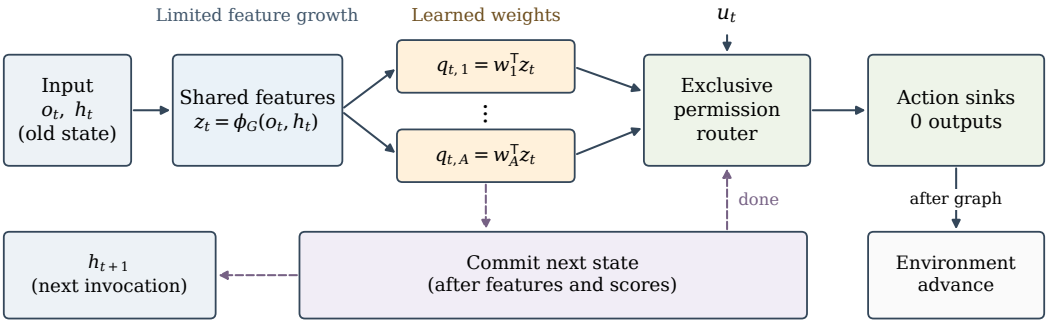


Figure 1: A concrete GDFG controller. Shared features feed per-action scores; an exclusive router activates one action. The state branch reads the old state and commits the next state. Dashed purple paths show commit/control relations. The template supplies these interfaces, CMA optimizes score coefficients, and scheduled proposals can extend the feature graph.

type declarations, compare implicit and explicit state encodings, and measure execution costs as graph size increases. Fourth, a parameterized action-controller example and larger application studies demonstrate how the substrate supports different optimization and reuse workflows. LayerGrow, GraphEvo, and the CMA-based controller optimizer provide distinct navigation procedures for this shared representation.

The implementation is available through the anonymous review repository at `https://anonymous.4open.science/r/AdaptoFlux-CF54`. Protocols and the provenance of the supplementary measurements are specified in Appendices A–H.

2 Positioning and Related Work

Typed program composition, graph reuse, and effect ordering have substantial prior foundations. Strongly typed genetic programming constrains compositions through types Montana (1995); Cartesian Genetic Programming provides graph-based reuse Miller & Turner (2015); and MT-CGP explicitly supports mixed data types Harding et al. (2012). LLVM SelectionDAG combines typed dataflow with chain dependencies for side-effect ordering LLVM Project (n.d.). These precedents motivate our distinction between data edges and control dependencies.

AdaptoFlux brings these mechanisms together as a runtime-facing substrate for symbolic synthesis: a registry accepts heterogeneous Python operations, a graph exposes typed data ports, external-state operations share the execution interface, and synthesized subgraphs can be registered as methods. Search procedures operate on this interface. DreamCoder studies learned reusable abstractions in a functional language Ellis et al. (2021); Popper combines logical background knowledge with failure-driven constraint learning Cropper & Morel (2021); symbolic-policy and language-model-guided discovery emphasize search quality Landajuela et al. (2021); Romera-Paredes et al. (2024). These approaches offer complementary optimization and abstraction mechanisms. Our experiments characterize the executable representation and its interaction with concrete optimizers. A possible future extension is to use a language model to propose candidates for the same execution and evaluation interface (Appendix R). Appendix I develops the comparisons; Appendix M reports the static symbolic-regression benchmarks against gplearn and PySR Cranmer (2023).

3 System Architecture

3.1 Method Pool

A method is registered as $m = \langle \mathrm{impl}, \sigma_{\mathrm{in}}, \sigma_{\mathrm{out}}, \mathrm{M} \rangle$, comprising its implementation, positional input and output type signatures, and metadata. Types are declared atoms, such as `scalar`, `pos_type`,

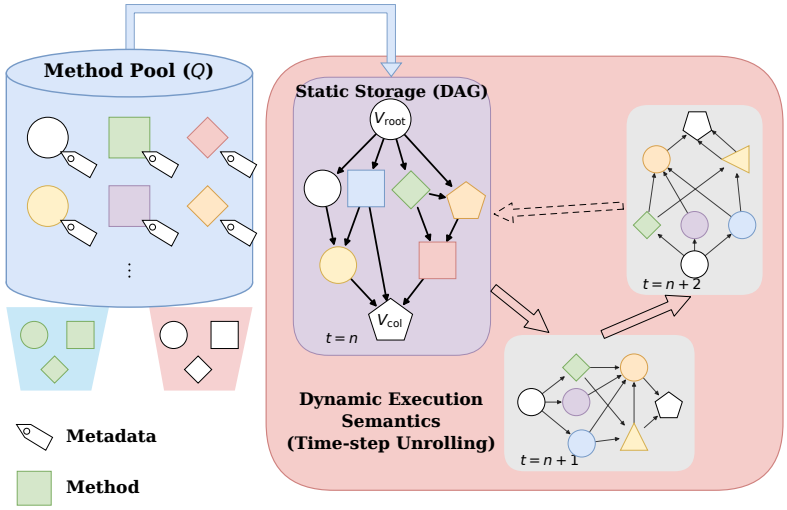


Figure 2: AdaptoFlux architecture. The Method Pool defines callable interfaces, the stored GDFG carries data dependencies, and the runtime connects successive steps through state. Effect-ordering plans add control dependencies to the per-step schedule.

or `target_type`. Each input position lists the atoms it accepts; each output port declares the atom it emits. Arity and positional routing determine which outputs can supply which inputs.

Metadata has two roles. Search annotations, including groups and sampling weights, select a view $\mathrm{Q}_{\text{view}} \subseteq \mathrm{Q}$ of the pool. Execution annotations declare the resources a method reads and writes, denoted R_m and W_m, or identify a method as pure. Undeclared effects are treated conservatively as unknown. The same resource name must describe accesses to the same external state. A registry entry can also contain a reusable subgraph; its effect contract aggregates those of its constituents. Appendix P.4 separates these execution contracts from search preferences.

3.2 GDFGs and Stateful Execution

A data graph $G = \langle V, E_d \rangle$ is a DAG with input, operation, and output-aggregation interfaces. Every operation node is bound to a registered method. An edge connects a particular output port to a particular input position, and is admissible when its emitted type belongs to that position's accepted type set. The data-graph hypothesis class is

$$\Omega_{\mathrm{Q},\Sigma} = \{G : \ G \text{ is a DAG}, \ m_v \in \mathrm{Q}, \ \text{and every data edge respects its port types}\}. \tag{1}$$

Declared type compatibility is checked during graph construction. The declarations are callable interface contracts: method implementations must also respect their declared inputs and outputs.

State transitions require a selected order. For fixed input x_t, a node computes data outputs and may update the current external state. Let $\pi = (v_1, \ldots, v_m)$ be a selected topological order. Writing F_v for the state component of node execution, one step is

$$s^{(0)} = s_t, \qquad s^{(k)} = F_{v_k}(\mathbf{in}_{v_k}, s^{(k-1)}, x_t), \qquad s_{t+1} = s^{(m)}. \tag{2}$$

The choice of π matters. Starting from $s = 1$, independent nodes that apply $s \leftarrow s + 1$ and $s \leftarrow 2s$ produce either 4 or 3. Both orders may satisfy the same data DAG. We therefore define the stateful executable program by $P = (G, \pi, E_c)$, where E_c contains control dependencies that preserve the selected order of conflicting accesses.

Conflict-aware scheduling. Two nodes conflict when

$$W_u \cap (R_v \cup W_v) \neq \varnothing \quad \text{or} \quad W_v \cap R_u \neq \varnothing. \tag{3}$$

For each conflicting pair with $u \prec_\pi v$, the augmented graph $G^+ = (V, E_d \cup E_c)$ contains a dependency path from u to v. The implementation constructs these paths using resource read/write

frontiers; control dependencies carry completion information, while function arguments come only from E_d. Shared reads and accesses to disjoint resources remain available for parallel execution. Unknown methods are conservatively ordered with other effectful methods.

With complete effect declarations, deterministic operations, the same inputs and initial state, and successful execution, legal schedules of G^+ produce the same outputs and final state. Adjacent independent operations can be exchanged without changing their inputs or resource observations; conflicting operations retain their order. Appendix A gives the execution conditions and validation. If operation-node updates admit increments I_v applied by $\oplus$, and those increments can be evaluated from the same pre-step state and commute on disjoint resources, Equation 2 reduces to

$$s_{t+1} = s_t \oplus \bigoplus_{v \in V_{\mathrm{proc}}} I_v(\mathbf{in}_v, s_t, x_t). \tag{4}$$

Training and persistence. Before a candidate is evaluated, the runtime freezes its initial order. Stable node identities retain that order through accepted substitutions, graph growth, copying, and save/load operations. An explicit order change defines a new candidate for evaluation. For effectful batches, the runtime finishes a complete sample before starting the next. The effect-ordering feature is enabled by default and can be disabled; the selected mode is stored with the model. Disabled mode follows data dependencies and node-wise batching, with no additional state-ordering guarantee.

Across time steps, external state or explicit state inputs connect successive executions of the same stored graph. A finite unrolling uses forward-in-time connections and remains acyclic (Appendix P.6). The hand-written Snake example uses mutually exclusive action permits; the controller in Figure 1 additionally exposes a read-old/commit-new internal-state boundary.

3.3 Navigation and Reuse

LayerGrow samples type-compatible operations to extend a graph, evaluates the candidate, and accepts or rolls back the extension. **GraphEvo** refines a fixed graph skeleton by replacing its bound methods, and can extract subgraphs into reusable methods. **CombinedTrainer** composes growth and refinement with pool filtering. Their algorithmic descriptions and original experiments are retained in Appendices J.3 and K. The action-controller study uses CMA to optimize numerical score coefficients and scheduled proposals to grow arithmetic features, illustrating a second optimization interface on the same form of executable structure.

3.4 Execution Cost and Parallelism

For V operations, E_d data edges, E_c control edges, batch size N, and average operation cost $\bar{C}_{\mathrm{op}}$, inference separates plan preparation, scheduling, routing, and computation:

$$T_{\mathrm{infer}} = T_{\mathrm{prepare}} + T_{\mathrm{sched}}(N, G^+) + O\big(N(|E_d| C_{\mathrm{route}} + V \bar{C}_{\mathrm{op}})\big). \tag{5}$$

The revised runtime validates the graph and reconstructs its plan on each call. Pure graphs use one batched traversal, whereas effect-ordered stateful batches use N complete traversals. The threaded implementation sorts ready nodes and creates a worker pool for each traversal; these costs belong to T_{sched}. Resource declarations and the selected ordering determine preparation and control bookkeeping. The historical backend's measurements and complexity breakdown are given in Appendix J.

Parallel benefit depends on the ready graph width, critical path, operation cost, and backend. The original simulated non-blocking workload achieved a mean $10.25\times$ speedup with 16 workers. The supplementary Python scalar CPU workload instead runs faster serially (Section 4.6). Reporting both regimes distinguishes available graph concurrency from the cost of realizing it.

3.5 Search-Space Structure

Type routing removes inadmissible port compositions before evaluation. Search annotations then select among admissible candidates; effect contracts determine their execution. Increasing the pool can introduce useful operations and simultaneously increase the choices explored under a fixed budget. LayerGrow's cost accumulates across repeated graph extensions, while GraphEvo refinement

Table 1: Semantic type priors improve stateful synthesis. All conditions use the same nine operator bodies and nominal search caps. Evaluation calls report the realized cost; intervals are Wilson 95% intervals across 30 runs.

Type declarations	Successful runs	95% interval	Mean calls
Position / target	29/30 (96.7%)	83.3–99.4%	59.47
Merged quantity	15/30 (50.0%)	33.2–66.8%	81.67
All scalar	1/30 (3.3%)	0.6–16.7%	738.67

holds the skeleton fixed. The experiments below measure these effects separately from execution scalability.

4 Experimental Results

We report three evidence groups: the original navigation experiments, supplementary controlled measurements, and applications developed with additional optimizers. The four supplementary studies of type priors, state encoding, execution scaling, and controller replay used the original frozen ATF backend. The revised effect-aware runtime has a separate regression suite and frozen-model replay audit. Appendix A records this version separation.

4.1 Structural Search with LayerGrow

On Titanic Cukierski (2012), LayerGrow uses nine scalar operations and a 2,500-attempt budget per run. Across ten runs per maximum-layer setting, mean best accuracy reaches 0.763 ± 0.041 at a configured maximum of 20 layers. Higher requested maxima produce little further improvement (means 0.769–0.783 for settings 25–50) and more budget exhaustion. At requested maxima 45 and 50, all runs exhaust the attempt budget; the mean reached depths are 19.6 and 18.7. Thus the observed failure concerns this growth procedure's progress under its budget. Figure 4 and Appendix K.1 give the full depth and cost results.

4.2 GraphEvo and Reusable Subgraphs

The task chain $f_1(x) = 2(x + 1)$, $f_2(x) = f_1(x) + 3$, and $f_3(x) = 0.5f_2(x)$ evaluates whether extracted structure can support a later task. Under the controlled two-layer setting, evolved methods yield Task 3 success in 13/100 runs, compared with 0/100 without evolution and 12/100 with a hand-supplied prior. The three-layer setting yields 9/100. Extraction enables reuse under the chosen initialization constraints, while deeper initialization changes the search difficulty. Appendix K.2 reports all conditions and Figure 5 separates partial and complete transfer chains.

4.3 Semantic Type Priors in Stateful Synthesis

The supplementary Minimal Stateful Attractor experiment holds nine operator implementations fixed and varies only their declared types. The conditions distinguish position and target roles, merge them into one quantity type, or represent all values as scalars. Each condition uses the same 30 search seeds. Each evaluation exhaustively tests the 20 unequal start/target pairs in $\{-2, -1, 0, 1, 2\}$, with at most four graph steps per episode. Strict run success requires solving all 20 pairs.

Table 1 shows a clear contribution from semantic role information: merging position and target types reduces success from 29 to 15 runs, and flattening all types reduces it to one. Initialization already solves 20, 4, and 0 runs, respectively; refinement supplies the remaining successes. Type declarations improve both the initial candidate distribution and the set of permitted replacements. The realized evaluation budgets differ because stopping conditions and available replacements differ. Appendix B reports these budgets, seed-level outcomes, and final-model audits. The original 10-seed experiment, which used a different evaluation protocol and obtained 8/10 versus 0/10, is retained separately in Appendix K.4.

Table 2: Execution consistency of the revised runtime. Graph-level replay uses frozen models; targeted tests separately exercise scheduling and persistence behavior.

Validation	Observed result
Models / model–initial-state pairs	90 / 1,800
Serial versus three-worker replay	All 4,934 paired steps match
Saved evaluation metrics	All accuracy and loss values match
Small mixed-resource graph	All legal schedules preserve state and reads
Independent-branch synchronization test	Confirms actual concurrent execution
Implementation regression suite	85 tests and 46 subtests pass

Table 3: Matched state encodings on the original executor. Records include repeated final graphs; timing summarizes 90 final models with five paired repetitions per model.

Comparison quantity	Result
Graph records / paired initial conditions	1,007 / 20,140
Paired environment steps	70,670
Numerical values, state, trajectories, metrics	All matched
Additional operation nodes	0
Mean additional state edges	2.189
Explicit/implicit time ratio: median [quartiles]	1.137 [1.086, 1.197]

4.4 Execution Consistency under Alternative Schedules

The revised executor preserves each evaluated candidate's selected state-access order through acceptance, copying, and model reloads. Structural edits produce candidates for evaluation; serial and threaded executions of a fixed candidate agree. We evaluate this at two levels. First, targeted tests exercise non-commuting writes, read/write conflicts, disjoint resources, unknown effects, and zero-output action nodes. Second, the 90 final models from the type experiment are replayed on all 20 initial conditions in serial and three-worker modes. Every trajectory and saved loss/accuracy value agrees, covering 1,800 model–initial-state pairs and 4,934 paired environment steps (Table 2).

The mixed-resource test enumerates every legal topological schedule of a six-operation graph containing independent resources and conflicting accesses. A separate randomized check compares inferred control dependencies against 150 resource-access configurations. The synchronization test uses a barrier inside independent methods: both workers must enter before either completes. This confirms that consistency is achieved while retaining available concurrency. For a chain of 250 methods with unknown effects, the plan requires 249 control edges, demonstrating the frontier construction's compact treatment of conservative ordering.

The persistence tests reorder stored nodes and edges and preserve stable node identities. Loaded programs retain their evaluated order and resource declarations. Training tests cover candidate acceptance, rollback, graph growth, and composite-method extraction. Together, the replay and targeted tests connect the execution contract in Section 3.2 to the paths used by training and deployment. Appendix A specifies the complete contract and the optional execution modes.

4.5 Implicit and Explicit State Encodings

For each of 1,007 candidate/final-model records, we translated implicit state accesses into immutable explicit-state inputs and outputs while preserving the selected access order. Across 20,140 paired initial conditions and 70,670 paired environment steps, node values, state transitions, trajectories, and final metrics agree. This establishes an operational correspondence between the two encodings for the evaluated programs.

The explicit encoding adds an average of 2.189 state edges over the 90 final graphs and no operation nodes. Its paired execution-time ratio relative to the implicit implementation has median 1.137, including state-object construction and adapter dispatch. Table 3 summarizes the correspondence.

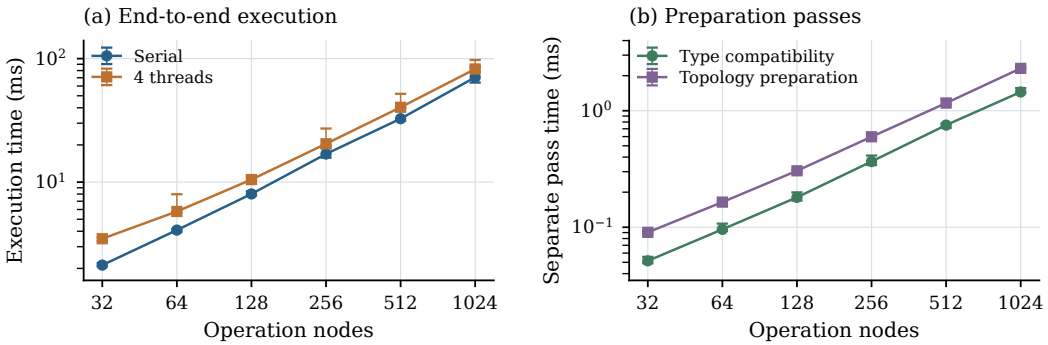


Figure 3: Fixed-depth scaling on the original backend. Medians summarize five graph seeds per size; error bars show their minimum–maximum range. Each metric uses two warm-ups and seven repetitions. The declared-type pass and topology preparation are measured separately; inference includes its own preparation. Full measurements are in Appendix D.

Table 4: Nested atomic-pool expansion. Numerical pass rate averages ten task families and four seeds; maze completion averages two seeds. Both values are percentages.

Operators	Numerical input pass rate	Maze completion
6	14.01	11.43
9	11.45	58.01
15	7.72	37.89
20	11.35	6.93

The translation gives every state-reading or state-writing operation an immutable (p, g) token input and output. Token edges follow the original access order, and a final edge returns the new state. Numerical argument binding and output aggregation are unchanged. During the explicit run, the unrelated implicit global state is deliberately altered; explicit outputs remain unchanged and that global state is not modified. This check verifies that state dependence has moved into the graph ports rather than remaining hidden in the method wrappers. The two implementations thus expose the same transition through different interfaces: shared external resources in one, explicit state values in the other. Appendix C details the translation and timing protocol.

4.6 Scaling Graphs, Pools, and Execution Workloads

Single-graph execution. At fixed depth eight and batch size 32, we vary width to obtain 32–1,024 operation nodes and 68–2,176 edges, with five graph seeds at each size. Serial time increases from 2.127 to 70.586 ms across this 32-fold node increase. At the largest size, a separate declared-type compatibility pass takes 1.448 ms and topological preparation takes 2.310 ms. Four-worker time is 82.379 ms: threading is slower for these Python scalar CPU operations. All 30 graphs agree with an independent numerical reference (maximum absolute error 2.22×10^{-16}). Figure 3 shows the measured curves; 1,024 nodes is the largest tested point, and the overhead-dominance threshold has not been reached in this sweep.

Atomic operator-pool expansion. A follow-on search implementation compares nine pools containing 6–20 operators across 12 numerical tasks and four action tasks. Pool expansion has a non-monotonic effect under fixed candidate and interaction budgets (Table 4). The six-operation arithmetic core gives the strongest aggregate numerical pass rate along the nested expansion; adding increment, decrement, and negation gives the strongest maze result. The different responses show that useful operator coverage and the distribution of candidates jointly shape search quality.

Numerical conditions use 131,072 training input presentations per task and seed, 256 candidate opportunities, and a 32-node graph cap. Action conditions use 65,536 training episodes, 128

Table 5: Library capacity in the 40-stage application. Costs count environment interactions and average ten paired seeds; lower is better. Return cost measures recovery of previously encountered rule families.

Configuration	Capacity	Mean attainment cost	Return cost
Direct reuse	32	1,792.32	1,749.76
Direct reuse	128	1,691.52	1,512.96
Full combination	32	1,855.04	1,802.88
Full combination	128	1,683.20	1,472.00

Table 6: Held-out action-task completion (%). Each trained model is evaluated on 2,048 episodes. Mean and range summarize five training seeds; individual results are in Appendix E.

Task	Completion criterion	Step cap	Mean	Seed range
Catch	Catch all 16 balls	160	92.10	61.28–100.00
Maze	Reach $(7, 7)$	1,024	92.74	75.98–98.88
Snake	Collect 8 food items	512	43.55	0.05–93.12
Bird	Pass all 32 pipes	1,024	100.00	100.00–100.00

candidate opportunities, and a 24-node cap. Numerical success in the table is the fraction of test inputs with relative error at most 1%. The extensions change operator composition and arity along with pool size. Appendix F gives the full nine-pool comparison and the path-feedback procedure. This experiment identifies a practical design choice: pool views should expose operations relevant to the task while controlling the alternatives presented to a finite-budget optimizer.

Sustained execution and reusable libraries. A continual-learning application runs 80 sequences of 40 stages across ten training seeds. Each observation has 64 fields, one of 32 identities, and six available actions. The sequence presents 20 rule families and then returns to them in reverse order. The completed runs record 9,830,400 valid training actions and 229,376,000 evaluation input rows: 239,206,400 input rows in total undergo native-ATF risk-output checks. These counts measure application workload; graph calls can process whole arrays.

Attainment cost counts interactions up to the earliest checkpoint after which balanced risk accuracy of at least 90% and optimal-action rate of at least 80% in the carrying condition are maintained. A stage that never attains this criterion receives a cost of 3,200. Increasing capacity from 32 to 128 lowers mean attainment cost by 5.62% for direct reuse and 9.26% for the full configuration (Table 5). The full configuration's return cost decreases by 18.35%, while its first-exposure cost changes little. Additional retained structures therefore help most when earlier tasks are revisited. At capacity 32, the full configuration does not meet its predeclared advantage criterion over direct reuse. A separate shorter capacity sweep shows no stable gain; its capacity-32 and capacity-64 runs produce identical trajectories because occupancy never exceeds 21 entries. The results distinguish retention capacity from the search and selection mechanism used with it. Appendices G and H provide all eight conditions and the short-sequence comparison.

4.7 A Learned Action-Scoring Controller

The structure in Figure 1 computes

$$z_t = \phi_G(o_t, h_t), \qquad q_{t,a} = \sum_j w_{a,j} z_{t,j}, \qquad a_t = \mathrm{route}(q_t, u_t). \tag{6}$$

The router uses a supplied draw u_t when its selection rule is stochastic. The template supplies shared features, per-action score branches, exclusive permits, zero-output action sinks, and state-update interfaces. CMA learns the score coefficients; scheduled structure proposals can add arithmetic features or change state-emission sources. Table 6 reports held-out completion across five training seeds, with a separate model for each task.

Bird succeeds consistently, while Snake exposes substantial training-seed variation. In the validation-selected representative system, Catch, Maze, and Snake have no subsequent feature-growth event;

Bird has four. Its 44 shared features supply 88 coefficients. For example, $z_{16} = \texttt{gapBottom} + h_1^{\text{old}}$ enters the flap and coast scores with weights approximately $+3.001$ and -6.538, directly illustrating how one state-dependent calculation serves competing action branches. Feature proposals occur at fixed training-budget milestones, so the contribution of optimized coefficients and the occurrence of structural growth can be inspected separately in the saved controller.

A frozen bird model selected by validation was adapted from 314 nodes/411 edges to 316 nodes/565 edges by making its input, state-commit, routing, and observation interfaces explicit. Running this adapted graph in the original ATF executor matches the source controller and an independent interpreter for 20,480 steps over 20 fixed test episodes, with zero difference in features, scores, state, and actions. The replay connects the learned controller to the original execution interface. Appendix E gives the five-seed results and adapter specification.

The adapter exposes a complete control step: feature and score nodes read the old state; a single commit node writes the next state; a completion dependency releases the router; the selected action sink records the action; and the environment advances after graph execution. Thus shared computation, state update, and action emission each have an explicit place in the program. The example shows how the representation carries a learned numerical controller while preserving its discrete execution structure.

LIMITATIONS

The stateful synthesis task isolates a supplied semantic type prior in a small environment. A remaining comparison is independent search with an explicit-state GP/CGP system under matched realized budgets. The larger applications use additional optimizers and task templates, and their performance depends on these choices. Effect ordering relies on accurate resource declarations; arbitrary Python side effects and runtime exceptions are outside static type compatibility. New-runtime preparation overhead requires its own performance measurements. The graph sweep has not located the practical upper bound or independently varied edge density. Python CPU threading is slower in the measured scalar workload, while the restricted static symbolic-regression benchmark favors specialized solvers on several formulas. These findings identify concrete next steps in search, backend engineering, and evaluation.

5 CONCLUSION

AdaptoFlux provides a common executable interface for typed dataflow, shared computation, and stateful actions. Persistent ordering resolves ambiguous shared-state access without changing method arguments. The experiments show that semantic type roles improve stateful synthesis, matched explicit and implicit encodings preserve program behavior, and execution cost remains measurable across substantially larger graphs and sustained application workloads. The learned action-scoring example connects these mechanisms in a controller whose behavior is reproduced by the original executor. The resulting substrate supports several optimization strategies while making their type, state, and execution contracts explicit.

Our ongoing research investigates how a single continually learning agent can use AdaptoFlux to acquire executable knowledge from interaction, retaining learned computational subgraphs together with their applicability conditions and supporting evidence for reuse across tasks. We aim to use unexpectedly beneficial outcomes and contradictory feedback to guide exploration and local graph revision, allowing knowledge to accumulate and adapt as task rules change within bounded memory and computation budgets.

AI USE STATEMENT

AI assistants supported the revision through implementation assistance, supplementary experiment scripting, result checking, literature lookup, English drafting and editing, and figure and layout preparation. Numerical results are derived from saved execution records and analysis scripts. The software checks include matched-state replay, independent numerical references, and regression tests. The authors are responsible for the paper's claims and the verification of AI-assisted material.

ETHICS STATEMENT

This work follows the ICLR Code of Ethics. It uses public benchmark data and synthetic tasks, involves no human-subject intervention, and reports no personally identifiable experimental data. Automatically generated programs can invoke external actions; applications should validate registered methods and isolate execution environments. Appendix T discusses interpretability, misuse, and bias inherited from supplied methods.

REPRODUCIBILITY STATEMENT

The appendices specify method pools, seeds, evaluation criteria, runtime versions, and measurement protocols. The supplementary type study retains final graphs and complete evaluation traces; the state audit retains paired trajectories; the scaling study retains per-seed repetitions and independent reference checks; and the controller case retains its frozen model and stepwise replay. The original implementation and experiment materials are linked through the anonymous repository. Additional evidence is identified by its protocol and artifact names in Appendices A–H; its execution records are kept separate from historical experiments. Data-graph type checks, effect-ordering assumptions, and finite-unrolling arguments are specified in Sections 3.2–3.4 and Appendix P.6.

REFERENCES

W. Ross Ashby. *Design for a Brain: The Origin of Adaptive Behaviour*. Springer Dordrecht, 2nd edition, 1960. doi: 10.1007/978-94-015-1320-3.

Miles Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582*, 2023.

Andrew Cropper and Rolf Morel. Learning programs by learning from failures. *Machine Learning*, 110(4):801–856, 2021.

Will Cukierski. Titanic - machine learning from disaster. `https://kaggle.com/competitions/titanic`, 2012. Kaggle.

Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*, pp. 835–850, 2021. doi: 10.1145/3453483.3454080. URL `https://doi.org/10.1145/3453483.3454080`.

Simon Harding, Vincent Graziano, Juergen Leitner, and Juergen Schmidhuber. MT-CGP: Mixed type cartesian genetic programming. In *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation*, pp. 751–758. ACM, 2012. doi: 10.1145/2330163.2330268.

Mikel Landajuela, Brenden K Petersen, Sookyung Kim, Claudio P Santiago, Ruben Glatt, Nathan Mundhenk, Jacob F Pettit, and Daniel Faissol. Discovering symbolic policies with deep reinforcement learning. In *International Conference on Machine Learning*, pp. 5979–5989. PMLR, 2021.

LLVM Project. The LLVM target-independent code generator: Introduction to SelectionDAGs. `https://llvm.org/docs/CodeGenerator.html#introduction-to-selectiondags`, n.d. Accessed November 2026.

Julian Miller and Andrew Turner. Cartesian genetic programming. In *Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation*, pp. 179–198, 2015. doi: 10.1145/2739482.2756571. URL https://dl.acm.org/doi/10.1145/2739482.2756571.

David J Montana. Strongly typed genetic programming. *Evolutionary computation*, 3(2):199–230, 1995.

Brenden K Petersen, Mikel Landajuela, T Nathan Mundhenk, Claudio P Santiago, Soo K Kim, and Joanne T Kim. Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. *arXiv preprint arXiv:1912.04871*, 2019.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

Silviu-Marian Udrescu and Max Tegmark. Ai feynman: A physics-inspired method for symbolic regression. *Science Advances*, 6(16):eaay2631, 2020. doi: 10.1126/sciadv.aay2631.

Norbert Wiener. Cybernetics. *Scientific American*, 179(5):14–19, 1948.

A State-Access Ordering and Execution Consistency

The revised executor makes the order of conflicting state accesses part of the stored program. This supplies the missing ordering information when two nodes have no data-dependency path but access the same external state. The original data edges continue to carry method arguments; additional control dependencies only determine when a node may execute.

Execution versions and evidence. We distinguish the original executor, denoted V0, from the revision with optional effect ordering, denoted V1. The three-level type experiment, matched state encoding experiment, CPU scaling measurement, and frozen-controller replay below were completed with V0 and frozen protocols. Their measurements remain associated with that implementation. V1 is evaluated separately by execution-consistency tests and frozen-model regression. The subsequent pool and continual-learning studies use their stated search procedures around the ATF representation and execution interfaces. This separation identifies which implementation produced each result while retaining the original experimental records.

Program order and conflict relation. For each method instance v, let R_v and W_v be its declared read and write resource sets. Two instances conflict when

$$W_u \cap (R_v \cup W_v) \neq \varnothing \quad \text{or} \quad W_v \cap (R_u \cup W_u) \neq \varnothing. \tag{7}$$

Before the first evaluation of a candidate, a topological order τ of its data graph is selected and stored using stable node identities. For each conflicting pair, control dependencies preserve its order in τ. The implementation maintains the latest writer and outstanding readers for each resource; transitive dependencies avoid storing every conflicting pair. Unknown effects form conservative barriers among effectful methods, and a wildcard resource denotes access to all resources. Read-only accesses and accesses to disjoint resources retain their available parallelism.

All data and control dependencies point forward in τ, so their union is acyclic. Assuming deterministic methods, complete resource declarations, and identical initial state and inputs, successful executions respecting this union produce the same outputs and final state. To see this, any two legal linear extensions are connected by adjacent swaps of incomparable nodes. Such nodes have neither a data dependency nor a conflicting state access, so each swap preserves their outputs and state updates. Repeating these swaps gives the claim. Resource declarations therefore concern actual shared objects, including random-number streams and external callbacks; type compatibility alone does not establish this property.

Counterexample and positive control. Starting at $s = 1$, two independent data branches that perform $s \leftarrow s + 1$ and $s \leftarrow 2s$ yield 4 or 3 under opposite topological orders. Declaring both branches as writers of the same resource introduces the chosen control order and preserves the corresponding result in serial and threaded execution. The test suite also uses synchronization barriers to verify that independent branches actually overlap in time. It exercises read/write conflicts, wildcard resources, zero-output actions, graph edits, storage reordering, and failure propagation.

Training and persistence. Candidates are prepared before evaluation. An accepted candidate retains the evaluated graph and its semantic order; method replacement preserves stable node identities. Graph growth, replacement, and subprogram extraction validate the resulting dependencies. Explicitly changing the order creates a new candidate to evaluate. Saved plans contain the order, ports, method signatures, and effect declarations; loading checks these contracts before replacing the live model. The plan stores execution semantics rather than a copy of the external environment.

Optional execution mode. V1 enables effect ordering by default and records the choice with the model. With ordering enabled, a stateful batch completes one sample's graph before the next sample begins. With ordering disabled, scheduling uses data edges alone and retains node-major batch processing. Both modes validate graph ports and preserve the chosen setting through training, cloning, and persistence. Switching modes changes the program's execution contract and calls for a new evaluation. The ordering guarantee applies to successful executions; external effects already performed before a method failure are not rolled back.

Table 7: Type resolution under the frozen V0 search protocol. Wilson intervals describe the run-level success proportion. Evaluation counts are means per run.

Type condition	Successes	95% interval	Evaluations	Distinct graphs
Fine	29/30	83.3–99.4%	59.47	6.87
Coarse	15/30	33.2–66.8%	81.67	5.60
Flat	1/30	0.6–16.7%	738.67	22.77

Frozen-model regression. All 90 final models from the type experiment were replayed in V1 on their 20 initial conditions. Across 1,800 model–initial-state pairs and 4,934 paired environment steps, serial and three-worker execution matched in trajectories, accuracy, and loss, including the saved V0 outcomes. The associated implementation suite passes 85 tests and 46 subtests. These results validate the integrated execution path without repeating model training.

B Effect of Type Resolution on Stateful Search

Separating the roles of position and target produces the strongest search result in the finite attractor task: 29 of 30 runs solve every initial condition. Merging those roles lowers success to 15 of 30, and using scalar types throughout lowers it to 1 of 30. The experiment directly exposes the contribution of the supplied type prior.

Protocol. The three conditions share the nine original method implementations, arities, groups, weights, and registration order. Fine types distinguish `pos_type` and `target_type`. Coarse types merge both into `quantity`, which remains distinct from `scalar`. Flat types use `scalar` throughout. Every condition retains the goal-aware movement operator. Formal seeds are 110600–110629; pilot seed 110500 is excluded. Python and NumPy random generators are reset to the run seed for each condition.

GraphEvo uses three fixed-mode initial models, three initialization layers, 30 refinement steps per cycle, five maximum evolution cycles, the full candidate pool, and no method-pool evolution. Early stopping is enabled at zero loss. Every evaluation exhaustively visits the 20 ordered pairs $(p, g) \in \{-2, -1, 0, 1, 2\}^2$ with $p \neq g$, resetting the external position and target for each episode. A graph receives at most four executions to reach the target. Loss is mean absolute final distance; a successful run reaches the target from all 20 initial states. Evaluation performs no random sampling and checks that it leaves the search random-number states unchanged.

Search progression and accounting. Initialization alone succeeds in 20, 4, and 0 runs, respectively. Subsequent refinement raises these totals to 29, 15, and 1. Mean final accuracies are 0.9667, 0.5033, and 0.3200, with mean losses 0.0667, 0.9900, and 1.0667. Nominal caps are shared; early stopping and the available replacements produce the different actual evaluation budgets in Table 7. "Distinct graph" means an ordered attributed-graph signature, with literal display names removed; it is an accounting unit rather than a functional equivalence class.

The V0 early-stop path exits before updating some refinement counters and the best-model snapshot. We therefore evaluate the saved final graph and record every evaluation callback. Reconstructing accepted replacements from these callbacks gives totals of 9, 13, and 19, compared with native logged counters of 0, 1, and 18. This bookkeeping correction changes neither the trained graphs nor their measured outcomes. The earlier ten-seed experiment in Appendix K.4 uses a different episode-sampling protocol and remains a separate experiment. The present result establishes improved search success from a role-sensitive type prior in this fully enumerated task.

C Matched Explicit and Implicit State Encodings

Explicit state threading and implicit external-state access reproduce the same program behavior when they preserve the same state-access order. We verified this correspondence across all exported records from the type experiment, including unsuccessful candidates.

Table 8: Matched state encoding results using V0. Ratios include the common adapter interface and state-token construction/extraction.

Quantity	Result
Candidate/final-model records	1,007
Paired initial conditions	20,140
Paired environment steps	70,670
Native graph executions, both encodings	141,340
Numerical, state, trajectory, and metric comparisons	All matched
Mean additional state edges, final graphs	2.189
Median explicit/implicit elapsed-time ratio	1.137
Ratio interquartile range	1.086–1.197

Construction. For each run and condition, the candidate catalogue contains the first 20 distinct ordered graph signatures encountered, where available, plus the final graph even when already recorded. The resulting 1,007 records include repeated final graphs. Every state-reading or state-writing method is translated to consume and return an immutable (p, g) state token. Token edges follow the original serial state-access order, including reads. Each access adds one state input/output port and one state edge; a final edge returns the resulting state. No computation node is added. Numerical argument binding and output aggregation remain unchanged.

Execution comparison. Both encodings use the original native serial executor on the same 20 initial conditions and four-step episode limit. Checks compare per-node numerical outputs, state after each access, collapsed output, each environment transition, and the recorded final loss and accuracy. During explicit execution, the global implicit state is deliberately assigned unrelated values, which verifies that the token-based methods neither read nor modify it.

Timing. Each of the 90 final graphs receives one complete warm-up and five paired timing repetitions over all 20 episodes; the order of the two encodings is randomized within each pair. The median ratio is computed per model and then summarized across models. It measures this Python object-port implementation, including state construction, dispatch, and extraction. Both conditions use the same already-searched candidates, so the experiment isolates encoding and execution; independent search over explicit-state programs is a separate comparison.

D Controlled Single-Graph Execution Scaling

At fixed depth and batch size, the native executor processes graphs from 32 to 1,024 operations with consistent numerical results. Over this 32-fold increase in operation count, median serial time increases from 2.127 to 70.586 ms. Type compatibility and topological preparation remain small relative to graph execution throughout the measured range.

Graph and workload design. Graphs have eight layers of width $w \in \{4, 8, 16, 32, 64, 128\}$, four input features, and batch size 32. There are $8w$ operation nodes, two boundary nodes, and $2(8w) + w$ edges. Each operation has two inputs. Its first predecessor slot uses a permutation of the preceding layer, ensuring every earlier node contributes to an output, while the second slot selects a distinct predecessor. The first layer selects two of the four input features. Final-layer outputs are aggregated with the native average operation.

Each scalar operation starts from $z = 0.6a + 0.4b$ and performs 16 iterations of $z \leftarrow 0.98z + 0.02\tanh(0.9z + 0.05)$. This is ordinary CPU arithmetic, with vectorization disabled. Five independently wired graphs are evaluated at each width. Two disjoint pilot graphs precede the frozen formal protocol. The input seed is 2026110600, and formal graph seeds at width w are $51000 + 1000w + i$, $i \in \{1, \ldots, 5\}$.

Measurement and validation. Each metric receives two warm-ups and seven timed calls, with shuffled case and metric order. Timing uses a monotonic high-resolution clock without memory tracing. The environment is Windows, Python 3.12.9, NumPy 2.1.3, and NetworkX 3.4.2, on a

Table 9: V0 execution scaling, in milliseconds for batch size 32. Values are medians over five graph-level timing medians. Speedup is the median of paired serial/threaded ratios, so it need not equal the ratio of the displayed medians.

Operations	Edges	Type pass	Topology	Serial	4 threads	Speedup
32	68	0.051	0.090	2.127	3.478	0.610
64	136	0.096	0.164	4.086	5.782	0.697
128	272	0.181	0.305	8.022	10.482	0.754
256	544	0.367	0.598	16.894	20.436	0.805
512	1,088	0.753	1.163	32.621	40.360	0.808
1,024	2,176	1.448	2.310	70.586	82.379	0.811

16-core/24-thread Intel host. Serial timing includes topological sorting; threaded timing includes dependency bookkeeping and thread-pool creation/destruction. The separate topology column is not deducted from either. The type pass is an additional per-edge measurement calling the native type compatibility predicate on declared ports, including the final aggregation contract. It is distinct from an integrated whole-program compiler check.

All 30 graphs satisfy the specified counts, depth, acyclicity, input slots, and declared type compatibility. A deliberately incompatible signature is rejected. Both execution paths agree with an independently implemented NumPy reference at tolerance 10^{-12}; maximum observed reference error is 2.22×10^{-16}. Separately traced allocation peaks exclude pre-existing graphs and inputs, and measure Python allocations rather than total resident memory. At 1,024 operations, the median serial and threaded peaks are 3.60 and 5.33 MB, respectively.

Observed scaling regime. Four Python threads are slower than serial execution for this scalar CPU workload. Together with the earlier simulated I/O experiment, the measurements show how operator cost and GIL behavior determine whether available graph parallelism improves latency. The largest tested graph has 1,024 operations; the point at which preparation dominates has not been reached. Node and edge counts grow together at fixed arity, so this experiment measures a controlled width expansion rather than an independent edge-density sweep.

E Shared-Feature Graphs for Action Selection

A shared-feature graph provides a concrete connection between numerical computation, persistent state, and discrete actions. One feature can contribute to several action scores while a mutually exclusive routing stage selects a zero-output action node. The same organization supports the four evaluated task families.

Controller representation and optimization. For observation o_t and previous internal state h_t, the graph computes

$$z_t = \phi_G(o_t, h_t), \qquad q_{t,a} = \sum_j w_{a,j} z_{t,j}, \qquad a_t = \mathrm{route}(q_t, u_t), \tag{8}$$

where u_t supplies any stochastic routing draw. Feature calculations read the old state, and a single commit stage writes the state used on the next step. The shared configuration has four storage slots arranged as two independent state channels. A decay factor of 0.9 and clipping are part of its specified update rule.

The training procedure provides the shared-feature interface, per-action score branches, route selection, and state-update rule. CMA optimizes the score coefficients. Configurations permitting growth propose eight additional arithmetic features at each of the prescribed 15%, 35%, 55%, and 75% budget milestones and retarget state-emission sources. Thus learned coefficients and proposed features sit within a declared controller template. Models are selected by validation success, with progress breaking ties. Each task uses its own selected model; the task identity is supplied externally.

The bird results are consistent across seeds, while Snake shows substantial variation. In the validation-selected representative system (seed 104), Catch, Maze, and Snake have no subsequent feature-growth

Table 10: Held-out completion rates (%) of the selected controller portfolio. Each cell uses 2,048 episodes; all five training seeds are shown. The mean row averages training seeds within each task.

Training seed	Catch	Maze	Snake	Bird
101	99.22	96.09	91.65	100.00
102	61.28	75.98	0.78	100.00
103	100.00	98.88	32.18	100.00
104	100.00	96.24	93.12	100.00
105	100.00	96.53	0.05	100.00
Mean	92.10	92.74	43.55	100.00

event; Bird has four. Its controller contains 44 features and 88 coefficients. One saved feature is $z_{16} = \mathrm{gapBottom} + h_{1,t}$ and contributes with coefficients about $+3.001$ and -6.538 to the flap and coast scores. This is a direct example of shared numerical computation feeding distinct action branches.

Execution on the original ATF backend. The frozen representative Bird graph has 314 nodes and 411 edges. An explicit adapter expands the input, constant-trigger, state-commit, and observation interfaces, giving 316 nodes and 565 edges. Every feature operation, score coefficient, product, and accumulation is represented by graph operations. The commit stage sends a completion token to the router; enabled zero-output action sinks append action receipts to the step context, and the environment advances after graph execution.

We replay the first 20 consecutive episodes of the pre-existing test stream (environment seed 200000104; episode indices starting at 83000000), using the same external random draw in all executors. Across 20,480 steps, V0 serial execution agrees exactly in features, scores, state, actions, and trajectories with the portable interpreter and the original kernel function body. A further 2,048 of these steps match the previously archived trace. All 49 boundary probes and their 49 insertion-order-reordered counterparts pass. This replay establishes executable transfer of the frozen controller through the stated adapter; the performance results in Table 10 come from the preceding training and held-out evaluation.

F Sensitivity to the Atomic Method Pool

Expanding the atomic pool changes search quality non-monotonically. The best aggregate numerical result in this comparison comes from the six-operation core, while the best maze result comes from the nine-operation extension. The comparison covers nine pools spanning 6–20 operations.

Pools and search protocol. The core contains identity, addition, subtraction, multiplication, safe division, and duplication. The nested extensions add increment, decrement, and negation (9 operations); square, ReLU, absolute value, three-input mean, maximum, and minimum (15); then sine, cosine, two-argument arctangent, scaling, and safe square root (20). Five further pools vary polynomial, reciprocal, and trigonometric combinations. All use the same path-feedback procedure: each distinct executed route receives one terminal-loss update per episode, with exponential averaging coefficient 0.08, temperature 0.12, and exploration fraction 0.2. Candidates grow from identity scaffolds with periodic extension.

Numerical tasks use 131,072 training input presentations per task and seed, sampled from 2,048 fixed training examples, 256 candidate opportunities, and a 32-node limit per graph. Twelve tasks are grouped into ten equally weighted families, averaging the three equivalent two-variable multiplication tasks before aggregation. Four training seeds are used. Testing uses 1,024 inputs; the reported pass rate is the fraction with relative error at most 1%. Action tasks use 65,536 training episodes per task and seed, 128 candidate opportunities, a 24-node graph limit, and two seeds. Checkpoint evaluation uses 512 episodes per route-execution mode. Candidate generation, pool composition, and path feedback define this subsequent search host.

The numerical comparison comprises 432 records, including 32 reused baseline records; the action comparison comprises 144 newly trained runs. The broader 928-run study additionally includes

Table 11: Atomic-pool sensitivity. Numerical values average ten task families and four seeds. Maze values average two seeds under within-episode route switching. Both columns are percentages with their own evaluation units.

Pool	Operations	Numerical input pass rate	Maze completion
Arithmetic	6	14.01	11.43
Square	7	11.47	17.97
Basic	9	11.45	58.01
Nonlinear	15	7.72	37.89
Extended	20	11.35	6.93
Algebra	8	11.32	12.89
Polynomial	9	8.57	31.84
Balanced	12	13.62	27.44
General	16	10.83	14.16

mechanism ablations, so that total is not the number of pool-size comparisons. More operations offer new expressions while increasing the alternatives explored within the same nominal budget. Because operator identities and arities change with the extensions, the table describes practical pool sensitivity. Numerical input pass rates and action completion rates are reported separately, preserving their distinct meanings.

G Large Continual-Learning Execution Workload

The ATF execution core also supports a sustained workload with 80 complete learning sequences and 3,200 stages. Across training and checkpoint evaluation, 239,206,400 input rows are checked against native ATF outputs. This measures the substrate's use inside a long-running application, complementing the single-graph measurements in Appendix D.

Environment and design. Each observation has 64 fields, one of 32 identities, and six available actions. The 40-stage sequence first presents 20 deterministic rule families and then returns to them in reverse order. Fields and identities are assigned by seed. Training uses 3,072 interactions per stage, at most 2,048 candidates per search refresh, 6,144 stored observations, and a 48-derived-node cap per candidate graph. Ten seeds are used: 41113, 41213, 41317, 41423, 41527, 41631, 41741, 41847, 41957, and 42061.

Six mechanisms are compared at concept-library capacity 32: direct reuse, small-set retrieval, retrieval with prediction history, reserved fresh search, reward/complexity selection, and their full combination. Direct reuse and the full combination are additionally run at capacity 128, yielding 80 sequences. Paired conditions share observations, exploration draws, action sets, and nominal training and candidate limits. ATF executes the risk-expression graph, identity lookup, and action-value computation. Candidate proposals, selection, and library management are supplied by the surrounding learner.

Counting execution. The completed runs contain 9,830,400 effective training actions and 229,376,000 checkpoint-evaluation rows. Their sum is the 239,206,400 native-checked input rows reported above. One graph call can receive a NumPy array containing many rows, and the action-value head can perform a matrix computation within one node. Consequently, input rows, graph calls, concept entries, and graph nodes are distinct scale measures. The independent training units remain the ten seeds.

Learning outcomes. Learning cost is the earliest checkpoint after which both balanced risk accuracy of at least 90% and optimal-action rate of at least 80% in the carrying condition are maintained. Unsuccessful stages receive the fixed cost 3,200. A rule/first-or-return configuration counts as mastered when at least eight of ten seeds meet the criterion, giving 40 configurations in total.

The prespecified advantage criterion for the full combination at capacity 32 requires at least eight of ten seeds to reduce cost by at least 10%, without increasing average training loss. It is not met: zero seeds attain the cost threshold, mean cost increases by 3.50%, and loss increases by approximately

Table 12: Continual-learning results across ten paired seeds. Lower cost and training loss are better. Loss is accumulated reward shortfall relative to an oracle action over the 40 stages.

Mechanism (capacity)	Mean cost	Return cost	Training loss	Mastered/40
Direct reuse (32)	1,792.32	1,749.76	62,809.12	21
Small-set retrieval (32)	1,777.60	1,762.56	62,942.44	21
Retrieval + history (32)	1,720.32	1,648.00	62,617.83	23
Reserved fresh search (32)	1,817.92	1,739.52	63,083.45	21
Reward/complexity (32)	1,800.64	1,767.68	62,681.52	21
Full combination (32)	1,855.04	1,802.88	63,203.22	20
Direct reuse (128)	1,691.52	1,512.96	61,926.77	21
Full combination (128)	1,683.20	1,472.00	61,921.29	21

Table 13: Short-sequence library capacity comparison. The actual peak counts show whether the configured limit was reached. All capacities master 14 of the 15 sequence-stage configurations under the frozen criterion.

Capacity	Transfer cost	Training loss	Mastered/15	Peak entries
8	226.13	2,993.39	14	8
16	238.93	3,000.39	14	16
32	234.67	2,997.98	14	21
64	234.67	2,997.98	14	21

0.63%. Retrieval with history has the lowest mean cost among the capacity-32 conditions. Higher-order parity remains difficult: the three- and five-input XOR rules have approximately chance-level risk accuracy, with no seed reaching the mastery criterion on first or return exposure.

Completion audit. An initial 16 MiB serialized-state guard stopped part of the experiment. The formal completion uniformly raises that guard to 256 MiB while retaining the training, candidate, and concept budgets. Fifty-two completed sequences are preserved with identical file hashes; the remaining 28 start from their initial conditions. Three discarded prefixes account for 343,296 additional actions, recorded separately from the effective training count. Final frozen replay passes for all 80 models. Peak serialized state for the full combination is approximately 4.67 MiB at capacity 32 and 18.67 MiB at capacity 128.

H Reusable-Library Capacity and Return to Earlier Tasks

Larger libraries help most when earlier tasks must be recovered after a longer sequence. In the 40-stage experiment, increasing capacity from 32 to 128 lowers mean cost by 5.62% for direct reuse and 9.26% for the full combination. For the full combination, return-stage cost drops from 1,802.88 to 1,472.00 (18.35%), whereas first-exposure cost changes from 1,907.20 to 1,894.40. At the same capacity 128, the full combination and direct reuse are close: their mean cost difference is -8.32, with an exploratory paired-bootstrap 95% interval of $[-103.68, 87.68]$. The larger-storage result therefore concerns capacity and reuse over time.

Paired short-sequence comparison. A complementary experiment changes only the library capacity to 8, 16, 32, or 64, retaining the learner, tasks, admission and replacement rules, search budgets, and five training seeds. Three five-stage relationship sequences per seed give 60 sequences, 300 stages, and 460,800 interactions. Transfer cost averages the new-identity and return stages; a failure receives the fixed cost 1,600. This is a paired capacity sensitivity test on the same seeds.

Increasing capacity in these short sequences produces no stable learning gain. All three larger-capacity conditions have zero of five seeds achieving the prespecified 20% transfer-cost improvement. Capacities 32 and 64 generate identical models, learning curves, libraries, and action trajectories in all 15 paired sequences because occupancy never exceeds 21 entries. Meanwhile, the mean final single-model derived-node count is 15.87, 15.20, 15.19, and 15.19. Library capacity controls retained

reusable structures; the final selected graph can remain small even when the collection of stored structures grows.

Evidence organization. The four V0 supplement records are organized under the relative artifact directories `experiments/rebuttal_20261106/type_search`, `state_encoding`, `scaling`, and `action_example`; each contains its protocol, source-version record, and measurements. The subsequent studies are identified by their controller portfolio, atomic-pool sensitivity, 40-stage continual-learning, and short-sequence capacity protocols. Their source snapshots and row-count audits distinguish training, evaluation, and diagnostic records. The results above use only completed formal records; planned extensions and withdrawn records do not enter the tables.

ORIGINAL EXPERIMENTS AND TECHNICAL DETAILS

I DETAILED RELATED WORK AND POSITIONING

Typed graph representations. Cartesian Genetic Programming (CGP) represents programs as graphs with reusable intermediate values Miller & Turner (2015). Mixed Type CGP (MT-CGP) extends this representation to multiple data types Harding et al. (2012); typed program graphs are therefore an established foundation. Strongly typed GP Montana (1995) likewise uses declared interfaces to constrain valid compositions. AdaptoFlux builds on these ideas with a common registration and execution interface for heterogeneous host-language callables, including external-state operations, together with interchangeable search procedures and reusable graph modules.

Data dependencies and effect ordering. LLVM's SelectionDAG explicitly distinguishes typed data values from chain dependencies that order side-effecting operations such as memory accesses and calls LLVM Project (n.d.). The separation of data flow from effect order is established compiler practice. AdaptoFlux applies this principle to an executable symbolic-synthesis substrate: data edges bind callable arguments, while a saved execution plan orders declared conflicting resource accesses. Its operators may invoke application environments rather than target-machine instructions. The implementation maintains the execution contract through candidate evaluation, graph edits, extraction, and model persistence (Appendix A).

Functional abstraction and explicit state. DreamCoder Ellis et al. (2021) learns reusable abstractions within a functional program space. Mutable interaction can also be expressed functionally by passing state values through a program. AdaptoFlux offers an interface in which registered methods access external state directly. The paired translation in Appendix C makes the relation concrete: explicit state edges and implicit accesses reproduce the same tested trajectories when their access order agrees. This is a choice of programming interface and execution contract, rather than a separation in computability. DreamCoder's library-learning method and AdaptoFlux's graph-extraction procedure address related reuse goals with different search mechanisms.

Constraint-guided synthesis. Popper Cropper & Morel (2021) learns logical programs using a generate–test–constrain loop. Declared types in AdaptoFlux similarly restrict which compositions a search examines. Popper additionally derives constraints from failed hypotheses, whereas the current AdaptoFlux filters are supplied through type declarations and search metadata. Logical relations, explicit state arguments, and external-state callables provide different interfaces for representing transitions. The comparisons concern these interfaces and their execution, rather than an impossibility of expressing transitions in a logical language.

Symbolic regression and policy search. PySR Cranmer (2023), gplearn and AI Feynman Udrescu & Tegmark (2020) provide specialized procedures for recovering numerical expressions. Risk-seeking policy-gradient methods learn distributions over symbolic expressions Petersen et al. (2019), including control policies Landajuela et al. (2021), while language-model-guided discovery combines proposals with systematic evaluation Romera-Paredes et al. (2024). These methods emphasize search quality. AdaptoFlux supplies operator registration, graph construction and execution facilities that can host different optimizers; the CMA-trained controller in Appendix E illustrates this separation. The static comparisons in Appendix M report the behavior of the tested restricted method pools and probes.

State feedback and contribution. Repeated evaluation connects an acyclic stored graph to a changing environment. This organization is compatible with classical feedback systems Wiener (1948); Ashby (1960) and with explicit state-threading representations. The contribution is an integrated, inspectable substrate combining heterogeneous typed callables, persistent state-order contracts, graph reuse, and optimizer-independent evaluation. Its empirical support spans controlled type-prior comparisons, paired state encodings, structured action programs and execution-scale measurements.

J Complete Derivation of AdaptoFlux Inference and Training Complexity

Implementation version. The cost decomposition below describes the original execution backend used by the historical experiments and the controlled measurements in Appendix D. The revised executor additionally prepares a persistent semantic order and effect-control dependencies (Appendix A). Its preparation cost depends on the graph and declared resource accesses; the original timing measurements do not include that new pass. Scheduling operates on the union of data and control dependencies, and available parallelism is determined by this augmented graph.

This appendix provides a detailed derivation of the time and space complexity of the AdaptoFlux architecture during single inference, task-parallel inference, and under three main training strategies (LayerGrow, GraphEvo, CombinedTrainer).

J.1 Symbol Definitions and Basic Assumptions

To ensure consistency in the derivation, we first uniformly define general symbols, complexity constants, and basic assumptions.

J.1.1 General Symbol Table

Table 14: General Symbols

Symbol	Meaning
N	Inference batch size (number of samples)
V	Number of computation graph nodes (excluding `root` and `collapse`)
E	Number of computation graph edges (explicit data dependencies)
D_{in}	Input feature dimension
D_{out}	Output feature dimension
W	Average graph width (maximum number of concurrent nodes)
L_{crit}	Critical path length (longest data dependency chain)
P	Number of parallel scheduling threads/processes

Note: The above parameters are highly dependent on specific task configurations, data scales, and graph topologies. The framework design does not make fixed prior assumptions about them.

J.1.2 Complexity Constant Dependency Characteristics

Table 15: Complexity Constants and Dependencies

Constant	Meaning and Dependency
C_{copy}	Python object reference extraction overhead. Fixed at $O(1)$ (nanosecond-level memory pointer operations), independent of graph scale.
C_{sync}	Global lock acquisition/release and queue synchronization overhead. Fixed at $O(1)$ (microsecond-level OS primitives), potentially becoming a serial bottleneck under high concurrency.
$\bar{C}_{\text{op}}$	**Average method computational complexity** (where op denotes the underlying **operation**). **Strongly depends on method pool composition**: Pure Python loops are constrained by interpreter overhead; C/Numba extensions can release the GIL to achieve true parallelism. This constant directly determines the absolute performance boundary of the framework.
ρ_{lock}	Global lock contention factor. Dynamic range $[1/P, 1]$, actual value depends on graph width W, data flow topology, and thread scheduling strategy.

J.1.3 BASIC ASSUMPTIONS

Assumption 1 [Typical Deployment Scenario]
In typical deployment scenarios, the following conditions are met:

1. $N \gg V, E$ (Batch size is much larger than graph scale)
2. $D_{\text{in}}, D_{\text{out}}$ are constants or independent of N
3. $C_{\text{copy}}, C_{\text{sync}}$ are $O(1)$ constants
4. Method library size M is a bounded constant
5. Search hyperparameters (L, A, C, R, S, f) are bounded constants

Assumption 2 [Operator Implementation]
Operator implementations satisfy:

1. Pure Python operators: $\bar{C}_{\text{op}}$ includes significant interpreter overhead
2. I/O or C extension operators: Can release GIL, supporting true parallelism

J.2 INFERENCE PHASE COMPLEXITY ANALYSIS

J.2.1 SINGLE INFERENCE: FOUR-STAGE DECOMPOSITION

Definition [Single Inference]
Single inference refers to the process where the model, in deployment state, performs forward propagation on a given input batch to generate prediction results. In the AdaptoFlux architecture, the inference process is modeled as data flow execution on a Directed Acyclic Graph (DAG).

Stage 1: Input Normalization and Preprocessing The system first converts input data into an internal sample list structure. The complexity of this step depends on the input data type:

- **Dense Numerical Matrix**: Involves memory copying and type checking, with complexity $O(N \cdot D_{\text{in}})$
- **Complex Objects (dtype=object)**: Main overhead lies in reference counting and list construction, still maintaining a linear relationship $O(N \cdot D_{\text{in}})$

Stage 2: Graph Topological Sorting To ensure correct node execution order dependencies, the system performs topological sorting on the computation graph. This operation depends only on the graph structure and is independent of the sample size N:

$$T_{\text{topo}} = O(V + E) \tag{9}$$

Stage 3: Node Layer-by-Layer Execution This is the core computational part of the inference process. The system traverses each node $v \in \mathcal{V}$ in topological order. For each node, the following sub-operations are performed:

1. **Edge Data Extraction**: Based on incoming edge configurations, feature data for N samples is extracted from predecessor node outputs. The total number of incoming edges in the graph is E, and the data extraction complexity is:

$$T_{\text{extract}} = O(E \cdot N \cdot C_{\text{copy}}) \tag{10}$$

2. **Operator Computation**: Executes the computation method bound to the node. Let the single-sample computational complexity of node v be $C_{\text{op}}(v)$. If the method supports vectorization, $C_{\text{op}}(v)$ approaches the efficiency of underlying kernel optimization; if it degrades to sample-by-sample loops, $C_{\text{op}}(v)$ includes significant Python interpreter overhead.
   The total complexity of the node execution phase is:

$$T_{\text{compute}} = O\left(N \cdot \sum_{v \in \mathcal{V}} C_{\text{op}}(v)\right) = O(N \cdot V \cdot \bar{C}_{\text{op}}) \tag{11}$$

Stage 4: Result Aggregation and Collapse The final stage of inference maps the outputs of specified nodes to global coordinates and performs collapse processing. This process requires traversing the final feature vectors of N samples:

$$T_{\text{collapse}} = O(N \cdot D_{\text{out}}) \tag{12}$$

Total Complexity Derivation Combining the four stages, the total time complexity of AdaptoFlux single inference is:

$$\begin{aligned} T_{\text{infer}}(N, V, E) &= T_{\text{input}} + T_{\text{topo}} + T_{\text{extract}} + T_{\text{compute}} + T_{\text{collapse}} \\ &= O(N \cdot D_{\text{in}}) + O(V + E) + O(E \cdot N \cdot C_{\text{copy}}) \\ &\quad + O(N \cdot V \cdot \bar{C}_{\text{op}}) + O(N \cdot D_{\text{out}}) \\ &= O\left(N \cdot (D_{\text{in}} + E \cdot C_{\text{copy}} + V \cdot \bar{C}_{\text{op}} + D_{\text{out}}) + V + E\right) \end{aligned} \tag{13}$$

Dominant Term Simplification
In actual deployment, typically $N \gg V, E$ and $D_{\text{in}}, D_{\text{out}}$ are constants. At this point, inference time is primarily dominated by operator computation:

$$T_{\text{infer}} \approx O(N \cdot V \cdot \bar{C}_{\text{op}}) \tag{14}$$

The framework scheduling overhead of AdaptoFlux grows **linearly** with model structural complexity V, but the total time strongly depends on the implementation efficiency of specific operators (e.g., whether vectorized, whether GIL is released).

Space Complexity and Implementation Residue *Remark [Full Intermediate Result Cache]* The current implementation retains a full intermediate result caching strategy (`node_outputs` stores all node outputs), with space complexity:

$$S_{\text{cache}} = O(N \cdot V \cdot S_{\text{data}}) \tag{15}$$

where S_{data} is the average size of intermediate features.

This mechanism is a functional residue from early architecture design: initially intended to support optimization algorithms requiring access to intermediate activations (such as architecture search), but in the current pure inference version, these intermediate data are not actually utilized nor returned externally. This leads to unnecessary memory occupation. Future plans include introducing a release strategy, i.e., releasing a node's output after all its successor computations are completed.

J.2.2 Task-Parallel Inference: Six-Step Execution Flow Decomposition

In task-parallel scenarios, the following extended symbols are introduced:

- P: Number of thread pool worker threads (`num_workers`)
- C_{sync}: Synchronization overhead for global lock acquisition/release, dictionary updates, and queue operations
- $L_{\text{crit}} = \sum_{v \in \mathcal{P}_{\text{crit}}} C_{\text{op}}(v)$: Cumulative computational cost of the Critical Path
- $\rho_{\text{lock}} \in [1, P]$: Global lock contention factor
  - Low load: $\rho_{\text{lock}} \approx 1/P$ (ideal parallelism)
  - High concurrency contention: $\rho_{\text{lock}} \to 1$ (fully serial)

Based on the actual execution trajectory of `infer_with_task_parallel`, the time complexity can be derived step-by-step:

Step 1: Input Normalization and Graph Initialization (Main Thread) Data format conversion and construction of `in_degree_remaining` and `ready_queue` depend only on N and graph structure:

$$T_1 = O(N \cdot D_{\text{in}} + V + E) \tag{16}$$

Step 2: Main Thread Dynamic Scheduler The `while` loop executes V times `queue.get` and `executor.submit`:

$$T_2 = O(V) \tag{17}$$

This part is completely serial.

Step 3: Lock-Protected Data Extraction (Serialization Bottleneck) The first `with lock:` block in the code wraps incoming edge traversal and feature extraction:

```
extracted = [sample_output[output_idx] for sample_output in src_output]
```

The total number of incoming edges in the graph is E, and each edge requires traversing N samples to generate a reference list. The total cost of this operation is $O(E \cdot N \cdot C_{\text{copy}})$. Since it is protected by a global mutex lock, **all threads execute this phase serially**. Considering lock contention, the actual time consumption is:

$$T_3 = O(E \cdot N \cdot C_{\text{copy}} \cdot \rho_{\text{lock}}) \tag{18}$$

Step 4: Lock-Free Operator Computation (Parallel Acceleration Zone) After data extraction, the lock is released, and threads independently execute `func` calls. The total computational workload is:

$$W_{\text{comp}} = N \cdot \sum_{v \in \mathcal{V}} C_{\text{op}}(v) \tag{19}$$

According to DAG scheduling theory (Brent's Theorem), the actual time consumption under P threads is limited by the critical path and computational share:

$$T_4 = O\left(N \cdot \max\left(L_{\text{crit}}, \frac{1}{P}\sum_{v \in \mathcal{V}} C_{\text{op}}(v)\right)\right) \tag{20}$$

Note: If an operator is marked `vectorized=True`, $C_{\text{op}}(v)$ is significantly reduced, but large N may trigger CPU cache and memory bandwidth contention. Moreover, due to framework characteristics, vectorization generally struggles to offset scheduling overhead.

Step 5: Lock-Protected Status Update and Successor Triggering The second `with lock:` block executes `node_outputs` writing and in-degree decrementing. The graph triggers V lock acquisitions and E queue pushes in total:

$$T_5 = O(V \cdot C_{\text{sync}} + E \cdot C_{\text{sync}}) \tag{21}$$

This part is also serialized, but $C_{\text{sync}} \ll C_{\text{copy}}$, so it is usually negligible.

Step 6: Result Aggregation (Collapse) (Main Thread) After `f.result()` blocking ends, the main thread sequentially collects `collapse` incoming edge data, sorts, and calls the collapse function:

$$T_6 = O(N \cdot D_{\text{out}}) \tag{22}$$

Total Complexity Formula Combining the six stages, the precise time complexity of task-parallel inference is:

$$\begin{aligned} T_{\text{parallel}}(N, V, E, P) &= \sum_{i=1}^{6} T_i \\ &= O\Big(N \cdot D_{\text{in}} + V + E \\ &\quad + E \cdot N \cdot C_{\text{copy}} \cdot \rho_{\text{lock}} \\ &\quad + N \cdot \max\left(L_{\text{crit}}, \frac{1}{P}\sum_{v \in \mathcal{V}} C_{\text{op}}(v)\right) \\ &\quad + V \cdot C_{\text{sync}} + N \cdot D_{\text{out}}\Big) \end{aligned} \tag{23}$$

Typical Scenario Simplification
In typical deployment scenarios ($N \gg V, E$, and $C_{\text{copy}}, C_{\text{sync}}$ are constants), the dominant terms simplify to:

$$T_{\text{parallel}} \approx O\left(N \cdot \max\left(L_{\text{crit}}, \frac{1}{P}\sum_{v \in \mathcal{V}} C_{\text{op}}(v)\right) + E \cdot N \cdot C_{\text{copy}} \cdot \rho_{\text{lock}} + N \cdot D_{\text{out}}\right) \tag{24}$$

J.2.3 Engineering Constraint Analysis

Remark [Lock Granularity Defect]
The current implementation places data extraction ($O(E \cdot N)$) within the global lock, causing this part to be unparallelizable. When E is large or graph width is high, $\rho_{\text{lock}} \to 1$, and increasing P may reduce throughput due to lock contention.

Remark [GIL Limitation]
`ThreadPoolExecutor` is constrained by the CPython Global Interpreter Lock. Pure Python intensive operators can only achieve concurrency, not true parallelism.

Amdahl's Law Analysis: Actual speedup satisfies

$$S(P) \leq \frac{1}{\alpha + (1-\alpha)/P} \tag{25}$$

where α is the proportion of serial execution within the lock.

Remark [Coupling of Vectorization and Thread Count]
If nodes have enabled vectorization, a single thread already approaches the hardware computational upper limit. At this point, $P > 1$ will cause resource oversubscription, leading to L1/L2 cache thrashing and memory bandwidth saturation.

Remark [Vectorization]
Due to the framework's weak support for vectorization and the need for significant scheduling overhead, operators with vectorization enabled often fail to bring performance improvements and may even degrade performance.

Optimal Thread Count Estimation:

$$P^* \approx \frac{\text{Hardware Physical Core Count}}{\text{Average Vectorization Parallelism}} \tag{26}$$

J.3 Detailed Analysis of Navigation Probe (Training Strategy) Complexity

Note: Main Text §3.5 states the final complexity results and their interpretation; this appendix provides the complete derivations, symbol tables, and constant-factor discussions.

J.3.1 LayerGrow Strategy: Layered Generate-Evaluate-Rollback

Symbol Definition

Precise Derivation of Candidate Generation Complexity C_{gen} `process_random_method` is responsible for randomly matching compatible methods for each output slot and grouping them. Its core steps include:

1. **Grouping D slots by data type**: $O(D)$
2. **Constructing "Type → Method Input" reverse index**: Traversing I_{avg} inputs of M methods, $O(M \cdot I_{\text{avg}})$
3. **Matching compatible method inputs for each slot**: Worst-case traversal of all method inputs, $O(D \cdot M \cdot I_{\text{avg}})$
4. **Grouping by method call signature and post-processing**: $O(D)$

Table 16: LayerGrow Symbols

Symbol	Meaning
L	Maximum added layers (`max_layers`)
A	Maximum attempts per layer (`max_attempts`)
N	Number of input samples for evaluation
V_i	Number of graph nodes at iteration i, $V_i = V_0 + i \cdot k$
V_0	Initial number of nodes
k	Average new nodes per layer
$T_{\text{infer}}(N, V_i)$	Single inference complexity, $O(N \cdot V_i \cdot \bar{C}_{\text{op}})$
C_{gen}	Candidate generation complexity
C_{struct}	Graph structure modification (add/remove layer) complexity, $O(k)$
C_{copy}	Model snapshot deep copy complexity, $O(V_i + E_i + M_i)$
C_{eval}	Evaluation function computation complexity (loss/accuracy)
D	Number of edges pointing to `collapse` node (number of output slots)
M	Total number of methods registered in the method library
I_{avg}	Average number of input slots per method
$\bar{C}_{\text{op}}$	Average operator computational complexity (including vectorization efficiency factor)
p_{accept}	Candidate acceptance probability

Therefore, the worst-case complexity of candidate generation is:

$$C_{\text{gen}} = O(D \cdot M \cdot I_{\text{avg}}) \tag{27}$$

Clearly, C_{gen} grows linearly with method library size M and input complexity I_{avg}, and is usually small compared to the inference complexity $T_{\text{infer}}(N, V_i)$ of the evaluation phase. In actual implementation, although the constant factor of C_{gen} is large, since T_{infer} involves complete forward propagation, typically:

$$C_{\text{gen}} \ll T_{\text{infer}}(N, V_i) \tag{28}$$

i.e., **candidate generation overhead is negligible, and the evaluation phase dominates the time consumption of a single attempt**.

Core Overhead of a Single Attempt Each candidate attempt (inside the `attempt` loop) contains the following sequential operations:

1. **Candidate Generation**: Call `process_random_method()`, complexity $O(C_{\text{gen}})$
2. **Graph Structure Modification**: Call `append_nx_layer()` to temporarily add a new layer, complexity $O(C_{\text{struct}})$
3. **Model Inference**: Execute one complete forward propagation to get model output, complexity $T_{\text{infer}}(N, V_i)$
4. **Evaluation Function Calculation**: Call loss/accuracy evaluator, complexity C_{eval}
5. **Decision and Rollback**: If candidate is rejected, call `remove_last_nx_layer()` to restore graph structure, complexity $O(C_{\text{struct}})$

Single Attempt Complexity:

$$\begin{aligned} T_{\text{attempt}}(i) &= O(C_{\text{gen}} + C_{\text{struct}} + T_{\text{infer}}(N, V_i) + C_{\text{eval}}) \\ &\approx O(T_{\text{infer}}(N, V_i)) \quad (\text{since } T_{\text{infer}} \gg C_{\text{gen}}, C_{\text{struct}}, C_{\text{eval}}) \\ &= O(N \cdot V_i \cdot \bar{C}_{\text{op}}) \end{aligned} \tag{29}$$

Configurability of Evaluation Function *Remark [Hardcoded Evaluation]*
In the current implementation, `_evaluate_loss` and `_evaluate_accuracy` are hardcoded to execute by default, which is technical debt from early design. Subsequent optimizers do not necessarily need to compute both loss and accuracy simultaneously, and some scenarios (such as reinforcement learning) may not require accuracy evaluation at all. Benefiting from AdaptoFlux's plugin-style evaluator design, theoretically supports:

1. **Skipping Redundant Evaluation**: If only loss signal is needed, accuracy calculation can be disabled, reducing C_{eval} overhead by 50%
2. **Environment Signal Reuse**: In scenarios like reinforcement learning, reward signals can be obtained directly from the environment, where $C_{\text{eval}} \approx O(1)$
3. **Delayed Evaluation**: Execute full evaluation only after candidate acceptance, calculating lightweight proxy metrics only during the attempt phase

Expected Overhead of Single Layer Growth For the i-th layer ($0 \leq i < L$), the algorithm attempts at most A times. Let the acceptance probability be p_{accept} (dependent on data and candidate space), then the expected number of attempts is $1/p_{\text{accept}}$. In the worst case ($p_{\text{accept}} \to 0$), full A attempts are executed:

$$T_{\text{layer}}(i) = O(A \cdot T_{\text{infer}}(N, V_i)) = O(A \cdot N \cdot (V_0 + i \cdot k) \cdot \bar{C}_{\text{op}}) \tag{30}$$

Total Training Complexity Derivation Summing over all L layers and considering graph scale grows linearly with layers:

$$\begin{aligned} T_{\text{LayerGrow}} &= \sum_{i=0}^{L-1} T_{\text{layer}}(i) \\ &= O\left(A \cdot N \cdot \bar{C}_{\text{op}} \cdot \sum_{i=0}^{L-1} (V_0 + i \cdot k)\right) \\ &= O\left(A \cdot N \cdot \bar{C}_{\text{op}} \cdot (L \cdot V_0 + k \cdot \frac{L(L-1)}{2})\right) \end{aligned} \tag{31}$$

Dominant Term Analysis: When the initial graph is small ($V_0 \ll k \cdot L$) and k is constant, the dominant term is quadratic:

$$T_{\text{LayerGrow}} = O(A \cdot N \cdot \bar{C}_{\text{op}} \cdot k \cdot L^2) \tag{32}$$

i.e., quadratic complexity with respect to maximum layers L, and linear with respect to attempt count A, evaluation batch N, and operator complexity $\bar{C}_{\text{op}}$.

Impact of Sparsely Triggered Operations

- **Best Model Snapshot**: Executed `copy.deepcopy()` only when a better model is found, frequency usually much lower than $L \cdot A$. Let trigger count be S ($S \ll L \cdot A$), then extra overhead is $O(S \cdot (V_{\text{best}} + E_{\text{best}} + M_{\text{best}}))$, usually negligible.
- **Rollback Operation**: When `on_retry_exhausted="rollback"`, multiple layers may be rolled back continuously. In the worst case, L layers are rolled back, but total rollback count is constrained by global attempt limit `max_total_attempts`, not changing the dominant complexity order.
- **Early Stopping Mechanism**: If `enable_early_stop=True` and model converges quickly, actual executed layers $L_{\text{actual}} \ll L$, significantly reducing actual runtime, but not changing worst-case complexity.

Space Complexity

- **Graph Structure Storage**: $O(V_L + E_L)$, where $V_L = V_0 + L \cdot k$, grows linearly with layers
- **Best Model Snapshot**: $O(V_{\text{best}} + E_{\text{best}} + M_{\text{best}})$, proportional to best model scale
- **Attempt History Record**: $O(L \cdot A)$ metadata records, each $O(1)$, total overhead $O(L \cdot A)$

Total Space Complexity:

$$S_{\text{LayerGrow}} = O(V_0 + L \cdot k + E_0 + L \cdot e + M_{\text{best}}) \tag{33}$$

where e is the average number of new edges per layer.

Table 17: GraphEvo Symbols

Symbol	Meaning
N	Number of input samples for evaluation (mini-batch)
V_i, E_i	Graph nodes and edges at iteration i, $V_i = V_0 + i \cdot k$
C	Maximum evolution cycles (`max_evo_cycles`)
I	Number of diverse initialization candidate models (`num_initial_models`)
L_{init}	Random layers for each initialization candidate (`max_init_layers`)
R	Maximum refinement steps per cycle (`max_refinement_steps`)
S	Number of snapshots required to trigger evolution (`evolution_trigger_count`)
f	Snapshot recording frequency (`evolution_sampling_frequency`)
M	Total methods registered in method pool
I_{avg}	Average input slots per method
$\bar{C}_{\text{op}}$	Average operator computational complexity
V_{sub}	Average nodes of subgraph processed in evolution phase

J.3.2 GraphEvo Strategy: Four-Stage Closed-Loop Optimization

Symbol Definition

Phase 1: Diverse Initialization This phase generates I random initial graphs and selects the best one:

1. For each candidate, randomly add L_{init} layers: each layer requires calling `process_random_method` ($O(D \cdot M \cdot I_{\text{avg}})$) and `append_nx_layer` ($O(k)$)
2. Perform one complete evaluation for each candidate: $T_{\text{infer}}(N, V_0 + L_{\text{init}} \cdot k)$
3. Select best candidate: $O(I)$

Initialization Phase Complexity:

$$\begin{aligned} T_{\text{init}} &= O\left(I \cdot [L_{\text{init}} \cdot (D \cdot M \cdot I_{\text{avg}} + k) + T_{\text{infer}}(N, V_0 + L_{\text{init}} \cdot k)]\right) \\ &\approx O(I \cdot L_{\text{init}} \cdot N \cdot V_{\text{init}} \cdot \bar{C}_{\text{op}}) \quad \text{(since } T_{\text{infer}} \text{ dominates)} \end{aligned} \tag{34}$$

where $V_{\text{init}} = V_0 + L_{\text{init}} \cdot k$.

Phase 2: Node-wise Refinement This phase performs local search optimization within C evolution cycles. It must be clarified that **GraphEvo's refinement process does not change the graph topology**, only greedily replacing methods bound to existing nodes, so graph scale V and E remain constant at initialized values V_{init} and E_{init} during refinement.

At most R refinement steps are executed per cycle

Each refinement step includes:

1. Select optimizable node: $O(V_{\text{init}})$ (traverse non-frozen nodes)
2. Get compatible method list: $O(M \cdot I_{\text{avg}})$ (type matching), can be reduced to $O(1)$ if hash index is enabled
3. Attempt method replacement: $O(1)$ (only update node method pointer, no add/delete edges)
4. Evaluate new structure: $2 \cdot T_{\text{infer}}(N, V_{\text{init}})$ (loss + accuracy)
5. Decision and rollback: $O(1)$ (restore method pointer)

Single Step Refinement Complexity:

$$T_{\text{refine}}^{(\text{step})} = O(M \cdot I_{\text{avg}} + T_{\text{infer}}(N, V_{\text{init}})) \approx O(T_{\text{infer}}(N, V_{\text{init}})) \tag{35}$$

(since $T_{\text{infer}} \gg M \cdot I_{\text{avg}}$)

Single Cycle Refinement Complexity: Since topology is fixed, cumulative complexity of R refinement steps is:

$$T_{\text{refine}}^{(\text{cycle})} = O\left(\sum_{j=1}^{R} T_{\text{infer}}(N, V_{\text{init}})\right) = O\left(R \cdot N \cdot V_{\text{init}} \cdot \bar{C}_{\text{op}}\right) \tag{36}$$

Total Refinement Complexity: Summing over all C cycles, graph scale remains unchanged:

$$T_{\text{refine}}^{\text{total}} = O\left(C \cdot R \cdot N \cdot V_{\text{init}} \cdot \bar{C}_{\text{op}}\right) \tag{37}$$

i.e., refinement phase is linear with respect to cycle count C and step count R, reflecting the significant pruning effect of the topology-fixed strategy on the combinatorial search space.

Phase 3: Method Pool Evolution (Optional) This phase abstracts new methods based on historical snapshots, triggered sparsely:

1. **Snapshot Recording**: Record once every f cycles, total $\lfloor C/f \rfloor$ times, each deep copy $O(V + E + M)$
2. **Evolution Trigger**: Trigger after accumulating S snapshots, total trigger count $T_{\text{evo}} = \lfloor \frac{C}{f \cdot S} \rfloor$
3. **Single Evolution Execution**:
   - Subgraph extraction and alignment: $O(S \cdot V \cdot E)$
   - Graph isomorphism clustering (if enabled): $O(S^2 \cdot V^3)$ (worst case, using VF2 algorithm)
   - New method abstraction and registration: $O(V_{\text{sub}} \cdot M)$

Total Evolution Phase Complexity:

$$\begin{aligned} T_{\text{evolve}} &= O\left(\frac{C}{f} \cdot (V + E + M) + \frac{C}{f \cdot S} \cdot (S^2 \cdot V^3 + V_{\text{sub}} \cdot M)\right) \\ &\approx O\left(\frac{C \cdot S \cdot V^3}{f}\right) \quad \text{(dominant term)} \end{aligned} \tag{38}$$

Cubic complexity with respect to graph scale V, but low trigger frequency ($S \geq 3$, $f \geq 1$), actual overhead is controllable.

Phase 4: Modular Compression (Design Residue, Optional Post-processing) *Remark [Technical Debt Note]*
The modular compression function (`_phase_modular_compression`) in current code is a residue from early architecture design technical debt:

- **Not involved in main training loop**: Compression is executed only as a one-time post-processing after training ends
- **Function deprecated**: Disabled by default (`enable_compression=False`), and numerical compression mode marked as `UNFINISHED`
- **Configurable extension point**: Interface retained to allow users to customize compression algorithms

If user explicitly enables symbolic compression and matches K_{comp} subgraphs, its single execution complexity is:

$$T_{\text{compress}} = O\left(K_{\text{comp}} \cdot (V_{\text{sub}} \cdot E_{\text{sub}} + N \cdot V_{\text{sub}}) + V + E\right) \tag{39}$$

Since $K_{\text{comp}} \ll V$ and executed only once, T_{compress} impact on total training time is negligible.

Total Training Complexity Combining the first three phases (ignoring optional compression), the worst-case time complexity of GraphEvo is:

$$T_{\text{GraphEvo}} = O(\underbrace{I \cdot L_{\text{init}} \cdot N \cdot V_{\text{init}} \cdot \bar{C}_{\text{op}}}_{\text{Initialization}} + \underbrace{C \cdot R \cdot N \cdot V_{\text{init}} \cdot \bar{C}_{\text{op}}}_{\text{Refinement}} + \underbrace{\frac{C \cdot S \cdot V_{\text{init}}^3}{f}}_{\text{Evolution}}) \tag{40}$$

Typical Configuration Simplification: In typical configurations ($I, L_{\text{init}}, R, S, f$ are bounded constants, $V \ll N$), the dominant term is $O(N \cdot V_{\text{init}} \cdot \bar{C}_{\text{op}} \cdot (I \cdot L_{\text{init}} + C \cdot R))$ from initialization and refinement phases. If evolution is enabled, graph scale is large, and graph isomorphism clustering is enabled, then $O(C \cdot V_{\text{init}}^3)$ may become a bottleneck.

Impact of Sparse Triggering and Early Stopping

- **Early Stopping Mechanism**: If `enable_early_stop=True` and model converges quickly, actual cycles $C_{\text{actual}} \ll C$, reducing C term to C_{actual}, significantly reducing runtime
- **Evolution Sparse Triggering**: Evolution phase triggers only after accumulating S snapshots, and $S \geq 3$, $f \geq 1$, so actual trigger count $T_{\text{evo}} \leq C/3$, avoiding high-overhead graph isomorphism clustering every cycle
- **Refinement Step Limit**: `max_total_refinement_attempts` can globally limit evaluation count, preventing computational explosion due to excessive R

Space Complexity

- **Graph Structure Storage**: $O(V + E)$, grows linearly with refinement
- **Method Library Storage**: $O(M \cdot I_{\text{avg}})$
- **Snapshot Cache**: $O(S \cdot (V + E + M))$, S is trigger threshold, usually $S \ll C$
- **Intermediate Result Cache**: $O(N \cdot V \cdot S_{\text{data}})$ (if enabled, technical debt)

Total Space Complexity:

$$S_{\text{GraphEvo}} = O(S \cdot (V + E + M) + N \cdot V \cdot S_{\text{data}}) \tag{41}$$

If intermediate cache is disabled and S is bounded, space overhead is linear with respect to V.

J.3.3 CombinedTrainer Strategy: Closed-Loop Combinatorial Optimization

Table 18: CombinedTrainer Extended Symbols

Symbol	Meaning
C_{outer}	Outer evolution cycles (`num_evolution_cycles`)
L, A	LayerGrow max layers and attempts per layer
C_{ge}, R	GraphEvo internal cycles and refinement steps per cycle
G, T_{ga}	Genetic algorithm population size and evolution generations (if enabled)
f_{gen}	Genetic screening trigger interval (`genetic_interval`)
V_{new}	New nodes added per layer growth, $V_{\text{new}} = L \cdot k$
V_{old}	Historical node count
V_{max}	Final graph scale, $V_{\text{max}} = V_0 + C_{\text{outer}} \cdot L \cdot k$

Extended Symbol Definition

Sub-Trainer Complexity Review

- **LayerGrow**: $T_{\text{LG}} = O(A \cdot N \cdot \bar{C}_{\text{op}} \cdot k \cdot L^2)$
- **GraphEvo**: $T_{\text{GE}} = O(C_{\text{ge}} \cdot R \cdot N \cdot V_{\text{init}} \cdot \bar{C}_{\text{op}} + \frac{C_{\text{ge}} \cdot S \cdot V_{\text{init}}^3}{f})$
- **Genetic Screening**: $T_{\text{GA}} = O(G \cdot T_{\text{ga}} \cdot L_{\text{ga}} \cdot A_{\text{ga}} \cdot N \cdot V \cdot \bar{C}_{\text{op}})$

CombinedTrainer Execution Flow and Complexity Decomposition **1. Initial Genetic Screening (Optional)**
If `genetic_mode` is `"once"` or `"periodic"`, execute genetic screening once before the loop:

$$T_{\text{init_ga}} = O(G \cdot T_{\text{ga}} \cdot L_{\text{ga}} \cdot A_{\text{ga}} \cdot N \cdot V_0 \cdot \bar{C}_{\text{op}}) \tag{42}$$

This step is executed only once, usually $G, T_{\text{ga}}, L_{\text{ga}}, A_{\text{ga}}$ are bounded constants, so $T_{\text{init_ga}} = O(N \cdot V_0 \cdot \bar{C}_{\text{op}})$, negligible.

2. Outer Evolution Loop (C_{outer} times)
Each loop serially executes LayerGrow and GraphEvo:

- **LayerGrow Phase**: Extend L layers starting from current graph, complexity $T_{\text{LG}}^{(i)} = O(A \cdot N \cdot \bar{C}_{\text{op}} \cdot k \cdot L^2)$
- **GraphEvo Phase**: Execute refinement starting from LayerGrow output. **Key Constraint**: This phase only performs node method replacement, graph topology $V_{\text{current}}^{(i)}$ remains unchanged. Therefore its complexity is linear with respect to cycles C_{ge} and steps R. If `refine_only_new_layers` is enabled, search space is further restricted to new nodes ($V_{\text{new}} = L \cdot k$):

$$T_{\text{GE}}^{(i)} = \begin{cases} O(C_{\text{ge}} \cdot R \cdot N \cdot V_{\text{current}}^{(i)} \cdot \bar{C}_{\text{op}} + \frac{C_{\text{ge}} \cdot S \cdot (V_{\text{current}}^{(i)})^3}{f}) & \text{Full Graph Refinement} \\ O(C_{\text{ge}} \cdot R \cdot N \cdot V_{\text{new}} \cdot \bar{C}_{\text{op}} + \frac{C_{\text{ge}} \cdot S \cdot V_{\text{new}}^3}{f}) & \text{New Nodes Only} \end{cases} \tag{43}$$

Single Cycle Complexity: $T_{\text{cycle}}^{(i)} = T_{\text{LG}}^{(i)} + T_{\text{GE}}^{(i)}$

3. Periodic Genetic Screening (Optional)
If `genetic_mode == "periodic"`, execute genetic screening every f_{gen} rounds, total trigger count $\lfloor C_{\text{outer}}/f_{\text{gen}} \rfloor$:

$$T_{\text{periodic_ga}} = O\left(\frac{C_{\text{outer}}}{f_{\text{gen}}} \cdot G \cdot T_{\text{ga}} \cdot L_{\text{ga}} \cdot A_{\text{ga}} \cdot N \cdot V_{\text{mid}} \cdot \bar{C}_{\text{op}}\right) \tag{44}$$

where V_{mid} is the average graph scale at trigger time.

Total Training Complexity Combining the above phases, the worst-case time complexity of CombinedTrainer is:

$$\begin{aligned} T_{\text{Combined}} = O(& \underbrace{C_{\text{outer}} \cdot A \cdot N \cdot \bar{C}_{\text{op}} \cdot k \cdot L^2}_{\text{LayerGrow Accumulation}} \\ & + \underbrace{C_{\text{outer}} \cdot \left(C_{\text{ge}} \cdot R \cdot N \cdot V_{\max} \cdot \bar{C}_{\text{op}} + \frac{C_{\text{ge}} \cdot S \cdot V_{\max}^3}{f}\right)}_{\text{GraphEvo Accumulation (Linear Refinement)}} \\ & + \underbrace{\frac{C_{\text{outer}}}{f_{\text{gen}}} \cdot G \cdot T_{\text{ga}} \cdot L_{\text{ga}} \cdot A_{\text{ga}} \cdot N \cdot V_{\text{mid}} \cdot \bar{C}_{\text{op}}}_{\text{Periodic Genetic}}) \end{aligned} \tag{45}$$

Typical Configuration Simplification: In typical configurations ($A, L, C_{\text{ge}}, R, S, f, G, T_{\text{ga}}, f_{\text{gen}}$ are bounded constants, $V \ll N$):

- If genetic screening is disabled: $T_{\text{Combined}} = O(C_{\text{outer}} \cdot N \cdot \bar{C}_{\text{op}} \cdot (A \cdot k \cdot L^2 + R \cdot V_{\max}))$

- If genetic is enabled and f_{gen} is large: Genetic term is lower-order, dominant terms remain growth and refinement phases
- If `refine_only_new_layers` is enabled: GraphEvo's V^3 term reduces to $(L \cdot k)^3$, $V_{\max}$ replaced by V_{new}, significantly reducing overhead at high graph scales

Impact of Sparse Triggering and Early Stopping

- **Global Early Stopping**: If `enable_early_stop=True` and model converges quickly, actual cycles $C_{\text{actual}} \ll C_{\text{outer}}$, reducing all C_{outer} related terms to C_{actual}, overall complexity decreases linearly
- **Genetic Sparse Triggering**: Genetic screening triggers only at f_{gen} intervals, and G, T_{ga} are usually small, so $T_{\text{periodic_ga}}$ usually does not dominate total complexity
- **New Node Refinement Optimization**: `refine_only_new_layers` reduces GraphEvo's search space from $O(V^3)$ to $O((L \cdot k)^3)$, and since topology is fixed, cycle complexity strictly reduces from $O(C_{\text{ge}}^2 R^2)$ to $O(C_{\text{ge}} R)$, significant effect when C_{outer} is large

Space Complexity

- **Graph Structure Storage**: $O(V_{\max} + E_{\max})$, grows linearly with outer loop
- **Method Library Storage**: $O(M \cdot I_{\text{avg}})$, genetic screening may reduce M
- **Genetic Population Cache**: $O(G \cdot (V + E + M))$ (if enabled), G usually bounded
- **Sub-Trainer Intermediate State**: $O(N \cdot V \cdot S_{\text{data}})$ (technical debt, recommended to disable during search phase)
- **Logs and Snapshots**: $O(C_{\text{outer}} \cdot (V + E))$ for recording results of each cycle

Total Space Complexity:

$$S_{\text{Combined}} = O(V_{\max} + E_{\max} + M \cdot I_{\text{avg}} + G \cdot (V + E + M) + N \cdot V \cdot S_{\text{data}}) \tag{46}$$

J.4 Engineering Constraints and Technical Debt Notes

Major Constraints of Current Implementation

1. **Full Intermediate Result Cache**: Leads to space complexity $O(N \cdot V \cdot S_{\text{data}})$
2. **Global Lock Granularity**: Data extraction placed within lock, causing parallel bottleneck, recommended to switch to fine-grained locks or lock-free structures
3. **GIL Limitation**: `ThreadPoolExecutor` cannot break through CPython Global Interpreter Lock, pure Python operators can only achieve concurrency, future migration to C++ or process pools may be needed for true parallelism
4. **Hardcoded Evaluation**: Loss and accuracy evaluation are hardcoded
5. **Modular Compression Residue**: `_phase_modular_compression` function is deprecated but retained in code

K Experimental Details

Compute Resources. All experiments were executed on standard workstations with multi-core CPUs and 60GB RAM. No GPU was used. Specifically, the parallel scalability experiments utilized a server with 32 CPU cores to evaluate multi-threading performance. The total wall-clock time to reproduce all main experimental results sequentially is estimated to be less than one week.

K.1 Experiment 1 Configuration and Full Statistical Results (LayerGrow on Titanic)

***Positioning**: This appendix summarizes the **minimal configuration set** and **full statistical results** required to reproduce Experiment 1. For experimental objectives, core findings, and framework-level conclusions, please refer to Main Text § 4.1.*

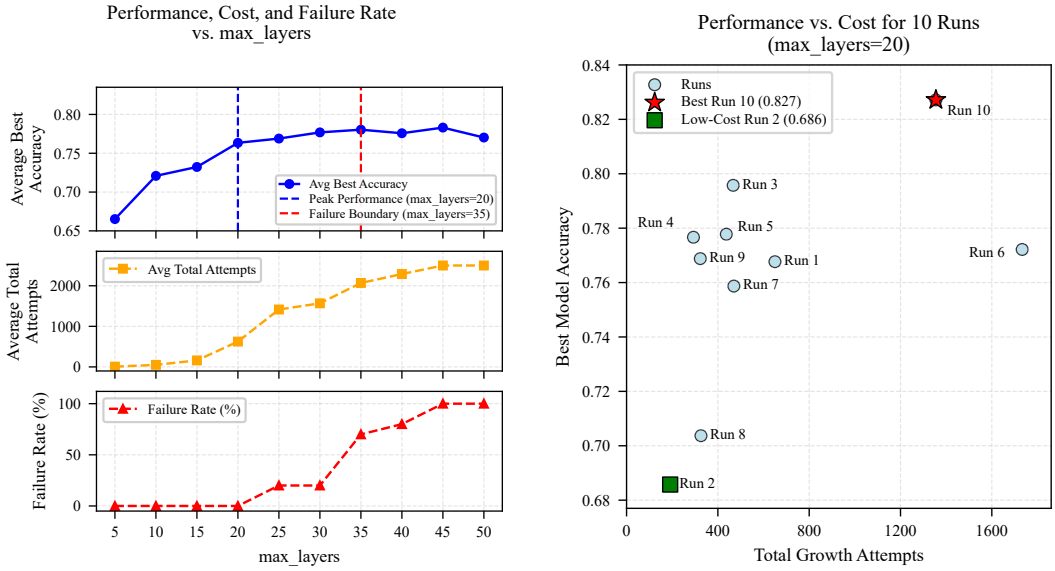


Figure 4: Original LayerGrow study. Left: performance, cost and attempt-budget exhaustion versus the configured maximum depth. Right: variation among ten runs with a maximum of twenty layers. Configured depth differs from realized depth.

K.1.1 Dataset and Task

Table 19: Dataset and Task Configuration

Item	Configuration
Dataset	Titanic Dataset. Only the training set is used for validation, stratified by `Survived` into 80% training / 20% testing.
Task Type	Binary Classification. *Note:* This is a benchmark for structural exploration capability, **not** for generalization performance evaluation. It aims to reveal saturation and failure boundaries of search strategies in discrete space.
Evaluation Metric	Test set accuracy (`best_model_accuracy`).

K.1.2 Method Pool Configuration (9 Scalar Operations)

```
# All methods have scalar input/output types, supporting type-safe
    routing
[
    "return_value",           # Identity mapping
    "return_two_values",      # Single input, dual output
    "add_values",             # Addition
    "calculate_difference",   # Subtraction
    "multiply_values",        # Multiplication
    "divide_values",          # Division (zero-protected)
    "increment",              # +1
    "decrement",              # -1
    "negate_value"            # Negation
]
```

Design Principles: *Type consistency + Low-dimensional data flow + Determinism. Excluding semantic heterogeneity interference to focus on validating the navigation capability of the LayerGrow probe in pure structural combinatorial space.*

K.1.3 HYPERPARAMETER GRID

Table 20: Hyperparameter Settings

Parameter	**Value/Description**
`max_layers`	`[5, 10, 15, 20, 25, 30, 35, 40, 45, 50]`
`max_attempts` (per layer)	10
Rollback layers	2
`decision_threshold`	0.0
Max Total Attempts	**2500** (Forced termination upon reaching limit is defined as "failure" and included in failure rate statistics)
Independent Repetitions	10 times/configuration
Random Seed	Base 42, expanded per run: $\text{seed} = 42r + \text{hash}(\texttt{max_layers}_L) \bmod 1000$ ($r \in [1, 10]$)

K.1.4 FULL STATISTICAL RESULTS

Table 21 reports detailed statistical metrics for each `max_layers` configuration. The data corresponds to the trend curves in Fig. 4.

- **Performance Saturation Point**: Mean best accuracy saturates at `max_layers=20` (0.763 ± 0.041, 0% failure); for larger budgets it fluctuates only within statistical noise (0.769–0.783 at 25–50 layers) while the attempt-budget exhaustion rate climbs toward 100%.
- **Attempt-budget exhaustion**: At configured maxima of 25 and 30 layers, 20% of runs exhaust the 2,500-attempt budget; at maxima of 45 and 50, all runs do. Their mean realized depths are only 19.6 and 18.7 layers, respectively (Table 21). These results locate a bottleneck of the tested growth strategy under a fixed attempt budget, rather than a failure to execute 45- or 50-layer graphs.
- **Path Dependence**: Even at the optimal configuration (20 layers), the standard deviation reaches 0.041, reflecting the sensitivity of discrete search to initial conditions.
- **Aggregation Rule**: All Avg Best Acc values are the mean, over the 10 independent runs, of the best model observed during each run—including runs terminated by the 2,500-attempt cap (failures)—consistent with the reporting convention in Main Text §4.1.

`max_layers`	Avg Best Acc	Std Dev	Avg Actual Layers	Avg Total Attempts	Failure Rate (%)
5	0.665	0.042	5.0	6.5	0.0
10	0.721	0.030	10.0	50.5	0.0
15	0.732	0.036	15.0	160.5	0.0
20	**0.763**	**0.041**	**20.0**	**624.5**	**0.0**
25	0.769	0.028	22.9	1414.9	20.0
30	0.777	0.025	26.3	1567.4	20.0
35	0.780	0.022	21.7	2069.2	70.0
40	0.776	0.020	23.3	2289.7	80.0
45	0.783	0.024	19.6	2500.0	100.0
50	0.770	0.032	18.7	2500.0	100.0

Table 21: Statistical results under different maximum layer settings. Failure rate is defined as the proportion of runs where `total_growth_attempts=2500`. The bolded row indicates the optimal balance point between performance and efficiency.

K.1.5 SUPPLEMENTARY NOTES AND REPRODUCTION GUIDELINES

- **Path Dependence Analysis Data**: The scatter plot in the right panel of Fig. 4 is generated from the per-run summary `experiments/LayerGrowTrainer_exp1/experiment_results_summary.csv` (shipped; columns `run`, `best_model_accuracy`, `total_growth_attempts`) by `generate_experiment_figures.py`; no separate plot-data CSV is maintained.

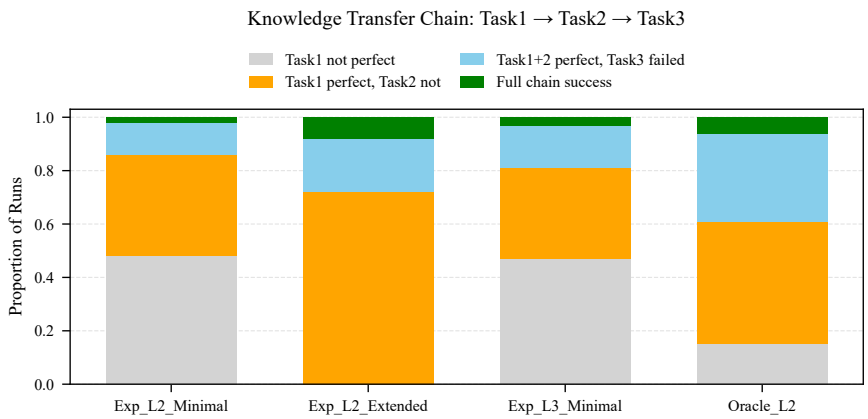


Figure 5: Original three-task reuse experiment: the green fraction denotes runs completing the full knowledge-transfer chain.

- **Complete Logs**: Candidate methods, rollback events and per-attempt accuracy are recorded for every run in `exp_max_layers_<L>/run_<r>/training_log.json` (100 files, ≈216 MB in total; keys `attempt_history` and `rollback_events`). They are regenerated by `main.py` and are not shipped, for size.
- **Reproduction Script**: All configurations and statistics are reproduced by: `experiments/LayerGrowTrainer_exp1/main.py`.

K.2 Experiment 2 Configuration Summary (Knowledge Transfer Validation)

***Positioning**: This appendix summarizes only the **minimal configuration set** required to reproduce Experiment 2. For experimental objectives, core findings, and framework-level conclusions, please refer to Main Text § 4.2.*

K.2.1 Task and Evaluation Protocol

Table 22: Task and Evaluation Configuration

Item	**Configuration**
Task Chain	$f_1(x) = 2(x+1)$ $\rightarrow f_2(x) = f_1(x) + 3$ $\rightarrow f_3(x) = f_2(x) \times 0.5$
Data Type	Scalar `scalar`, resampled for each repetition (to eliminate data bias)
Loss Function	MSE (Focuses on structural transfer validation, not numerical fitting upper bounds)
Success Criterion	The reported Success Rate counts runs with Task 3 accuracy > 0.95; the stricter "Complete Transfer Chain" criterion (Tasks 1/2 ≥ 0.999 *and* Task 3 > 0.95) applies only to the breakdown in Fig. 5
Evaluation Metrics	Task 3 Mean Accuracy $\pm$ Std Dev, Success Rate (Successful Runs / 100)

K.2.2 Method Pool Configuration (Three Variants, all operating on `scalar`)

```
# Minimal (5 atomic operations)
[
    "increment",          # x -> x+1
    "multiply_by_2",      # x -> 2x
    "add_3",              # x -> x+3
    "multiply_by_05",     # x -> 0.5x
    "identity"            # x -> x
]
```

```
# Extended (Minimal + 1 item)
[...above 5 items..., "add_2"]  # x -> x+2

# Oracle (Static Injection): Minimal pool + f1_direct (6 items)
[
    "increment", "f1_direct",   # x -> x+1  |  x -> 2(x+1)
    "multiply_by_2", "add_3", "multiply_by_05", "identity"
]
```

***Design Principle**: All methods have input/output types of `scalar`, excluding type-matching interference to focus on validating structural composition capabilities.*

K.2.3 Hyperparameters and Control Variables

Table 23: Hyperparameters and Control Variables

Parameter	**Value/Description**
`max_init_layers` (Tasks 1/2)	2 layers / 3 layers (two comparison groups)
Method Pool Evolution	Enabled (`Exp`) / Disabled (`Ctrl`) / Static Injection (`Oracle`)
Independent Repetitions	100 times/configuration
Random Seed Strategy	`42 + rep_id` (`rep_id` $\in [0, 99]$)
Data Sampling	Regenerated for each repetition to avoid data bias
Early Stopping Threshold	Task-level accuracy ≥ 0.999 considered "Perfect Mastery"

K.2.4 Supplementary Notes

- **Chain Analysis Definition**: "Complete Knowledge Transfer Chain" = Task 1 Perfect (Acc ≥ 0.999) $\wedge$ Task 2 Perfect (Acc ≥ 0.999) $\wedge$ Task 3 Success (Acc > 0.95), used to isolate structured transfer paths.
- **Accuracy Calculation Logic**: Based on *Relative Error Threshold*. A sample is considered correct if $\frac{|\hat{y}-y|}{|y|+10^{-8}} < 0.01$. Task accuracy is the proportion of correct samples in the dataset.
- **Method Pool Evolution Data**: The change in the total number of methods at each stage is computed from `exp3_results/<config>/rep<k>/metadata.json` by `plot_method_pool_evolution.py`; the rendered figure is shipped as `GraphEvo_exp3/method_pool_three_stage_evolution.png`.
- **Complete Statistics**: Accuracy for Tasks 1/2/3, method-pool snapshots and evolution trajectories are stored per run under `exp3_results/<config>/rep<k>/` (100 repetitions per configuration, ≈10 MB in total). They are regenerated by `exp3_main.py` and are not shipped; the configuration-level summary backing Table 24 is shipped as `exp3_results/summary_table.csv`.

Table 24 reports the end-to-end performance of Task 3 under various configurations (100 repetitions). Figure 5 further decomposes the success paths of the knowledge transfer chain.

Table 24: Performance comparison for Task 3 in Experiment 3. Evolution column: ✓= Method pool evolution enabled, ×= Disabled, — = Static injection baseline.

Group	Evolution	Max Layers	Method Pool	Avg Acc ± Std	Success Rate (%)
Ctrl_L2_Minimal	×	2	Minimal	0.0006 ± 0.0021	0
Ctrl_L2_Extended	×	2	Extended	0.0003 ± 0.0012	0
Ctrl_L3_Minimal	×	3	Minimal	0.0007 ± 0.0022	0
Exp_L2_Minimal	✓	2	Minimal	0.1308 ± 0.3360	**13**
Exp_L2_Extended	✓	2	Extended	0.1312 ± 0.3359	13
Exp_L3_Minimal	✓	3	Minimal	0.0908 ± 0.2859	9
Oracle_L2	—	2	Oracle	0.1206 ± 0.3247	12

K.3 Experiment 3 Configuration and Supplementary Data (Topology-Aware Task Parallelism)

K.3.1 Experimental Motivation and Core Findings

To validate the **structural-level parallel schedulability** of GDFG during the inference phase, we evaluated the scalability of a topology-aware dynamic task parallelism mechanism in a multi-core CPU environment. This experiment aims to address two framework-level questions:

1. Does the explicit dependency metadata in GDFG support **lock-free or low-conflict parallel scheduling**?
2. How do the topological characteristics of the graph constrain the **parallel efficiency boundaries** of the representation space on real hardware?

Experimental results show that under simulated I/O or non-blocking loads, shallow wide graphs achieved a mean speedup of $10.25\times$ on 16 cores, i.e. 64% of the ideal linear bound, with efficiency still near 90% at 8 cores. This confirms that the DAG structure of GDFG naturally supports fine-grained task parallelism. Further analysis indicates that parallel efficiency is strongly positively correlated with the graph's **Average Width** (Pearson $r \approx 0.99$ across the 30 graphs at 16 cores), rather than its total node count ($r \approx -0.16$) or depth ($r \approx -0.41$), suggesting that "wide and shallow" topological structures are more conducive to multi-core utilization. These measurements use simulated I/O or non-blocking operators. Type-compatible edges validate data interfaces; shared-state conflicts are handled separately by the execution contract in Appendix A.

This experiment confirms that the AdaptoFlux representation space possesses **native parallel potential**. Its performance is primarily constrained by the graph's average width and the scheduling granularity of the backend execution model. This provides clear guidelines for subsequent optimization: structurally, evolution should encourage the generation of high-concurrency, wide-shallow topologies to maximize the utilization of modern multi-core hardware; engineering-wise, migrating to GIL-free backends (such as C++ extensions or process pools) can further unlock the theoretical parallel upper bound. Detailed experimental configurations, structural metric analyses, and flame graph evidence are provided in the following subsections.

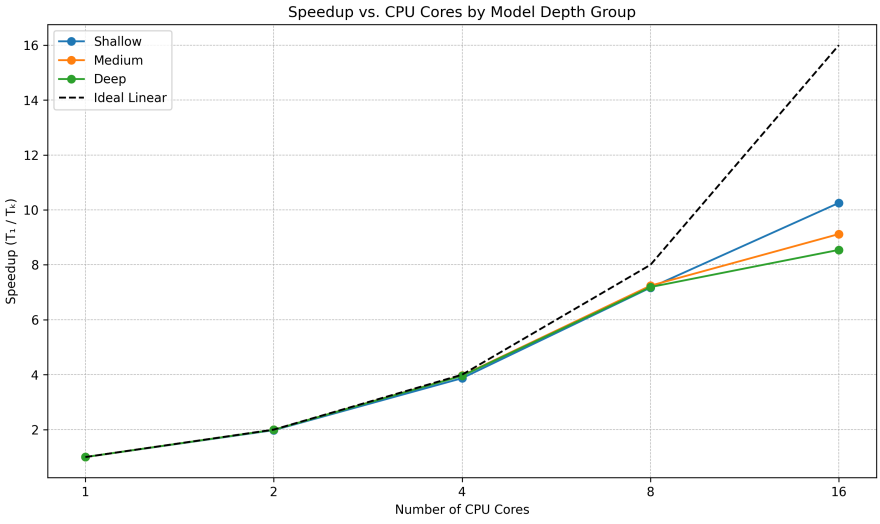


Figure 6: Average speedup under GIL-releasable scenarios. Different depth model groups under simulated non-blocking loads. The dashed line represents ideal linear speedup. Shallow wide graphs achieve a **10.25**$\times$ speedup on 16 cores.

K.3.2 Experimental Environment and Core Configuration

Hardware and Software Environment

- **Executor**: Dynamic task scheduler implemented based on Python `concurrent.futures.ThreadPoolExecutor`.
- **Hardware Platform**: Multi-core CPU environment on the same server, with other process interference isolated.
- **Parallel Granularity**: Number of threads $K \in \{1, 2, 4, 8, 16\}$, simulating concurrency capabilities of different core counts.
- **Load Simulation Modes**:
  - **GIL-Constrained Scenario (No-Sleep)**: Nodes execute pure Python lightweight computations, constrained by the Global Interpreter Lock, preventing true parallelism.
  - **GIL-Releasable Scenario (Sleep)**: every method in the pool calls `time.sleep(0.001)` inside its own implementation before returning, simulating a 1 ms I/O wait or C-extension call so that threads release the GIL and achieve CPU-level parallelism.

Control Variables

- **Fixed Variables**:
  - Dataset: Titanic (Titanic Survival Prediction).
  - Input Batch Size: Kept consistent.
  - Method Pool Configuration: Uses the simplified method pool from Experiment 1 (see Appendix K.1).
  - Random Seed: 42.
- **Variable Variables**:
  - Number of computation threads K.
  - Model topology structures (different evolution results from Experiment 1 training logs, see Appendix K.1).

K.3.3 Experimental Subjects: Detailed List of 30 DAG Models

In the main text, models are divided into shallow, medium, and deep groups based on depth. The table below lists the detailed source information for all 30 models participating in the experiment, including maximum allowed layers (`max_layers`), run sequence (`run`), model type (`model_type`), and actual constructed layers.

***Note**: Models are classified by actual layer count: Shallow (1–10 layers), Medium (11–20 layers), Deep (21–35 layers). Some models have actual layers less than `max_layers` due to early termination during evolution.*

K.3.4 Supplementary Analysis: Relationship Between Node Scale and Parallel Efficiency

The main text states that "total node count is not the core factor determining parallel efficiency." To verify this conclusion, we regrouped the 30 models by **total node count** and statistically analyzed their performance under different core counts.

Performance Statistics Grouped by Node Scale Analysis:
As seen in Table 26, although the large-scale group has a slightly higher absolute speedup under 16 cores ($9.03\times$ vs $8.92\times$), its efficiency fluctuation (standard deviation) is not significantly better than the small/medium groups. More importantly, the small-scale group achieved the highest average speedup ($10.19\times$) under 16 cores, directly refuting the intuitive assumption that "larger models are harder to parallelize."

Scatter Distribution of Node Count vs. Speedup Observation:
Fig. 7 shows that the vertical distribution of speedup has no obvious monotonic correlation with the horizontal axis of node count. Whether with dozens of small nodes or hundreds of large nodes, speedups are distributed within the $8\times$–$10\times$ range. This further confirms that **the total node count of the graph structure is not the core factor determining pipeline parallel efficiency**.

Table 25: Source and Structural Information of the 30 Models Used in Experiment 3

ID	max_layers	run	model_type	Actual Layers	Group
1	5	1	final	5	Shallow
2	5	5	final	5	Shallow
3	5	6	final	5	Shallow
4	10	1	best	9	Shallow
5	10	2	best	2	Shallow
6	10	4	best	8	Shallow
7	10	5	best	5	Shallow
8	10	8	best	6	Shallow
9	5	7	final	5	Shallow
10	5	8	final	5	Shallow
11	15	1	best	12	Medium
12	15	4	final	15	Medium
13	15	8	best	11	Medium
14	15	9	final	15	Medium
15	20	1	final	20	Medium
16	20	3	best	9	Shallow
17	20	4	final	20	Medium
18	20	5	final	20	Medium
19	20	7	best	17	Medium
20	20	9	final	20	Medium
21	25	6	final	25	Deep
22	25	7	final	25	Deep
23	25	10	best	23	Deep
24	30	2	best	22	Deep
25	30	3	final	30	Deep
26	30	7	best	15	Medium
27	30	8	final	30	Deep
28	35	2	best	27	Deep
29	35	5	final	35	Deep
30	30	1	best	11	Medium

Table 26: Average Performance of Models with Different Node Scales on Multi-Core CPUs (No-GIL Valid Data Only)

Node Scale Range	CPU Cores	Avg Speedup (S_K)	Avg Efficiency (E_K)	Efficiency Std Dev
Small ($\leq$93)	1	1.00×	100.0%	—
(10 models)	4	3.86×	96.6%	2.7%
	8	7.13×	89.1%	6.1%
	16	10.19×	63.7%	8.1%
Medium (94–190)	1	1.00×	100.0%	—
(10 models)	4	3.96×	99.0%	0.7%
	8	7.11×	88.9%	8.7%
	16	8.92×	55.8%	11.5%
Large ($\geq$191)	1	1.00×	100.0%	—
(10 models)	4	3.97×	99.1%	0.6%
	8	7.36×	92.0%	5.4%
	16	9.03×	56.5%	8.3%

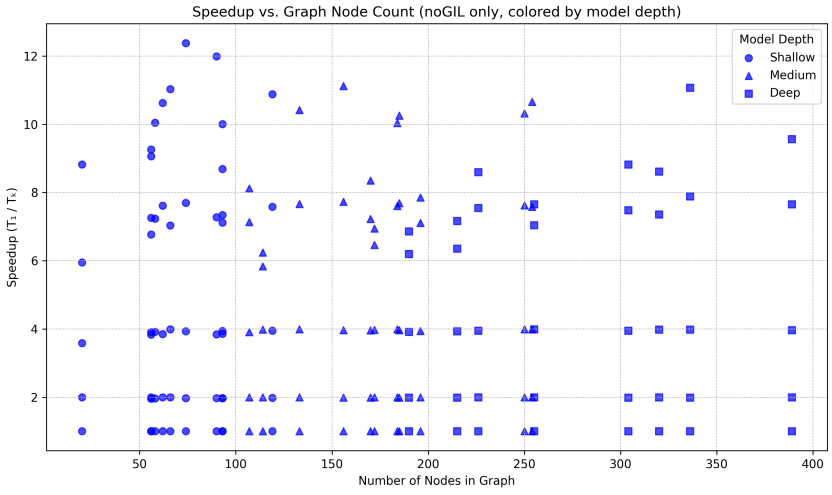


Figure 7: Scatter plot of node count vs. speedup (No-GIL data only). X-axis: Total nodes in graph; Y-axis: Speedup $S_K = T_1/T_K$. Different marker shapes represent model depth (Circle: Shallow, Triangle: Medium, Square: Deep).

K.3.5 Supplementary Analysis: Impact of Structural Heterogeneity Metrics

Besides depth and width, local connectivity characteristics of the graph (such as branching rate, out-degree) may also affect scheduling complexity. We extracted the following structural metrics for analysis:

- **Branching Rate**: Proportion of nodes with out-degree ≥ 2.
- **Average Out-Degree**: Average number of output edges for all nodes in the graph.

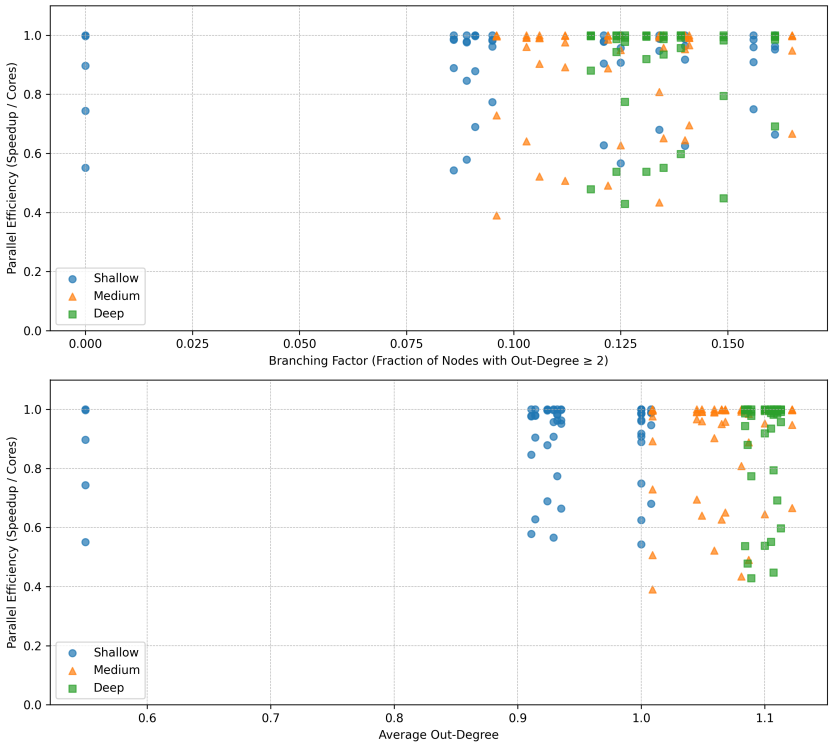


Figure 8: Scatter plots of structural heterogeneity metrics vs. speedup (No-GIL data only). (a) Branching Rate vs. Speedup; (b) Average Out-Degree vs. Speedup.

Analysis:

- **No Significant Correlation**: Fig. 8 shows that high branching rate or high average out-degree did not lead to a systematic decrease in speedup.
- **Coupling of Depth and Heterogeneity**: Notably, deep models (square markers) tend to cluster in regions of high branching rate and high out-degree. This indicates that as evolution layers deepen, the graph structure naturally tends towards more complex connection patterns. However, this increase in complexity did not impose a significant burden on AdaptoFlux's topology-aware scheduling, demonstrating that the in-degree-based dynamic scheduling algorithm has good robustness to local heterogeneity.

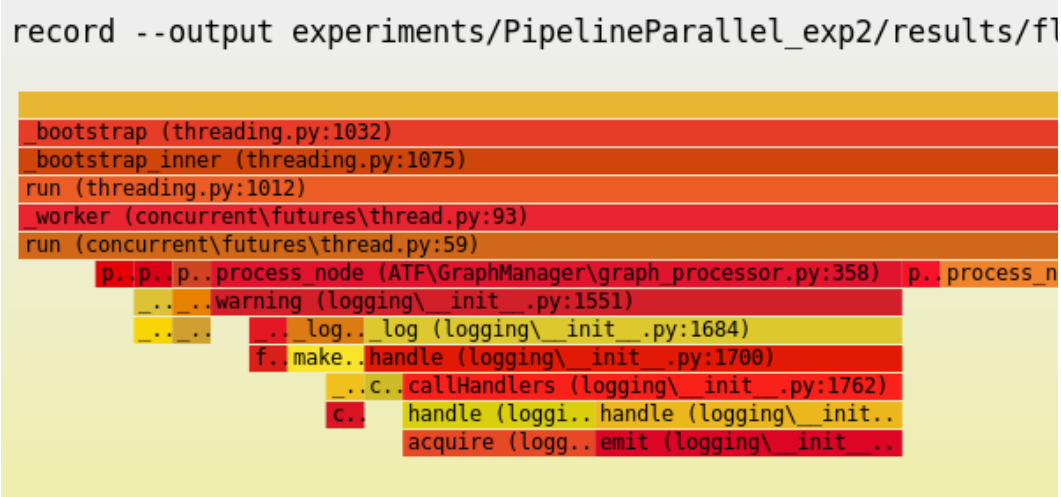
K.3.6 Micro-Performance Profiling: Flame Graph Evidence

To deeply understand the impact mechanism of GIL on task parallelism, we selected three representative models: Shallow (6 layers), Medium (15 layers), and Deep (25 layers), and collected execution flame graphs for both GIL-constrained and GIL-releasable scenarios.

Flame Graph Comparison Overview **Micro-Profiling Conclusion**:

(a) Shallow model (6 layers), pure Python computation

Left half of the recorded display



Right half of the recorded display

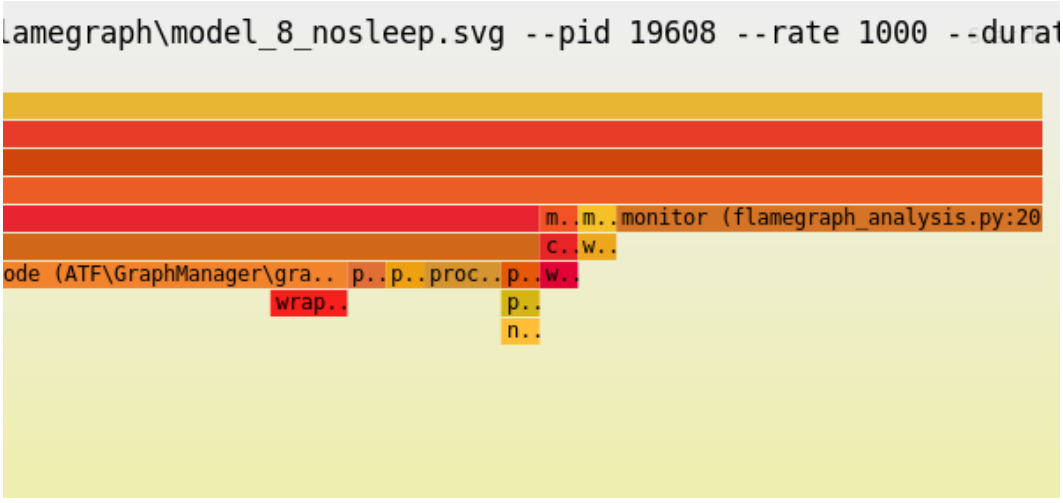
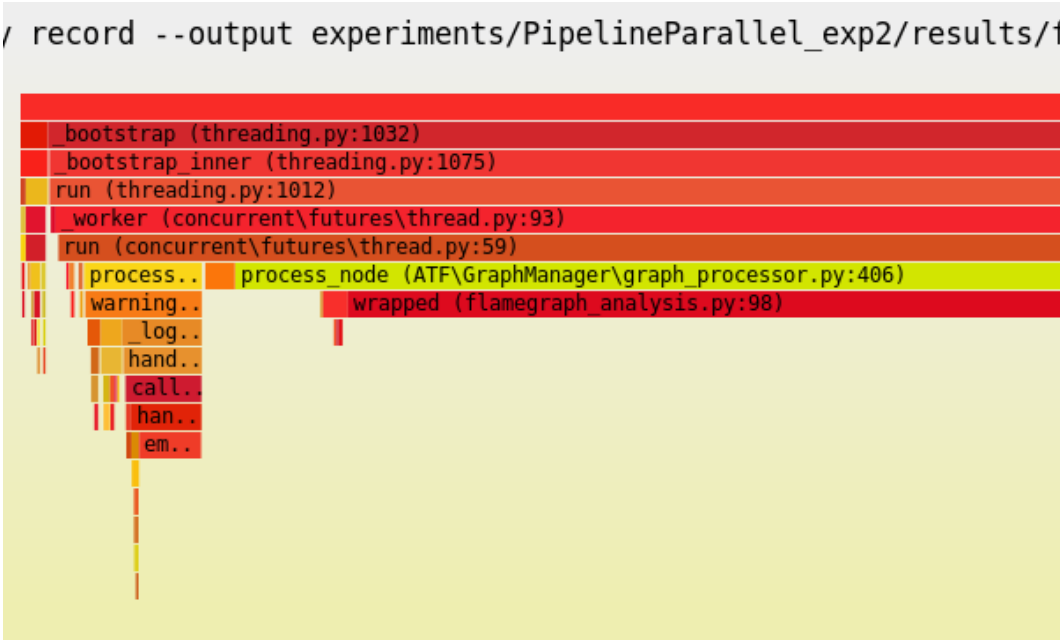


Figure 9: Flame graph comparison for representative models. Each original screenshot is enlarged as two horizontal halves, preserving its complete display. Panels (a), (c) and (e) use GIL-constrained pure Python computation; panels (b), (d) and (f) use GIL-releasable computation with injected `time.sleep`. The six panels continue on the following pages.

(b) Shallow model (6 layers), injected `time.sleep`

Left half of the recorded display



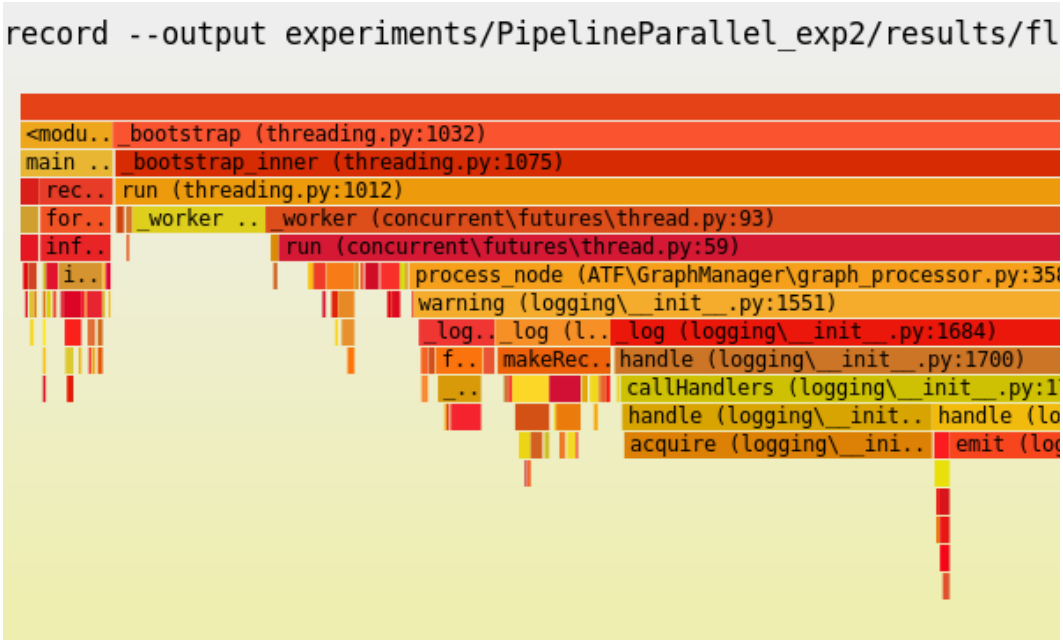
Right half of the recorded display



Figure 9 (continued): shallow model, GIL-releasable scenario.

(c) Medium model (15 layers), pure Python computation

Left half of the recorded display



Right half of the recorded display

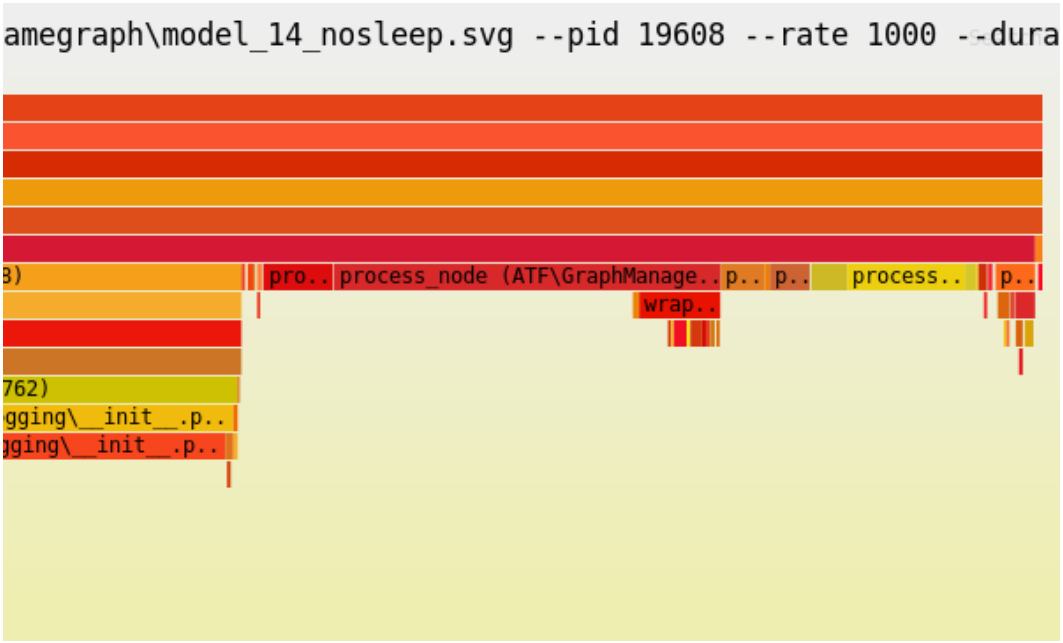
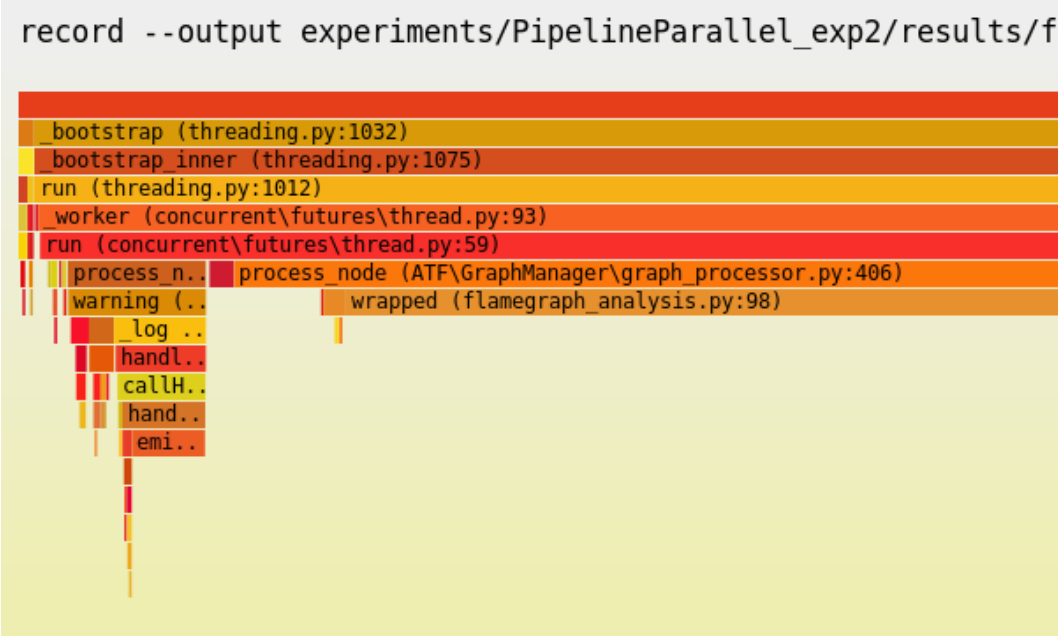


Figure 9 (continued): medium model, GIL-constrained scenario.

(d) Medium model (15 layers), injected `time.sleep`

Left half of the recorded display



Right half of the recorded display

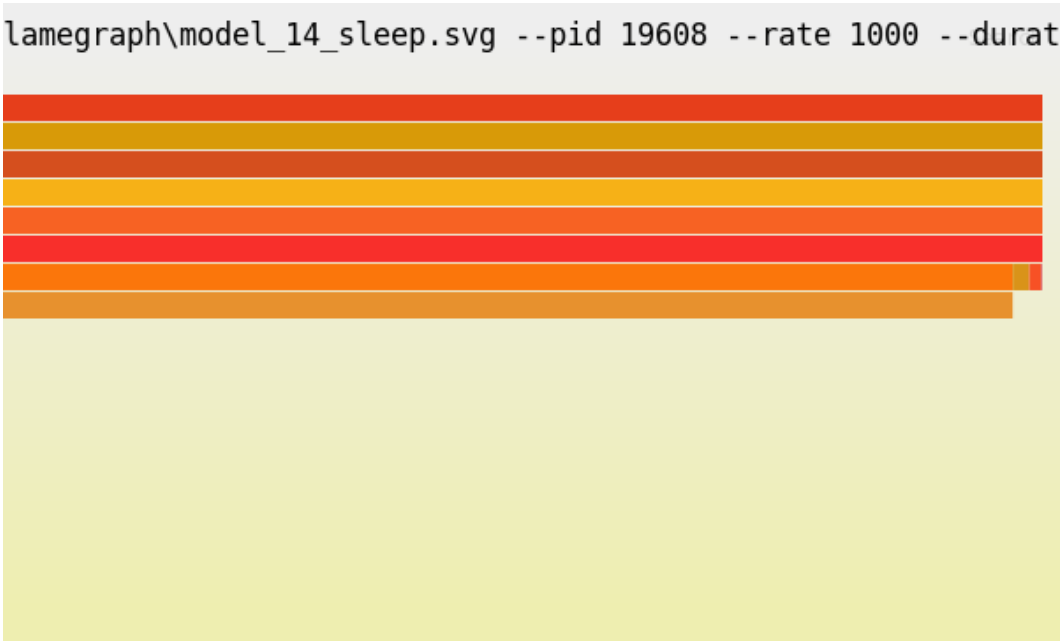
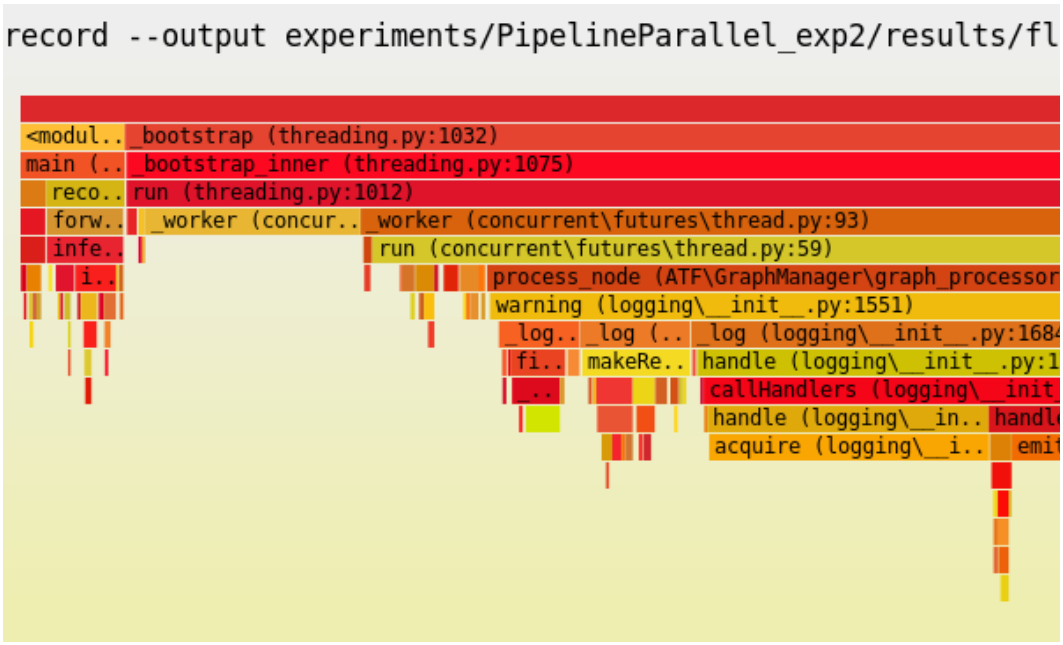


Figure 9 (continued): medium model, GIL-releasable scenario.

(e) Deep model (25 layers), pure Python computation

Left half of the recorded display



Right half of the recorded display

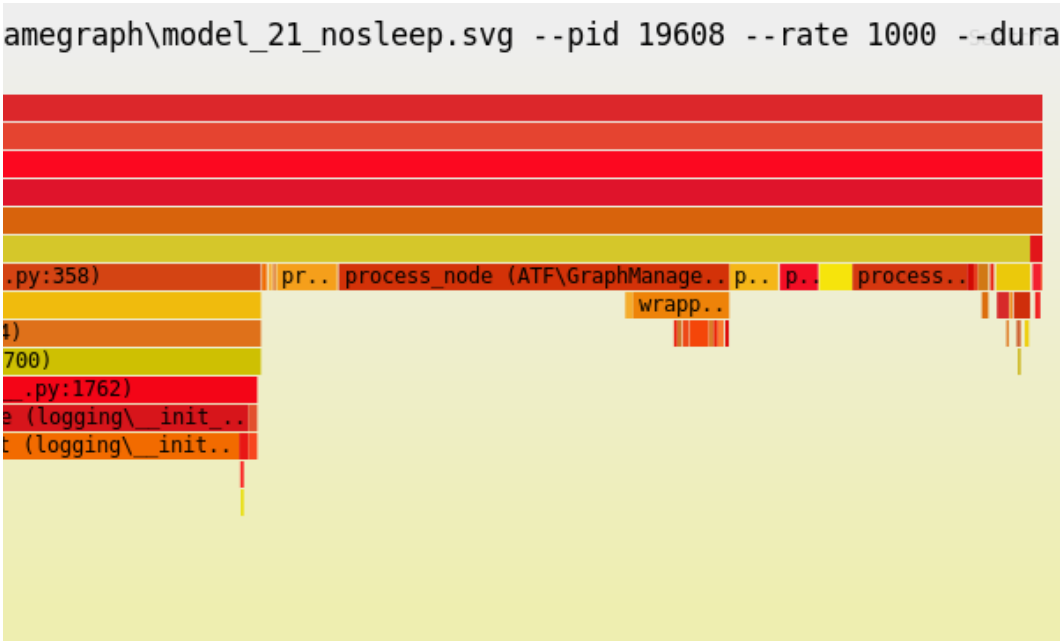
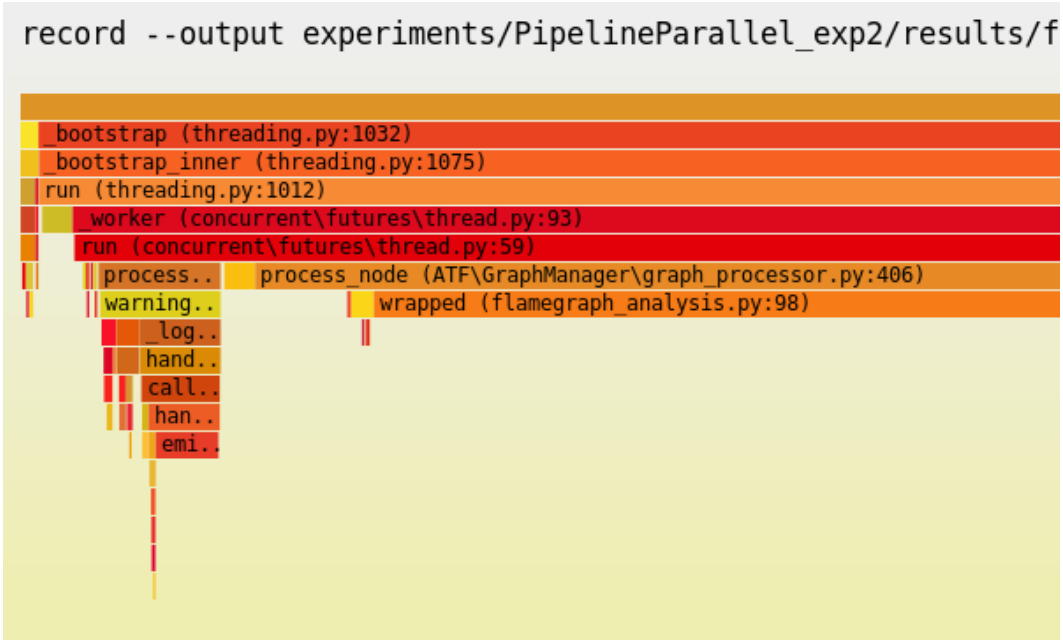


Figure 9 (continued): deep model, GIL-constrained scenario.

(f) Deep model (25 layers), injected `time.sleep`

Left half of the recorded display



Right half of the recorded display



Figure 9 (continued): deep model, GIL-releasable scenario.

- **Panels (a), (c), (e): GIL-Constrained**: The `wrapped` node (computation logic) is extremely narrow in the flame graph, almost drowned out by scheduler overhead (dependency tracking, lock contention, logging). This indicates that multi-threading cannot execute computations concurrently, only incurring additional synchronization costs.
- **Panels (b), (d), (f): GIL-Releasable**: The width of the `wrapped` function increases significantly, showing overlapping execution across different threads. This confirms that when node computation can release the GIL, AdaptoFlux's topology-aware scheduling mechanism effectively exposes node-level parallel potential, and the coarse-grained tasks introduced by `time.sleep` further reduce the relative overhead of context switching.

***Note**: In the flame graphs, the `wrapped` function may appear as `wrap..` or nested under `process_node` due to sampling rate or namespace folding; its true meaning is the node computation proxy function.*

Detailed Interpretation

1. **GIL-Constrained Scenario (Panels (a), (c), (e))**:
   - **Phenomenon**: The `wrapped` node (i.e., user-defined computation logic) is extremely narrow in the flame graph, almost invisible.
   - **Dominant Overhead**: CPU time is mainly consumed by internal scheduler logic, including:
     - Dependency checking and in-degree updating (`decrement_indegree`).
     - Thread lock contention (`threading.Lock.acquire`).
     - Logging and status synchronization.
   - **Conclusion**: In pure Python computation, multi-threading not only fails to parallelize computation but also leads to "negative acceleration" due to frequent context switching and lock contention.
2. **GIL-Releasable Scenario (Panels (b), (d), (f))**:
   - **Phenomenon**: The width of the `wrapped` function increases significantly, occupying the main part of the flame graph.
   - **Parallel Exposure**: Multiple `wrapped` blocks overlap on the timeline, indicating that different threads are executing different node computations simultaneously.
   - **Reduced Scheduler Overhead Ratio**: Although scheduling logic still exists, its relative proportion shrinks significantly because the coarse-grained tasks simulated by `time.sleep` make computation/wait time much larger than scheduling overhead.
   - **Conclusion**: Once node computation can release the GIL (e.g., I/O, C extensions, or explicit sleep), AdaptoFlux's topology-aware scheduling mechanism efficiently distributes tasks to idle cores, fully exposing parallel potential.

K.3.7 Scheduling Algorithm Pseudocode

AdaptoFlux employs a topological sorting-based dynamic task scheduling algorithm, as shown below:

```
def topological_dynamic_schedule(graph, thread_pool):
    # 1. Initialization: Calculate remaining in-degree for each node
    remaining_in_degree = {node: len(node.predecessors) for node in graph
        .nodes}

    # 2. Ready Queue: Find all nodes with initial in-degree 0 (excluding
        root)
    ready_queue = [node for node in graph.nodes if remaining_in_degree[
        node] == 0 and node != root]

    futures = []

    while ready_queue or futures:
        # 3. Submit Tasks: Take nodes from ready queue and submit to
            thread pool
```

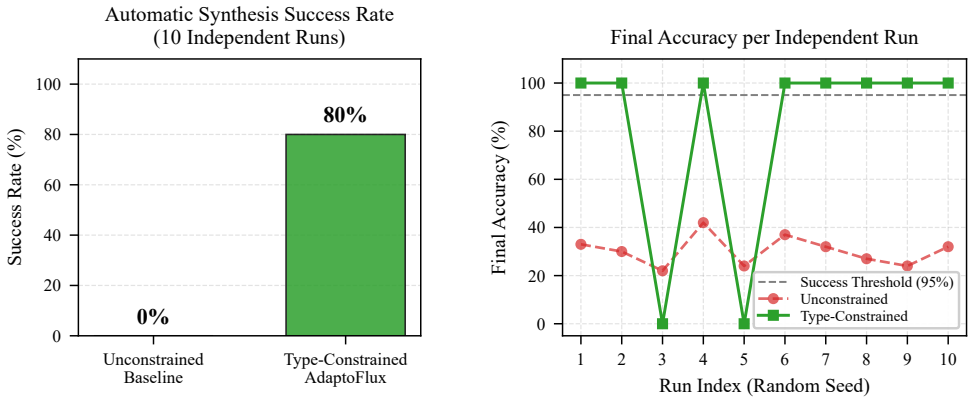


Figure 10: Original ten-seed stateful-synthesis experiment: success rates of 0/10 and 8/10, with per-run accuracy under its randomized evaluation protocol. The exhaustive thirty-seed comparison appears separately in Appendix B.

```
        while ready_queue:
            node = ready_queue.pop(0)
            future = thread_pool.submit(execute_node, node)
            future.add_done_callback(lambda f, n=node: on_node_complete(n
                , f))
            futures.append(future)

        # 4. Wait and Callback Processing
        for future in as_completed(futures):
            node = future_to_node_map[future]
            futures.remove(future)

            # 5. Update Successor Node Status
            for successor in node.successors:
                remaining_in_degree[successor] -= 1
                if remaining_in_degree[successor] == 0:
                    ready_queue.append(successor)

def execute_node(node):
    # Execute the method bound to the node
    result = node.method(*node.inputs)
    return result
```

Listing 1: Topological Dynamic Schedule

K.4 Experiment 4 Configuration and Full Statistical Results (Minimal Stateful Attractor)

Historical protocol. This section preserves the original ten-seed experiment and its randomized episode evaluation. The new three-condition, thirty-seed comparison in Appendix B evaluates all twenty unequal initial position–target pairs. These are separate protocols, and their run counts are reported separately.

***Positioning**: This appendix summarizes only the **minimal configuration set** required to reproduce the automatic stateful synthesis experiment of Main Text § 4.3. For the experimental objective and the framework-level interpretation, please refer to that section.*

K.4.1 Task and Evaluation Protocol

Protocol note (search vs. evaluation). Each run performs exactly one search (one refinement trajectory of the probe) and reports the graph it ends with; the 100 episodes are evaluation rollouts of

Table 27: Minimal Stateful Attractor: task and evaluation configuration

Item	Configuration
Environment (`MinimalState`)	Discrete position `pos` $\in \{-2, \dots, 2\}$ and target `target` $\in \{-2, \dots, 2\}$, resampled at every reset subject to `pos` $\neq$ `target`
Observation	`[pos, target]` at every step; the task goal is the state round(`pos`) = round(`target`)
Episode	One reset followed by at most 4 graph executions; after each execution the graph re-reads the updated observation (closed loop)
Runtime evaluation	100 episodes per run; a run counts as successful iff $\geq 95\%$ of its episodes reach the target state
Training-time signal	GraphEvo loss/accuracy probes use 20 episodes $\times \leq$4 steps, a reduced budget that keeps refinement cycles cheap
Independent runs	10 per configuration, seeds $42 +$ run_id (42–51); both `random` and `numpy` are seeded per run

that single graph, not 100 independent searches. "Success rate" throughout therefore counts runs—i.e. searches—whose final graph solves the task in at least 95% of the evaluation episodes.

This task is deliberately the smallest non-trivial instance that *requires* a stateful graph: since `pos` $\neq$ `target` at reset and the state can only change through a side-effecting action, no static input–output mapping can satisfy it, and the maximum initial distance ($|$`pos` $-$ `target`$| = 4$) exactly saturates the 4-step budget.

K.4.2 Method Pool Configuration (Two Variants, 9 Methods Each)

Both variants register the same nine methods with byte-identical implementations; they differ only in the declared type signatures (and hence in the root input types), so every functional edge is typed in the constrained variant and uniformly `scalar` in the baseline.

```
# Type-Constrained AdaptoFlux: signatures enforce the functional chain
read_current_pos    : scalar                -> pos_type     (state_read)
read_target         : scalar                -> target_type  (state_read)
calc_direction      : pos_type, target_type -> scalar       (
    pure_computation)
move_towards_target : scalar                -> scalar       (
    stateful_action)
# Five irrelevant distractors, all scalar -> scalar
add(x,y), multiply(x,y), sin_op(x), cos_op(x), identity(x)

# Unconstrained Baseline: the same nine methods with identical bodies,
    but
# every signature is scalar -> scalar and the root tokens are plain
    scalars,
# so any method may be wired into any slot.
```

***Design Principle**: Both variants register the same nine methods with byte-identical implementations; only the declared type signatures differ (together with the root input tokens they imply). In the constrained variant the typed edges force the functional chain `read_current_pos`/`read_target` → `calc_direction` → `move_towards_target`, whereas in the baseline every method may be wired into any slot. Any change in success rate is therefore attributable to type routing alone, not to a different operator inventory or to different operator semantics. The root node injects the observation as `pos_type`/`target_type` tokens in the constrained variant and as two `scalar` tokens in the baseline, while `move_towards_target` mutates the implicit environment state $\mathbf{s}_t$ as a side effect (Eq. 2) and returns the updated position.*

Table 28: Minimal Stateful Attractor: hyperparameters and controlled variables

Parameter	**Value/Description**
Probe	`GraphEvoTrainer` used in isolation
`init_mode` / `max_init_layers`	`fixed` / 3 layers
`num_initial_models`	3 initial graph skeletons per run
`max_refinement_steps`	30 local node replacements per cycle
`max_evo_cycles`	5
`candidate_pool_mode`	`all`
`enable_evolution`	`False` (method-pool filtering disabled, so the comparison isolates the type system)
`early_stop_eps`	0.0
Reported graph	The graph each run *ends with*, saved as `results/<config>/run_i/final/graph.gexf`; intermediate snapshots are kept in `best/`

K.4.3 Hyperparameters and Control Variables

K.4.4 Full Statistical Results

Table 29 reports the final accuracy of every run of both configurations. Accuracy is the proportion of the 100 evaluation episodes in which the target state is reached within 4 steps.

Table 29: Per-run final accuracy of the Minimal Stateful Attractor (10 independent runs per configuration). A run is successful iff final accuracy $\geq 95\%$.

Run	Seed	Unconstrained (Baseline)	Type-Constrained AdaptoFlux
1	42	33.0%	100.0%
2	43	30.0%	100.0%
3	44	22.0%	0.0%
4	45	42.0%	100.0%
5	46	24.0%	0.0%
6	47	37.0%	100.0%
7	48	32.0%	100.0%
8	49	27.0%	100.0%
9	50	24.0%	100.0%
10	51	32.0%	100.0%
Success Rate ($\geq 95\%$)		0/10 (0.0%)	8/10 (80.0%)
Mean Final Accuracy $\pm$ Std Dev		0.303 ± 0.060	0.800 ± 0.400

- **Type-constrained configuration**: 8 of 10 runs synthesized a graph whose evaluated form contains the functional core `root` $\rightarrow$ `calc_direction` $\rightarrow$ `move_towards_target`, i.e. the observation is read through the typed root tokens, the signed distance is computed by a pure node, and the resulting scalar drives a side-effecting action. Accuracy in these runs is exactly 1.0 rather than merely above threshold: with $|\texttt{pos} - \texttt{target}| \leq 4$ and a unit step per execution, the synthesized policy is a complete closed-loop solution of the task. Note that the action node is frequently *off* the collapse path (e.g. `move_towards_target` $\rightarrow$ `read_target` $\rightarrow$ `collapse`); execution is triggered by node presence in topological order, not by output connectivity, which is precisely the side-effect semantics of Eq. 2.
- **The two failures (seeds 44 and 46)** terminate on graphs that contain the pure computation node but not `move_towards_target`, so the graph never leaves its initial state and scores 0%. This is a search failure of the greedy refinement probe, not a type-routing rejection: a compatible completion exists in the pool.
- **Unconstrained baseline**: 0 of 10 runs reached the threshold; per-run accuracy ranges from 22% to 42% (mean 0.303 ± 0.060). The final graph of every run contains the action node but never the subtraction node: the scalar driving `move_towards_target` comes from

distractor nodes (`sin_op`, `cos_op`, `identity`), and in three runs (seeds 42, 43, 48) the graph contains no state-reading node at all, so it cannot condition on the observation. Without type routing the five distractors and the two state-reading nodes are interchangeable with the two functional nodes, so greedy replacement settles on compositions that are only partially correct or entirely static.

- **Reproducibility**: the comparison was executed twice on 2026-09-20—once in a clean checkout and once through `run_comparison.py` in the repository—and produced identical per-run accuracies, confirming that the reported outcome is determined by the seeds rather than by run-to-run noise.
- **Interpretation boundary**: the two configurations share the same search budget and the same nine methods. The result therefore isolates the contribution of the representation layer (type signatures plus the resulting routing) to the navigability of the space, and does not constitute evidence that the current probe scales to long-horizon control (Appendix Q).

K.4.5 Paired Analysis and Exact Test

The two configurations are executed with the *same* ten seeds (42–51, Table 29), so the runs form matched pairs rather than two independent samples: the seed fixes initialisation and data order, and the declared type signatures are the only factor that differs. Each pair falls into one of four cells (Table 30), and only the *discordant* pairs carry information about the effect, because pairs in which both conditions succeed or both fail occur equally often whether or not type routing matters.

Table 30: Paired outcomes of the minimal stateful synthesis task over the ten shared seeds. Rows: type-constrained search; columns: unconstrained baseline. Only the discordant cells b and c enter the McNemar test; the concordant cells a and d carry no information about the difference between the two conditions.

	Baseline: fail	Baseline: success
Constrained: success	$b = 8$	$a = 0$
Constrained: fail	$d = 2$	$c = 0$

With $b = 8$ and $c = 0$, the null hypothesis that a discordant pair is equally likely to favour either condition gives $B \sim \mathrm{Binomial}(8, \frac{1}{2})$, and the exact McNemar probability of the observed and more extreme outcomes is

$$p_{\text{one-sided}} = \sum_{k=8}^{8} \binom{8}{k} 2^{-8} = 2^{-8} = 3.9 \times 10^{-3}, \qquad p_{\text{two-sided}} = 2^{-7} = 7.8 \times 10^{-3}.$$

The paired test is the conservative choice: the independent-samples reading of the same counts gives Fisher's exact test, $\binom{10}{8} / \binom{20}{8} = 45/125970 = 3.6 \times 10^{-4}$, but it is not applicable here, because the baseline's ten failures are not ten additional independent observations—they are the partners of the ten matched runs. We therefore report the exact version (`scipy.stats.binomtest`, cross-checked against `statsmodels`' exact McNemar); the asymptotic χ^2 form with continuity correction, $(|b-c|-1)^2/(b+c) = 6.125$, returns $p = 1.3 \times 10^{-2}$ and is unreliable at $b + c = 8$. Ten pairs also bound the design from below: even if all ten discordant pairs favoured the constrained arm, the exact one-sided test could only reach $2^{-10} = 9.8 \times 10^{-4}$, which is why the knowledge-transfer experiment of Appendix K.2 uses 100 repetitions.

K.4.6 Supplementary Notes and Reproduction Guidelines

- **Execution Script**: `experiments/minimal_stateful_comparison/run_comparison.py` reproduces both configurations and writes the aggregate report.
- **Environment and Method Pools**: `env.py` (task state), `methods_constrained.py` and `methods_unconstrained.py` (the two 9-method pools).
- **Complete Logs**: `logs/experiment_log_*.log` contain the per-run accuracy and success flag; the log of the reported execution is `experiment_log_20260920_124456.log`.

- **Per-Run Graphs**: `results/<config>/run_i/{best,final}/` store the saved graph (`graph.gexf`) and the method-pool copy of every run; `results/stateful_synthesis_comparison.png` is the figure used in Fig. 10.

L Ablation Study on Structural Composition and Method Pool Adaptability in Geometrically Sensitive Tasks

This appendix systematically evaluates the structural composition capabilities of the AdaptoFlux framework and its adaptability to different method pools when handling nonlinear classification tasks with complex geometric decision boundaries. By designing strict controlled-variable experiments, we deeply explore the actual utility of different training strategies (single strategy vs. combined strategy) and genetic screening mechanisms in symbolic program synthesis.

L.1 Experimental Design and Task Definitions

To comprehensively examine the framework's performance under varying semantic complexities, we designed three classic nonlinear geometric classification tasks. These tasks share the same training framework (`CombinedTrainer`) and hyperparameter configurations, with the only variable being the **composition of the method pool**. Each experiment was independently repeated 100 times, with results reported as Test Accuracy $\pm$ Standard Deviation.

Task Descriptions

1. **Concentric Circles**:
   - *Challenge*: Data is distributed in concentric circles, which are linearly inseparable. The model needs to implicitly construct radial distance features, i.e., approximate $x_1^2 + x_2^2$.
   - *Method Pool Configuration*: Uses the **Restricted Method Pool**, containing 15 basic operations without trigonometric functions, forcing the model to utilize algebraic operations (such as square, addition) to construct the solution space.
2. **Two Spirals – Full Method Pool**:
   - *Challenge*: Data is distributed in intertwined double spirals, exhibiting strong rotational symmetry. The ideal solution typically involves polar coordinate transformations.
   - *Method Pool Configuration*: Uses the **Full Method Pool**, containing 20 operations, explicitly providing trigonometric functions ($\sin, \cos, \arctan 2$) to verify the upper-bound performance of the framework when equipped with suitable atomic operators.
3. **Two Spirals – Restricted Method Pool**:
   - *Challenge*: Same as above, but with trigonometric function support removed.
   - *Method Pool Configuration*: Uses the **Restricted Method Pool**, aiming to test whether the model can fit spiral boundaries through Taylor expansion approximations or complex algebraic combinations in the absence of direct geometric operators.

Detailed Method Pool Composition

The specific definitions of the two method pools used in the experiments are listed below.

Ablation Experiment Configurations

To systematically compare the effects of different training strategies, we defined the following six ablation configurations. All experiments fixed the random seed to 42 (incremented for each repetition), standardized input features using StandardScaler, with a training set size of 800 and a test set size of 200.

- **Group A: `LayerGrow_Only`**
  - Uses only `LayerGrowTrainer` to build the model. Graph Evolution (GraphEvo) and genetic screening are disabled. Aimed at evaluating the capability of pure structural growth strategies.

Table 31: Restricted Method Pool: 15 basic scalar operations, excluding trigonometric functions.

Method Name	Input Type	Function Description
`return_value`	[scalar]	Identity mapping
`add_values`	[scalar, scalar]	Addition
`calculate_difference`	[scalar, scalar]	Subtraction
`multiply_values`	[scalar, scalar]	Multiplication
`divide_values`	[scalar, scalar]	Division (with zero protection)
`return_two_values`	[scalar]	Copy input value (supports branching)
`increment`	[scalar]	Add 1
`decrement`	[scalar]	Subtract 1
`negate_value`	[scalar]	Negation
`square`	[scalar]	**Square** (Key nonlinear operation)
`relu`	[scalar]	ReLU activation: $\max(0, x)$
`absolute`	[scalar]	Absolute value
`mean_three`	[scalar, scalar, scalar]	Arithmetic mean of three numbers
`max_two`	[scalar, scalar]	Maximum of two numbers
`min_two`	[scalar, scalar]	Minimum of two numbers

Table 32: Full Method Pool: 20 operations, adding geometric and trigonometric functions to the restricted pool.

Method Name	Input Type	Function Description
(First 15 items same as Table 31)		
`sine`	[scalar]	Sine function
`cosine`	[scalar]	Cosine function
`atan2`	[scalar, scalar]	Polar angle calculation $\arctan 2(y, x)$
`sqrt_safe`	[scalar]	Safe square root (returns 0 if $x < 0$)
`scale`	[scalar, scalar]	Scaling: $x \times$ factor

- **Group B: `GraphEvo_Only`**
  - Runs only `GraphEvoTrainer`. Performs local node replacement optimization on a fixed initial shallow structure. Aimed at evaluating the capability of pure local search strategies.
- **Group C: `LayerGrow_Then_GraphEvo`**
  - Serially executes `LayerGrow` once to build a deep structure, followed by `GraphEvo` once for optimization. This serves as the single-round combination baseline.
- **Group D: `CombinedTrainer` (Three Variants)**
  - Uses the complete `CombinedTrainer` framework, executing 3 evolution cycles. The difference lies in the enabling strategy of Genetic Screening:
    1. `disabled`: Genetic screening is disabled. The full method pool is retained in each round. Equivalent to executing 3 independent rounds of "LayerGrow → GraphEvo" processes, allowing continuous structural deepening.
    2. `once`: Genetic screening is executed only once before the first round; subsequent rounds use the compressed sub-pool.
    3. `periodic`: Genetic screening is executed after each round (total 3 times), aiming to test the impact of continuous method pool compression on search efficiency and final performance.

Unified Hyperparameter Settings:

- **LayerGrow**: Max attempts 250. Group A allows up to 15 layers; other groups are limited to 10 layers (to prevent search stagnation caused by excessive depth in a single growth phase).
- **GraphEvo**: Group B runs up to 10 rounds; Groups C/D run 8 rounds per cycle.
- **CombinedTrainer**: Fixed 3 evolution cycles. Target sub-pool size is 8.
- **Genetic Algorithm Config**: Population size 20, generations 4 (reduced to 2 in `periodic` mode to save time), elite ratio 30%, mutation rate 15%, data sampling ratio 30%.
- **Early Stopping**: Enabled for all training, with loss change threshold set to 1×10^{-6}.

L.2 Experimental Results

Concentric Circles Task (Restricted Method Pool)

The Concentric Circles task primarily tests the model's ability to combine quadratic nonlinear features (x^2). Since the method pool includes the `square` operation, an exact solution theoretically exists.

Table 33: Performance Comparison on Concentric Circles Task (100 repetitions)

Strategy	Test Acc ($\pm$ Std)	Pool Size	Time (s)	# $\geq$0.99*	Layers	Nodes
CombinedTrainer_disabled	**0.8909 $\pm$ 0.0837**	20.1	633.1	**24**	7.03	25.80
LayerGrow_Then_GraphEvo	0.7927 $\pm$ 0.1354	16.9	21.6	0	3.74	11.63
LayerGrow_Only	0.7725 $\pm$ 0.0622	15.0	17.6	0	3.81	11.90
CombinedTrainer_once	0.7472 $\pm$ 0.1547	14.8	720.1	9	5.33	20.87
GraphEvo_Only	0.6985 $\pm$ 0.1080	17.7	3.4	2	3.03	8.57
CombinedTrainer_periodic	0.6292 $\pm$ 0.1424	9.8	68.6	0	3.06	9.85

* **High-Precision Success Count**: Number of runs with Test Accuracy $\geq$ 99%.

Observations:

- `CombinedTrainer_disabled` achieved the highest average accuracy (89.1%) and the most high-precision successful instances (24/100). This indicates that under unconstrained multi-round growth, the model has sufficient opportunity to explore deep structures containing correct square combinations.
- `CombinedTrainer_periodic` performed the worst (62.9%), with the pool size compressed to 9.8. This suggests that excessive genetic screening may have eliminated key

intermediate combinations or base operators required to construct concentric circle boundaries.

- Single strategies (Groups A/B) struggled to achieve high precision due to limitations in structural depth or local optima.

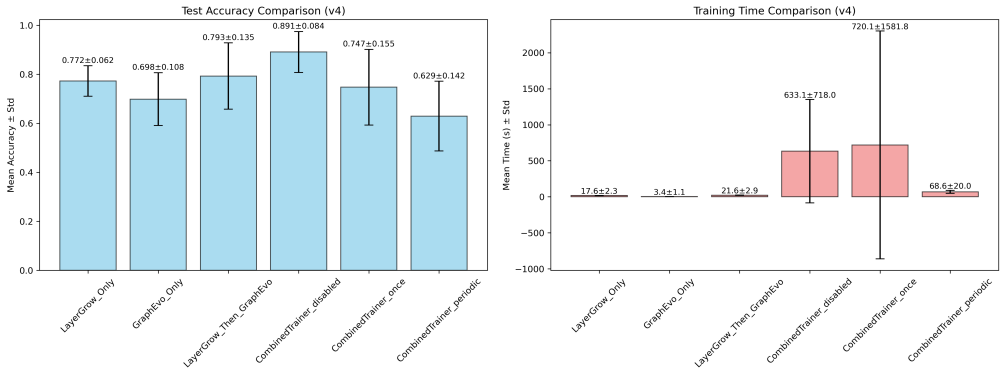
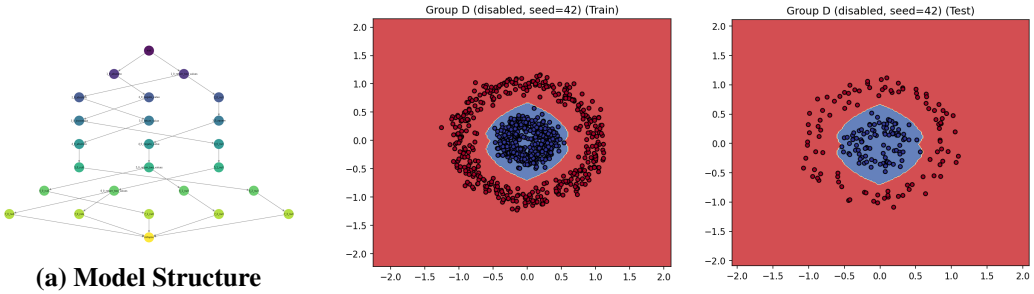


Figure 11: Comparison of Test Accuracy and Training Time for various strategies on the Concentric Circles task (error bars represent standard deviation).



(a) Model Structure **(b) Train Set Boundary** **(c) Test Set Boundary**

Figure 12: Structure and decision boundary visualization of the best model from `CombinedTrainer_disabled` on the Concentric Circles task. Note the clear square and addition combination logic visible in the structure graph.

Two Spirals Task (Full Method Pool)

This task introduces trigonometric functions to verify the framework's ability to utilize explicit geometric operators.

Table 34: Performance Comparison on Two Spirals Task (Full Pool, 100 repetitions)

Strategy	Test Acc ($\pm$ Std)	Pool Size	Time (s)	# $\geq$0.99*	Layers	Nodes
CombinedTrainer_disabled	**0.8493 $\pm$ 0.0338**	26.0	416.4	**1**	6.45	20.30
LayerGrow_Then_GraphEvo	0.8148 $\pm$ 0.0556	22.0	26.5	1	4.63	14.75
LayerGrow_Only	0.8053 $\pm$ 0.0389	20.0	27.4	0	4.77	15.67
CombinedTrainer_once	0.7823 $\pm$ 0.0793	15.9	742.6	0	6.15	20.75
GraphEvo_Only	0.7624 $\pm$ 0.0798	22.8	3.8	0	3.20	8.59
CombinedTrainer_periodic	0.7209 $\pm$ 0.1094	9.9	80.6	0	4.67	16.23

* **High-Precision Success Count**: Number of runs with Test Accuracy $\geq$ 99%.

Observations:

- Even with powerful operators like `atan2`, the Two Spirals task remains highly challenging. `CombinedTrainer_disabled` still maintained the lead, with an average accuracy of 84.9%.
- Notably, `CombinedTrainer_once`, despite having a moderate pool size, had the longest runtime (742.6s) and lower performance than the `disabled` mode. This may be because after the initial screening, the remaining sub-pool, although streamlined, limited the exploration diversity of new structures in subsequent multi-round evolutions.
- `GraphEvo_Only` performed better in the Full Pool than in the Restricted Pool (see Table 35), indicating that local optimization can more effectively fine-tune node semantics when rich semantic operators are available.

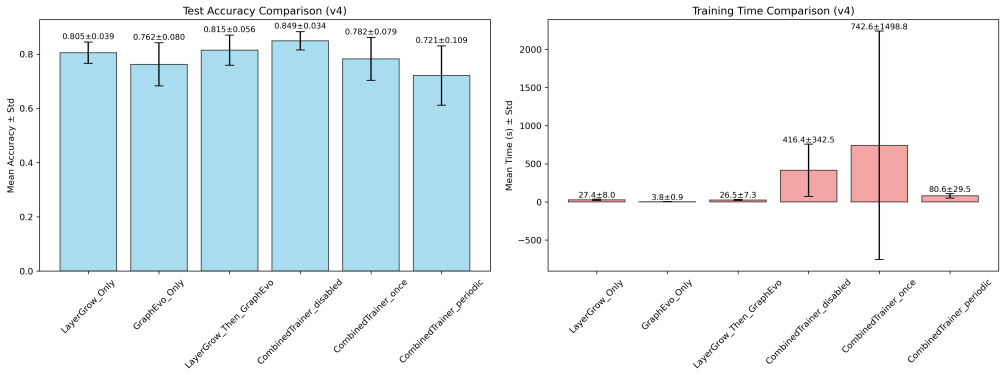
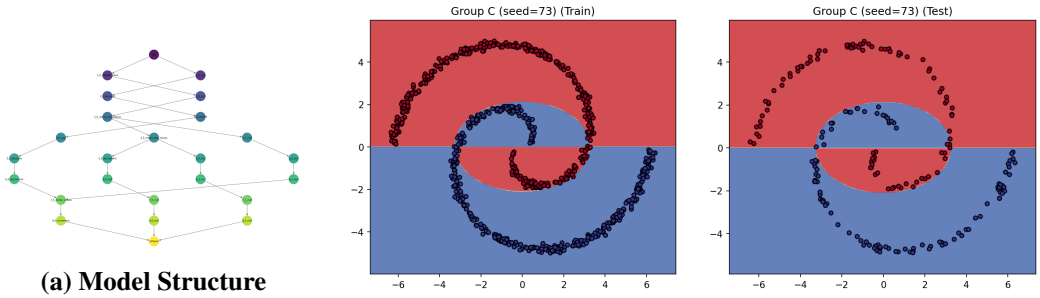


Figure 13: Performance comparison of various strategies on the Two Spirals task (Full Method Pool).



(a) Model Structure **(b) Train Set Boundary** **(c) Test Set Boundary**

Figure 14: Structure and decision boundary visualization of the best model from `CombinedTrainer_disabled` on the Two Spirals task (Full Pool). Direct calls to `atan2` nodes are visible in the structure.

Two Spirals Task (Restricted Method Pool)

This task removes trigonometric functions, forcing the model to approximate spiral boundaries using algebraic operations.

Observations:

- Surprisingly, `CombinedTrainer_disabled` in the Restricted Pool (83.9%) showed a minimal performance gap compared to the Full Pool (84.9%). This demonstrates that AdaptoFlux's **GDFG structural composition capability** possesses strong expressive redundancy, effectively approximating complex geometric functions through deep algebraic structures.
- However, `GraphEvo_Only` performance dropped significantly in the Restricted Pool (73.3% vs 76.2%), indicating that without high-level semantic operators, relying solely on

Table 35: Performance Comparison on Two Spirals Task (Restricted Pool, 100 repetitions)

Strategy	Test Acc ($\pm$ Std)	Pool Size	Time (s)	# $\geq$0.99*	Layers	Nodes
CombinedTrainer_disabled	**0.8386 $\pm$ 0.0390**	21.0	589.3	**1**	6.40	22.09
LayerGrow_Only	0.8019 $\pm$ 0.0355	15.0	26.0	0	5.62	20.69
LayerGrow_Then_GraphEvo	0.7989 $\pm$ 0.0421	17.0	25.2	0	4.91	16.61
CombinedTrainer_once	0.7791 $\pm$ 0.0644	15.9	505.3	0	5.64	20.06
CombinedTrainer_periodic	0.7381 $\pm$ 0.0762	10.0	78.6	0	4.50	14.89
GraphEvo_Only	0.7326 $\pm$ 0.0992	17.9	3.6	0	3.20	8.80

* **High-Precision Success Count**: Number of runs with Test Accuracy $\geq$ 99%.

local node replacement makes it difficult to escape local optima; it must rely on the structural diversity provided by `LayerGrow`.

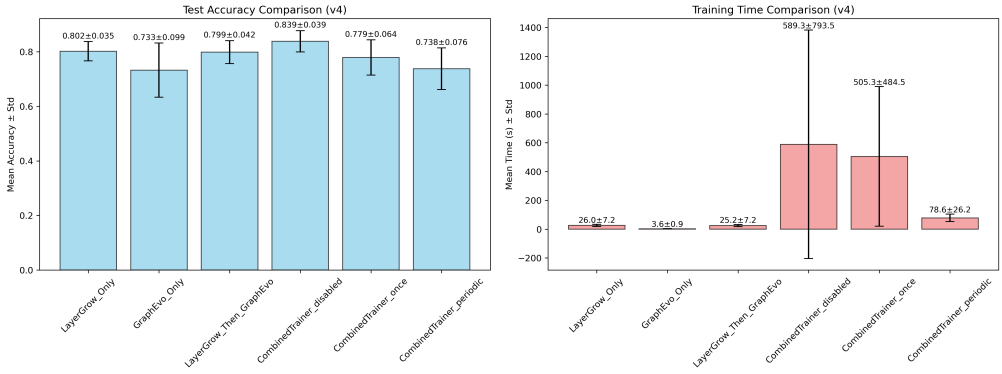
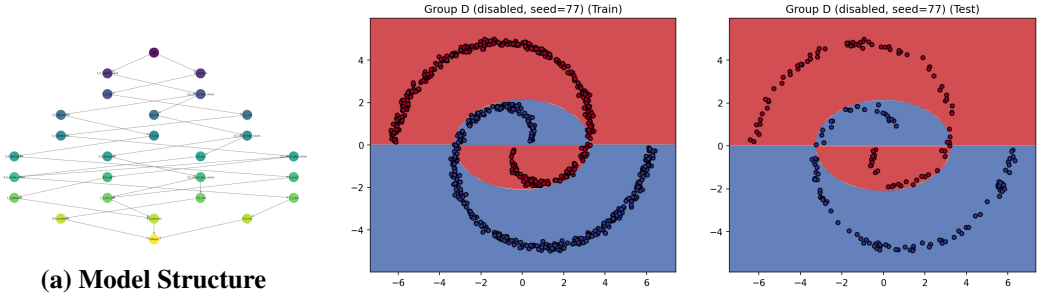


Figure 15: Performance comparison of various strategies on the Two Spirals task (Restricted Method Pool).



(a) Model Structure **(b) Train Set Boundary** **(c) Test Set Boundary**

Figure 16: Structure and decision boundary visualization of the best model from `CombinedTrainer_disabled` on the Two Spirals task (Restricted Pool). Note that its structure is more complex than the Full Pool model, approximating the spiral shape through multi-layer algebraic operations.

L.3 Analysis and Discussion

Based on the experimental results above, we draw the following core conclusions:

- **Method Pool Composition Determines Performance Upper Bound, but Structural Composition Can Compensate for Semantic Deficits**
  - In the Two Spirals task, the Full Pool slightly outperformed the Restricted Pool, verifying the natural advantage of geometric operators like `atan2`/`sin`/`cos` for polar coordinate tasks.

  - However, the gap was not substantial ($\sim$1%), and high-precision solutions were still obtained in the Restricted Pool. This proves that AdaptoFlux's GDFG structural composition capability has strong expressive redundancy, capable of approximating complex geometric functions through deep algebraic structures.

- **Multi-Round Combination (Disabled) Outperforms Single-Round and Over-Screening**
  - `CombinedTrainer_disabled` achieved the best average performance and high-precision success rate in all tasks. Through 3 rounds of "LayerGrow $\rightarrow$ GraphEvo" cycles, the model could gradually deepen the structure and refine semantics, proving the effectiveness of **deep structural exploration**.
  - In contrast, `LayerGrow_Then_GraphEvo` (single-round) was significantly inferior to the multi-round mode due to the depth limit of single growth.
- **Genetic Screening is a Double-Edged Sword: Diversity vs. Efficiency**
  - `CombinedTrainer_periodic` performed the worst in all tasks. Although it significantly reduced the method pool size (to $\sim$10) and shortened single-round time, **over-compression led to the loss of key nonlinear operations (such as `square`, `atan2`) or critical intermediate combinations**.
  - In geometrically sensitive tasks, **preserving method pool diversity is more important than streamlining the search space**. Genetic screening mechanisms should be used with caution; it is recommended to prioritize `disabled` or `once` modes when tasks are highly sensitive to specific atomic operations to avoid premature pruning of the search space.
- **Complementarity of Layered Growth and Graph Evolution**
  - `LayerGrow_Then_GraphEvo` outperformed separate `LayerGrow_Only` or `GraphEvo_Only` in all tasks.
  - `LayerGrow` is responsible for expanding the **macro-topological depth** of the structure, solving the problem of "whether it can be represented"; `GraphEvo` is responsible for **micro-semantic alignment**, solving the problem of "how accurately it is represented". The synergy of the two is the core driving force for AdaptoFlux to navigate discrete spaces.
- **Strong Interpretability and Symbolic Nature**
  - As shown in Figures 12, 14, and 16, the generated GDFG structures are clear and sparse. The mathematical expression of the decision logic can be directly deduced from the model structure graph (e.g., the $x_1^2 + x_2^2$ combination in the Concentric Circles task, or the `atan2` call in the Two Spirals task).
  - This **white-box characteristic** is the essential advantage of AdaptoFlux as a symbolic-structural hybrid paradigm, giving it potential application value in scientific computing and control fields requiring high credibility.

L.4 Conclusion

The experiments in this appendix demonstrate that AdaptoFlux exhibits powerful structural composition capabilities in complex geometric tasks. Its performance depends not only on the **completeness** of the method pool but also on the balance between **structural depth** and **semantic diversity** maintained by the training strategy.

For practical applications, we recommend:

- When exploring unknown complex tasks, prioritize using **`CombinedTrainer_disabled`** to maximize the exploration space.
- If computational resources are limited and the task semantics are known, try **`CombinedTrainer_once`** for moderate compression.
- Avoid using high-frequency periodic genetic screening (`periodic`) in the early stages to prevent destroying the connectivity of the solution space.

M STATIC SYMBOLIC REGRESSION WITH RESTRICTED METHOD POOLS

We compare the original AdaptoFlux probes with PySR and gplearn on eight Feynman formulas, using the four arithmetic primitives listed below. The experiment measures formula recovery, numerical accuracy and sensitivity to output aggregation. All results use numerical operators; stateful execution is evaluated separately in Sections 4.3 and E.

M.1 TASKS AND RESTRICTED EXPERIMENTAL SETTING

The eight target formulas are:

$$f_1(m, c) = m \cdot c^2 \quad \text{(Energy)}$$
$$f_2(m, a) = m \cdot a \quad \text{(Newton's Second Law)}$$
$$f_3(m, v) = 0.5 \cdot m \cdot v^2 \quad \text{(Kinetic Energy)}$$
$$f_4(q_1, q_2, r) = \frac{q_1 q_2}{r^2} \quad \text{(Coulomb's Law)}$$
$$f_5(n, T, V) = \frac{nT}{V} \quad \text{(Ideal Gas Law)}$$
$$f_6(I, R) = I \cdot R \quad \text{(Ohm's Law)}$$
$$f_7(v_1, v_2, t) = \frac{v_2 - v_1}{t} \quad \text{(Acceleration Definition)}$$
$$f_8(V, I) = V \cdot I \quad \text{(Electric Power)}$$

The common mathematical primitives are `add`, `sub`, `mul`, `div`. AdaptoFlux's layer-building interface also supplies `fanout` (variable copying) and `identity` (passthrough). These auxiliary methods support reuse in that particular construction interface; variable reuse is also intrinsic to a general DAG through multiple outgoing edges. Output aggregation is a further experimental configuration, varied among `first`, `prod` and `sum` below.

Deliberately Restricted Setting (a sanity check, not a race): Because gplearn and PySR are pure-symbolic tools that cannot exercise AdaptoFlux's method imports, we **actively disabled ATF's method-import capability**, reducing it to a pure symbolic synthesizer using only 5 basic operators. This isolates the expressiveness of the representation itself (graph structure vs. tree/expression) from differences in function sets, rather than staging a head-to-head comparison of search engines. The experiment therefore probes a lower bound on what the GDFG substrate can express under the current probes; it is not the operating mode for which AdaptoFlux is designed.

M.2 EXPERIMENTAL SETUP

All methods were run independently 10 times on each task with different random seeds. This experiment reports the **best-of-10 MSE, Exact Match status (Exact?), runtime (seconds), and final method pool size**. AdaptoFlux was tested with three different `collapse_mode` strategies:

`first` First valid output;

`prod` Product aggregation;

`sum` Sum aggregation.

Baseline methods include:

gplearn Classic Genetic Programming implementation, using the same 5-function set;

PySR Modern symbolic regression tool based on sparse regression and symbolic enumeration;

XGBoost Black-box gradient boosting tree model, used to contrast pure predictive performance with interpretability trade-offs.

To more accurately assess **program equivalence** in symbolic regression tasks, this experiment adopts a **numerical-symbolic joint discrimination strategy**. Specifically, we first determine if the model output is numerically highly consistent with the ground truth using the following function:

```python
def is_numerical_exact_match(y_pred, y_true, tol=1e-5):
    return np.all(np.abs(y_pred - y_true) < tol)
```

This function requires the prediction error for all test samples to be strictly less than 10^{-5}. However, relying solely on MSE or numerical matching may yield **false positives** (e.g., the model fits an expression that looks similar but is structurally incorrect) or **false negatives** (e.g., high MSE due to constant offset but correct structure). Therefore, in addition to automatic judgment, we further performed **manual symbolic parsing verification** on the **best expressions** output by ATF, gplearn, and PySR to confirm whether they are **algebraically equivalent** to the target formula (e.g., $x_0 \cdot x_1 \cdot x_1$ is equivalent to $m \cdot c^2$ under variable mapping).

Based on this dual verification mechanism, we constructed Table 41, which specifically records whether each method **truly recovered the target physical formula at the symbolic level**.

M.3 Results Overview

Tables 36–38 present the best-of-10 performance of AdaptoFlux under the three collapse modes; Table 39 summarizes the corresponding results for gplearn, PySR, and XGBoost; and Table 41 reports the symbolic-level exact match analysis. Best-of-10 tables are retained only because this is how the reference systems are conventionally reported on this suite; stability-oriented conclusions rely on the mean $\pm$ std statistics in Tables 42–44 and Figures 18–19.

Key findings are as follows:

- **gplearn and PySR achieved exact matches (Exact?=Yes) on most tasks**, with MSE typically below 10^{-12}, performing robustly especially in binary multiplication or simple division tasks (e.g., F_ma, OhmsLaw, Acceleration, Power, Coulomb)—as expected for systems specialized in exactly this static setting.
- **XGBoost failed to recover analytical expressions**, with MSE generally 2–4 orders of magnitude higher, highlighting its fundamental limitations in symbolic recovery tasks.
- **AdaptoFlux's performance is highly dependent on the collapse mode**:
  - The `prod` mode performed best in multiplication-dominated tasks (F_ma, OhmsLaw, Power, E_mc2, KE), achieving exact matches (MSE $\sim 10^{-13}$) for F_ma, OhmsLaw, and Power, comparable to gplearn/PySR;
  - The `first` mode succeeded in some tasks (e.g., F_ma) but exhibited poorer overall stability;
  - The `sum` mode generally performed worst, barely reaching high precision only in OhmsLaw and Power.
- **Complex tasks remain a bottleneck for the current probes**: for tasks involving explicit constants (e.g., 0.5 in KE) or three-variable nonlinear combinations (IdealGas, Coulomb), the restricted ATF failed to recover the exact formula, with MSE significantly higher than the specialized reference systems. As analyzed below, this reflects the limits of the lightweight local-search probes under a fixed uniform configuration rather than a representational deficit.
- **Native action support is not exercised here**: the "Supports actions" markers in the tables record only that the execution backend accepts side-effecting methods; that capability is deliberately disabled in this section and is demonstrated separately by the automatic stateful synthesis experiment (Section 4.3) and the Snake case study (Appendix N).

We tested three collapse modes, with `prod` performing best in multiplication-dominated tasks; thus, subsequent analysis focuses primarily on this mode.

To provide a more comprehensive view of performance stability beyond the "best-of-10" metrics reported in Tables 36–41, we visualize the full distribution of MSE and exact match success rates across all 10 independent runs in Figure 18 and Figure 19.

Figure 18 highlights the vast difference in convergence quality between simple multiplication tasks (e.g., `F_ma`) and complex multi-variable tasks (e.g., `IdealGas`). Notably, the `prod` mode exhibits extremely low variance in multiplication-dominated tasks, confirming that its superior performance is

Table 36: Parameter-free symbolic regression with collapse=first (best of 10 runs)

Task	Exact?	MSE	Runtime (s)	Method Pool	Notes
E_mc2	No	2e+03	4.0	5	Supports actions
F_ma	**Yes**	3e-12	0.3	5	Supports actions
KE	No	4e+03	42.0	8	Supports actions
Coulomb	No	1e+01	13.1	7	Supports actions
IdealGas	No	3e+04	11.0	7	Supports actions
OhmsLaw	**Yes**	5e-13	0.4	5	Supports actions
Acceleration	No	7e+00	10.7	7	Supports actions
Power	**Yes**	1e-12	0.4	5	Supports actions

Table 37: AdaptoFlux performance with `collapse=prod` (best of 10 runs). The "Exact?" column indicates **numerical exact match**: all absolute errors on the test set are $< 10^{-5}$. This is stricter than $\mathrm{MSE} < 10^{-5}$ and does not imply symbolic equivalence.

Task	Exact?	MSE	Runtime (s)	Method Pool	Notes
E_mc2	No	2e-11	0.1	5	Supports actions
F_ma	**Yes**	2e-13	0.0	5	Supports actions
KE	No	2e-11	0.1	5	Supports actions
Coulomb	No	9e+00	6.0	5	Supports actions
IdealGas	No	2e+04	19.8	8	Supports actions
OhmsLaw	**Yes**	2e-13	0.0	5	Supports actions
Acceleration	No	5e+00	11.2	7	Supports actions
Power	**Yes**	8e-13	0.0	5	Supports actions

Table 38: Parameter-free symbolic regression with collapse=sum (best of 10 runs)

Task	Exact?	MSE	Runtime (s)	Method Pool	Notes
E_mc2	No	8e+00	532.1	8	Supports actions
F_ma	No	4e+01	6.4	7	Supports actions
KE	No	8e+02	39.9	8	Supports actions
Coulomb	No	1e+01	8.6	6	Supports actions
IdealGas	No	3e+04	2.2	5	Supports actions
OhmsLaw	**Yes**	1e-12	0.5	5	Supports actions
Acceleration	No	1e+01	9.7	6	Supports actions
Power	**Yes**	2e-12	1.6	5	Supports actions

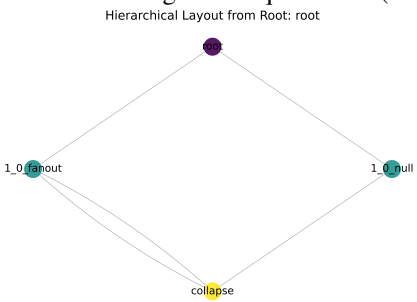
Table 39: Baseline performance of gplearn, PySR, and XGBoost (best of 10 runs)

Task	Method	Exact?	MSE	Runtime (s)
E_mc2	Gplearn	**Yes**	8.3e-12	17.86
	PySR	**Yes**	8.3e-12	20.29
	Xgboost	No	1.5e+02	0.04
F_ma	Gplearn	**Yes**	1.9e-13	16.40
	PySR	**Yes**	2.1e-12	20.49
	Xgboost	No	2.5e+01	724.38
KE	Gplearn	No	3.4e-11	72.71
	PySR	No	3.4e-11	20.31
	Xgboost	No	3.9e+02	0.03
Coulomb	Gplearn	**Yes**	6.8e-14	10.49
	PySR	**Yes**	2.7e-13	20.46
	Xgboost	No	6.4	0.02
IdealGas	Gplearn	No	1e-10	122.72
	PySR	No	2.2e-10	20.32
	Xgboost	No	4.5e+03	0.05
OhmsLaw	Gplearn	**Yes**	2.2e-13	9.24
	PySR	**Yes**	2.2e-13	20.40
	Xgboost	No	9.6	0.03
Acceleration	Gplearn	**Yes**	3.9e-14	28.84
	PySR	**Yes**	1.1e-13	20.46
	Xgboost	No	1.7	0.04
Power	Gplearn	**Yes**	8e-13	116.68
	PySR	**Yes**	1e-12	20.20
	Xgboost	No	3.1e+01	0.03

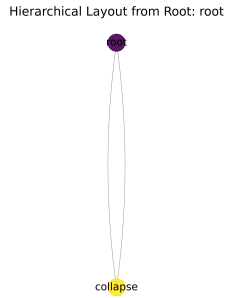
Table 40: Symbolic expressions recovered by gplearn and PySR

Task	Method	Expression	Exact?
E_mc2	gplearn	`mul(mul(X1, X0), X1)`	**Yes**
	PySR	`x0*x1*x1`	**Yes**
F_ma	gplearn	`mul(X0, X1)`	**Yes**
	PySR	`x0*x1 - 0.027047554 - 1*(-0.027047586)`	**No**
KE	gplearn	`mul(mul(X2, X1), mul(X0, X1))`	**Yes**
	PySR	`x0*x1*x1*x2`	**Yes**
Coulomb	gplearn	`div(mul(div(X1, X2), X0), X2)`	**Yes**
	PySR	`x0/((x2/((x1/x2))))`	**Yes**
IdealGas	gplearn	`div(X1, div(X2, X0))`	**Yes**
	PySR	`x0*x1/x2`	**Yes**
OhmsLaw	gplearn	`mul(X0, X1)`	**Yes**
	PySR	`x0*x1 - (-x0 + x0 - x1 + x1)`	**Yes**
Acceleration	gplearn	`div(sub(X1, X0), X2)`	**Yes**
	PySR	`(-x0+x1-6.078859e-6/(x2/((x2/(-16.520508))))))/x2`	**No**
Power	gplearn	`mul(X1, X0)`	**Yes**
	PySR	`x1*(x0 - 1*0.99999976) + x1`	**No**

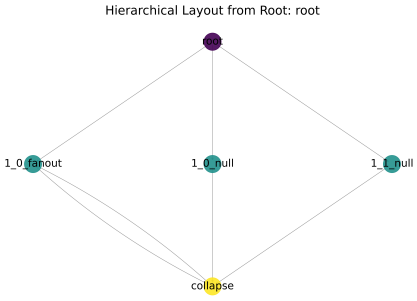
Figure 17: Best computation graphs generated by AdaptoFlux on eight Feynman formula tasks. Yes/No indicates algebraic equivalence (see Table 41).



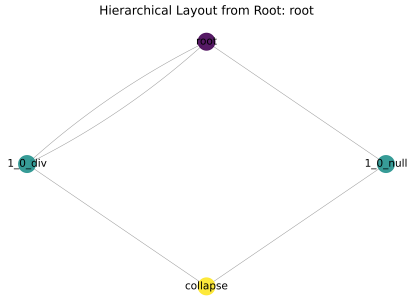
(a) E_mc2 (Yes)



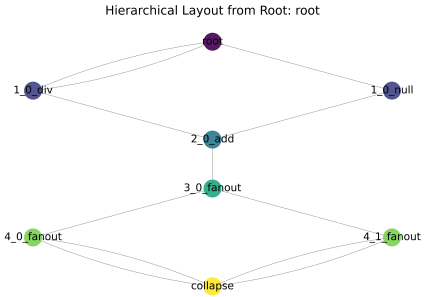
(b) F_ma (Yes)



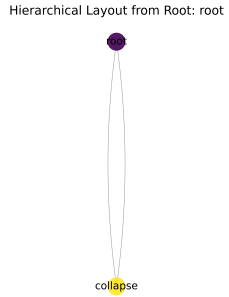
(c) KE (Yes)



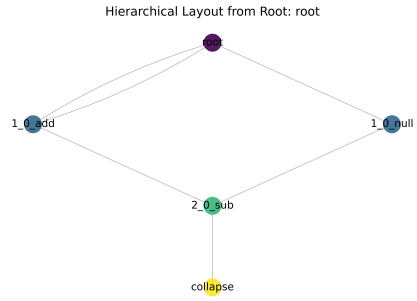
(d) Coulomb (No)



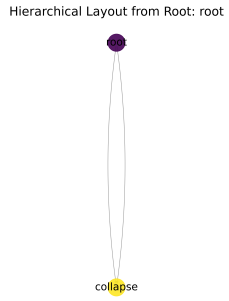
(e) Ideal Gas (No)



(f) Ohm's Law (Yes)



(g) Acceleration (No)



(h) Power (Yes)

Table 41: Symbolic exact match analysis: whether the best expression is **algebraically equivalent** to the ground-truth formula, as determined by manual inspection. This is stricter than numerical matching.

Task	AdaptoFlux (prod)	gplearn	PySR
E_mc2	**Yes**	**Yes**	**Yes**
F_ma	**Yes**	**Yes**	**No**
KE	**Yes**	**Yes**	**Yes**
Coulomb	No	**Yes**	**Yes**
IdealGas	No	**Yes**	**Yes**
OhmsLaw	**Yes**	**Yes**	**Yes**
Acceleration	No	**Yes**	**No**
Power	**Yes**	**Yes**	**No**

consistent rather than an outlier. Conversely, for complex tasks like `IdealGas` and `Coulomb`, all modes exhibit high variance and elevated MSE, indicating that the current search strategy struggles to consistently converge to the correct symbolic structure in high-dimensional spaces.

Figure 19 quantifies the stability of symbolic recovery. It clearly demonstrates that the `prod` mode achieves a 100% exact match success rate on tasks like `F_ma`, `OhmsLaw`, and `Power`, significantly outperforming `first` and `sum` modes in terms of reliability. This visual evidence reinforces the conclusion that the choice of collapse mode is critical for aligning with the mathematical structure of the target task.

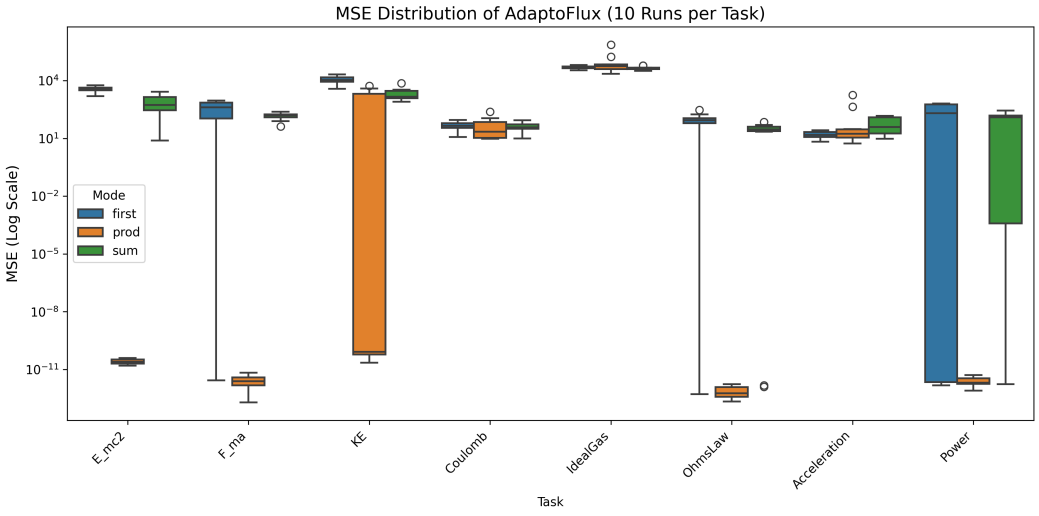


Figure 18: Distribution of Mean Squared Error (MSE) for AdaptoFlux across 8 Feynman tasks over 10 independent runs. The log-scale y-axis emphasizes the stability of the `prod` mode in multiplication tasks versus the high variance in complex tasks.

M.4 Analysis of Core Findings

1. **Under the restricted setting, the substrate recovered exact symbolic structures in multiplication-dominated tasks**: for F_ma, OhmsLaw, Power, E_mc2, and KE, the `prod` mode not only reached high numerical precision (MSE $\sim 10^{-13}$–10^{-11}) but also produced computation graphs that were **algebraically equivalent** to the target formulas under symbolic verification (Table 41), matching the specialized reference systems on these tasks. This demonstrates that the GDFG representation retains basic symbolic expressiveness—it can synthesize correct program structure, including formulas with explicit constants—even when stripped of method imports and driven by a lightweight probe.

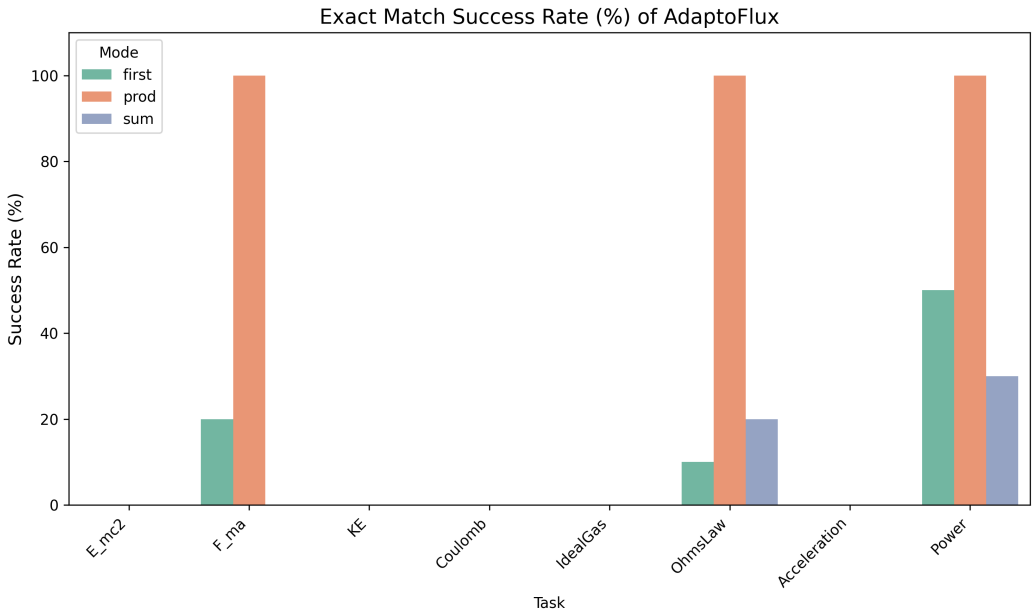


Figure 19: Exact Symbolic Match Success Rate (%) for AdaptoFlux across different collapse modes. The `prod` mode demonstrates robust stability (100% success) on multiplication-dominated tasks, whereas other modes show significant inconsistency.

Table 42: AdaptoFlux performance with collapse=`first` over 10 independent runs. Values report Mean ± Std for MSE and Runtime, along with Median, Min MSE, and Exact Match Success Rate.

Task	MSE (Mean ± Std)	Median	Min	Success Rate (%)	Runtime (s) ± Std
E_mc2	$3.50e+03 \pm 1.21e+03$	$3.47e+03$	$1.55e+03$	0.0	3.10 ± 1.75
F_ma	$4.31e+02 \pm 3.28e+02$	$4.13e+02$	$2.77e-12$	20.0	3.39 ± 6.96
KE	$1.16e+04 \pm 5.28e+03$	$1.09e+04$	$3.71e+03$	0.0	10.14 ± 11.53
Coulomb	$4.78e+01 \pm 2.11e+01$	$4.49e+01$	$1.19e+01$	0.0	7.90 ± 8.58
IdealGas	$4.87e+04 \pm 8.04e+03$	$4.70e+04$	$3.47e+04$	0.0	7.44 ± 4.85
OhmsLaw	$1.04e+02 \pm 7.77e+01$	$8.71e+01$	$5.28e-13$	10.0	1.51 ± 1.72
Acceleration	$1.58e+01 \pm 6.16e+00$	$1.49e+01$	$6.68e+00$	0.0	2.41 ± 3.23
Power	$2.81e+02 \pm 2.88e+02$	$2.05e+02$	$1.49e-12$	50.0	0.54 ± 0.30

Table 43: AdaptoFlux performance with collapse=`prod` over 10 independent runs. This mode demonstrates the highest stability and success rate in multiplication-dominated tasks (F_ma, OhmsLaw, Power).

Task	MSE (Mean ± Std)	Median	Min	Success Rate (%)	Runtime (s) ± Std
E_mc2	$2.64e-11 \pm 7.80e-12$	$2.53e-11$	$1.59e-11$	0.0	0.07 ± 0.01
F_ma	$2.88e-12 \pm 1.96e-12$	$2.42e-12$	$1.93e-13$	100.0	0.02 ± 0.00
KE	$1.31e+03 \pm 1.86e+03$	$8.29e-11$	$2.27e-11$	0.0	2.61 ± 3.14
Coulomb	$5.65e+01 \pm 6.85e+01$	$2.25e+01$	$9.40e+00$	0.0	9.20 ± 3.30
IdealGas	$1.28e+05 \pm 2.04e+05$	$5.55e+04$	$2.25e+04$	0.0	15.17 ± 7.40
OhmsLaw	$8.04e-13 \pm 5.15e-13$	$5.89e-13$	$2.21e-13$	100.0	0.02 ± 0.00
Acceleration	$2.36e+02 \pm 5.36e+02$	$1.73e+01$	$5.38e+00$	0.0	11.62 ± 4.05
Power	$2.65e-12 \pm 1.41e-12$	$2.07e-12$	$8.05e-13$	100.0	0.02 ± 0.00

Table 44: AdaptoFlux performance with collapse=`sum` over 10 independent runs. Generally exhibits higher variance and lower success rates compared to `prod`, particularly in complex tasks.

Task	MSE (Mean ± Std)	Median	Min	Success Rate (%)	Runtime (s) ± Std
E_mc2	$8.84e+02 \pm 7.94e+02$	$5.33e+02$	$7.73e+00$	0.0	154.79 ± 201.62
F_ma	$1.50e+02 \pm 5.70e+01$	$1.57e+02$	$4.11e+01$	0.0	12.78 ± 20.35
KE	$2.33e+03 \pm 1.88e+03$	$1.44e+03$	$7.84e+02$	0.0	159.76 ± 153.73
Coulomb	$4.27e+01 \pm 2.15e+01$	$3.81e+01$	$9.73e+00$	0.0	7.52 ± 5.12
IdealGas	$4.38e+04 \pm 9.34e+03$	$4.21e+04$	$3.14e+04$	0.0	11.50 ± 19.96
OhmsLaw	$3.06e+01 \pm 1.99e+01$	$2.98e+01$	$1.26e-12$	20.0	4.82 ± 3.52
Acceleration	$6.52e+01 \pm 5.45e+01$	$3.84e+01$	$9.58e+00$	0.0	19.94 ± 40.29
Power	$1.01e+02 \pm 9.30e+01$	$1.22e+02$	$1.76e-12$	30.0	25.39 ± 42.22

2. **ATF can correctly combine input variables containing constants, but its performance is highly sensitive to search configuration**: In the Kinetic Energy (KE) task, where the constant 0.5 in the target formula $0.5 \cdot m \cdot v^2$ is provided as an independent input dimension, AdaptoFlux successfully constructed the correct multiplication chain ($x_2 \cdot x_0 \cdot x_1 \cdot x_1$) in `prod` mode, achieving algebraic equivalence (Table 41); whereas it failed completely in `sum` or `first` modes (Tables 38, 36). This contrast clearly indicates that **AdaptoFlux's search effectiveness strongly depends on configurations aligned with the mathematical structure of the task**. For three-variable nonlinear tasks like IdealGas and Coulomb, ATF has not yet recovered the correct symbolic structure under the current uniform `prod` mode setting; however, manual inspection of its best solutions reveals that the output programs often miss only one or two key operations (such as a division or variable copy). This suggests that success rates could be significantly improved by introducing more flexible collapse strategies, increasing graph depth, or expanding method pool capacity—but to maintain fair comparison with baselines, this experiment did not adjust hyperparameters or search strategies for specific tasks.
3. **The choice of Collapse mode is critical**: The `prod` mode performed excellently in multiplication-like tasks, while `first` and `sum` modes were generally poor. This reveals a key design principle of AdaptoFlux: the output aggregation strategy must align with the mathematical structure of the target task.
4. **Inspectable errors**: the recovered program graphs (Fig. 17) expose which variables and operations contribute to each prediction, including cases with substantial numerical error.

M.5 Conclusion

With `prod` aggregation, the original AdaptoFlux probes recovered algebraically equivalent expressions for F_ma, OhmsLaw, Power, E_mc2 and KE. Recovery depended strongly on the aggregation mode and on the supplied operators. PySR and gplearn also solved Coulomb, IdealGas and Acceleration, where the tested ATF configurations failed. Together, the numerical and symbolic checks identify both successful compositions and the remaining search gaps under this arithmetic pool.

N Hand-Crafted Snake Controller

To verify AdaptoFlux's native support for side-effectful actions, we manually constructed a Generalized Data Flow Graph (GDFG) to control an agent in the classic Snake game. This entire process required no training or parameter tuning.

As shown in Figure 20, the graph consists of four logical layers:

- **Input Extraction Layer**: Reads the current game state (e.g., snake head position, food coordinates, presence of hazards ahead);
- **Perception and Reasoning Layer**: Computes spatial relationships (e.g., "hazard ahead," "food to the right");
- **Decision Logic Layer**: Executes symbolic rules based on pure functions from the method pool (e.g., "if hazard ahead, then turn left");

- **Action Execution Layer**: Invokes methods with side effects (such as `turn_left()`, `move_forward()`), directly modifying the environment state via `SNAKE_STATE.set_action()`.

The action layer contains three alternatives, `turn_left`, `turn_right` and `go_straight`, whose Boolean permissions are mutually exclusive. They are not connected as a sequential action chain: only the permitted branch updates the environment in a tick. This application-level exclusion explains the behavior of the original controller. The revised executor additionally supports declared resource ordering for graphs with several conflicting active operations. The stored graph remains static, and one graph evaluation selects each step's action.

Although this controller is simple in structure and unoptimized, it operates stably for hundreds of steps in a 10×10 grid, effectively avoiding self-collision and continuously tracking food. This result validates the core claim of AdaptoFlux: **it provides a unified and executable underlying framework for both pure computation and interactive behavior**—as illustrated by the explicit state-changing action interface. We have provided the running video of this model in the supplementary materials.

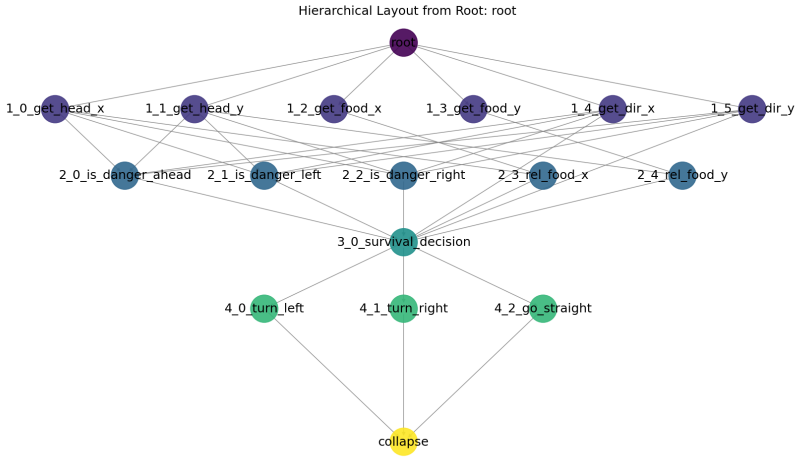


Figure 20: Hand-crafted GDFG for the Snake agent. Action-layer nodes execute operations with side effects (such as `turn_left`), directly modifying the environment state. Mutually exclusive permissions select one action branch within the same graph as the pure computations.

O Implementation Details and Reproducibility Guide

Appendix K records the original five experiment categories. The directory map below identifies their configurations, method pools, logs and execution scripts. The supplementary revision experiments and their frozen protocols are described in Appendices B–H.

O.1 Method Pool Interface and Decorator Implementation

The formal definition $m = \langle \text{impl}, \sigma_{\text{in}}, \sigma_{\text{out}}, \mathcal{M} \rangle$ from Main Text § 3.1 is bound in the prototype system via the `@method_profile` decorator. This mechanism transforms host language functions into atomic units parseable by the framework, automatically injecting type routing and metadata:

```
def method_profile(
    output_count: int | None = None,
    input_types=None,
    output_types=None,
    group: str = "default",
    weight: float = 1.0,
    is_internal_decorator: bool = False,
    vectorized: bool = False
):
```

```
    def decorator(func):
        func.output_count = output_count if output_count else len(
            output_types or [])
        func.input_types = input_types or []
        func.output_types = output_types or []
        func.group = group
        func.weight = weight
        func.is_internal_decorator = is_internal_decorator
        func.vectorized = vectorized
        return func
    return decorator
method_profile.is_internal_decorator = True
```

Listing 2: Original decorator interface used by the historical experiments

Mapping Relationship: Attributes injected by the decorator directly correspond to the formal tuple $\mathcal{M}$. When constructing graph edges, the framework enforces hard type-constrained routing by comparing `output_types` with downstream nodes' `input_types`; `group` and `weight` drive dynamic view projection and probe sampling, and neither participates in the definition of Ω (Appendix P.4).

The revised decorator adds `pure`, `reads` and `writes` declarations. These are execution contracts used to derive control dependencies; `group` and `weight` remain search preferences. Appendix A describes how the declarations and execution-mode setting are preserved.

O.2 Experiment Configuration and Repository Path Mapping

To ensure reproducibility transparency, all experiments adopt a self-contained directory structure. Table 45 establishes a precise index between main text experiments and repository assets.

Path Note: All scripts are located in the `experiments/` directory of the anonymous repository. The table below lists relative paths and core output files.

Table 45: Experiment Configuration and Repository Path Mapping

LayerGrow (Sec. 4.1, Appx. K.1)
`LayerGrowTrainer_exp1/main.py`
→ `experiment_results_summary.csv`

Knowledge Transfer (Sec. 4.2, Appx. K.2)
`GraphEvo_exp3/exp3_main.py`
→ `exp3_results/summary_table.csv`

Minimal Stateful Attractor (Sec. 4.3, Appx. K.4)
`minimal_stateful_comparison/run_comparison.py`
→ `logs/` + `results/stateful_synthesis_comparison.png`

Topology-Aware Parallel (Appx. K.3)
`PipelineParallel_exp2/scripts/benchmark.py`
→ `PipelineParallel_exp2/results/`

Geometric Classification (Appx. L)

- `Combined_exp4_circles/test_combined_trainer.py`
- `Combined_exp4_spiral/test_combined_trainer.py`
- `Combined_exp4_spiral_no_Trigonometric/test_combined_trainer.py`

→ `experiment_results/` (regenerated by the script; ≈0.5 GB per task, not shipped)

Feynman Regression (Appx. M)
`ExternalBaselines/run_baselines_*.py`
→ `ExternalBaselines/results/`

Snake Controller (Appx. N)
`embodied_snake/snake_main.py` + `snake.json`[†]
→ `snake_demo.mp4`

[†]Note: `snake.json` is a hand-crafted GDFG configuration file (not learned).

Note: All paths in the table are relative to the anonymous code repository root. Output artifact types vary by task, including: run logs and trajectories (`.log`/`.json`), statistical metric tables (`.csv`), execution demo videos (`.mp4`), and graph configuration files. The complete directory structure, environment dependencies, and one-click reproduction guides are detailed in the `README.md` file in the repository or supplementary materials.

Raw result tables—the LayerGrow configuration summary, the knowledge-transfer summary, the parallel-scheduling benchmark, the Feynman-baseline results and the minimal-stateful run logs—are shipped, so that the reported statistics can be recomputed independently. Larger outputs are regenerated by the corresponding scripts: the `training_log.json` files under `exp_max_layers_*/run_*/`, the contents of `exp3_results/<config>/rep<k>/`, `models/` and `plots/`, and the three geometric-classification `experiment_results/` directories (≈0.5 GB each). These outputs are excluded from the supplement for size.

O.3 Reproduction Environment and Running Guide

Software Dependencies: AdaptoFlux is developed based on Python 3.12+. The complete dependency list is in `requirements.txt` at the repository root, installable via `pip install -r requirements.txt`. The framework executes entirely on CPU, with no GPU dependencies. **Compute Resources:** All experiments were executed on standard workstations with multi-core CPUs and 60GB RAM. No GPU was used. Specifically, the parallel scalability experiments utilized a server with 32 CPU cores to evaluate multi-threading performance. The total wall-clock time to reproduce all main experimental results sequentially is estimated to be less than one week. **Statistical Protocol Alignment:** The random seed allocation, repetition counts, and success determination thresholds for each experiment strictly follow the regulations outlined in the Main Text and Appendices. All raw

logs have been fully exported, supporting independent recalculation of means, variances, and success rates.

P SEMANTIC TAXONOMY OF THE METHOD POOL

Main Text Section 3.1 introduced the engineering implementation of the Method Pool $\mathcal{Q}$ and its metadata-driven architecture. This section further provides two typical classification examples from a semantic perspective to demonstrate the flexibility and expressive potential of the method pool in handling heterogeneous operations. It is emphasized that the following classification systems serve only as **Illustrative Examples**, aiming to elucidate how the framework establishes structural order through metadata, rather than imposing hard constraints on the method pool. AdaptoFlux natively supports users defining arbitrary orthogonal classification views based on task requirements (e.g., by hardware affinity, computational complexity, or semantic role).

P.1 EXAMPLE 1: HETEROGENEOUS OPERATION CLASSIFICATION BASED ON FUNCTIONAL ATTRIBUTES

To natively bridge pure computation and state interaction, AdaptoFlux's method pool $\mathcal{Q}$ supports semantically tagging methods from two orthogonal functional dimensions: **Mapping** and **Action**. It is emphasized that these two dimensions are not mutually exclusive sets but allow overlapping feature labels—a method can simultaneously possess deterministic input-output mapping relationships and side effects that change system state or environment. This flexibility allows the framework to uniformly model a wide range of operations from pure mathematical formulas to embodied control instructions.

Mapping Perspective ($\mathcal{Q}_F$) Focuses on the input-output transformation relationship f : Input $\rightarrow$ Output. For a pure deterministic implementation, these methods compute repeatable outputs and support symbolic regression and feature extraction; declared types constrain their connections. Even if a method contains side effects, its mapping part can still be independently analyzed for data flow reasoning.

Action Perspective ($\mathcal{Q}_O$) Focuses on the effect produced on the system state $\mathbf{s}_{t+1} = \Phi(\mathbf{s}_t, \mathbf{x}_t)$ or the external environment after method execution. Such methods introduce time dependency and state evolution capabilities, forming the core of building dynamic systems and embodied agents.

Example Comparison and Overlap Cases

- **Pure Mapping Example**: $f(x) = x \times \frac{9}{5} + 32$ (Celsius to Fahrenheit). No state dependency, no side effects, only returns calculation results.
- **Pure Action Example**: `send_notification(msg)`. The primary purpose is to execute I/O operations; the return value is usually secondary or empty, with the core being the triggered external effect.
- **Mixed Case Example**: `update_and_return_score(state, action)`. This method both calculates a numerical score based on current state and action (mapping characteristic) and records the action in the historical trajectory while updating the environment state (action characteristic). AdaptoFlux allows such methods to simultaneously assume the dual roles of data transmission and state evolution in the graph.

P.1.1 SUBDIVISION OF ACTION POOL BY SIDE-EFFECT SCOPE

The action pool $\mathcal{Q}_O$ can be further subdivided based on the object affected by side effects (see Table 46). This classification determines the state update strategy of the graph execution engine during time-step unrolling.

Table 46: [Example] Action Pool Classification by Side-Effect Scope

Type	Affects Self-State	Affects External Env	Typical Application
Self Side-Effect	✓	×	Internal counter updates, memory storage
Env Side-Effect	×	✓	API calls, robot movement commands
Dual Side-Effect	✓	✓	Perception-action loops in embodied intelligence

P.2 Example 2: Reversibility Classification Based on Information Flow Attributes

For the function pool $\mathcal{Q}_F$, we can classify methods based on the ability to restore inputs from outputs. This perspective demonstrates the theoretical potential of the method pool in supporting reverse reasoning or lossless encoding tasks.

Table 47: Function Pool Classification by Reversibility

Type	Characteristic	Reverse Deduction Ability
Non-Reversible (N)	Many-to-One Mapping	Cannot uniquely restore; only probabilistic inference
Conditionally Reversible (CR)	Requires partial input or context	Can uniquely restore remaining inputs given partial inputs
Fully Reversible (FR)	One-to-One Mapping (Bijective)	Can uniquely restore original inputs solely from output

Fully Reversible Function Pool Example Based on the Fundamental Theorem of Arithmetic, construct the function $f(a, b) = 2^a \cdot 3^b$ $(a, b \in \mathbb{N})$. Due to the uniqueness of prime factorization, any output y can be uniquely inverted to (a, b). Such functions have theoretical value in scenarios requiring lossless state recovery or encoding/decoding, demonstrating AdaptoFlux's ability to handle complex mathematical properties.

P.3 Dynamic Construction and Task-Adaptive Filtering

The above classifications are not static labels but the foundation for dynamic filtering. AdaptoFlux allows dynamically constructing sub-pools through metadata projection. This is a search-side restriction that leaves Ω unchanged (Appendix P.4).

P.3.1 Method Registration and Global State

Methods are registered via the `@method_profile` decorator and can be associated with global state storage (such as memory dictionaries). This allows action functions to share information across time steps, achieving long-term memory without hard-coding state variables.

P.3.2 Task-Adaptive Sub-Pool Filtering Mechanism

Facing large-scale base method pools, the search space grows exponentially. To improve efficiency, subset filtering mechanisms can be introduced. Here we provide a simple example using a Genetic Algorithm (GA) to optimize method subsets:

1. **Initialization**: Randomly sample multiple fixed-size method subsets.
2. **Evaluation**: Run lightweight `LayerGrow` on each subset, using validation set accuracy as fitness.
3. **Evolution**: Iteratively optimize through selection, crossover, and mutation, eliminating redundant or interfering operators.

This process aims to find a "semantically complete and minimal" sub-pool. The performance fluctuation of `CombinedTrainer_periodic` in Appendix L reflects this trade-off: excessive compression may lead to the loss of key semantic primitives (such as trigonometric functions), thereby causing semantic collapse of the expression space. This further confirms that the composition of the method pool has a decisive impact on navigation efficiency.

P.4 Hypothesis Class and the Formal Role of Metadata

Data compatibility and execution contracts. For a registered pool Q and type system Σ, Eq. 1 defines the admissible data DAGs. Port signatures constrain which outputs may supply which arguments, and acyclicity admits a topological schedule. These checks concern declared interfaces. State-dependent execution additionally uses the program contract $P = (G, \pi, E_c)$: a frozen semantic order π and control dependencies E_c derived from declared resource accesses.

Search preferences select a view. For a filter ϕ, define $\mathrm{Q}_{\mathrm{view}}(\phi) = \{m \in \mathrm{Q} \mid \phi(\mathrm{M}_m) = \mathrm{True}\}$. Filtering by search tags such as `group` restricts the examined data graphs to $\Omega_{\mathrm{Q}_{\mathrm{view}},\Sigma} \subseteq \Omega_{\mathrm{Q},\Sigma}$. Sampling weights alter the distribution of proposals within that view. These preferences are distinct from `pure`/`reads`/`writes`: changing an effect declaration changes the execution contract and may change the required schedule.

Table 48: Interface constraints, execution contracts and search preferences have different roles.

Mechanism	Role	Operational effect
Type signatures and ports	Data compatibility	Admit connections to declared argument positions; the three-condition experiment varies these signatures.
Acyclicity	Data schedulability	Reject cycles in the stored data graph.
Read/write declarations; saved order	State execution contract	Determine control dependencies when effect ordering is enabled; preserve the selected order through evaluation and persistence.
Group and search tags	Candidate view	Select the subset of registered methods exposed to a probe.
Sampling weights	Proposal preference	Alter how often methods are proposed.
Reuse history	Search bookkeeping	Guide extraction and reuse; extracted methods retain their execution contracts.

Evidence. The type-prior results are in Appendix B, and execution-contract validation is in Appendix A. The original pool-composition experiments in Appendix L evaluate combined search configurations; a dedicated matched ablation isolating only the filter ϕ remains separate from those results.

P.5 Formalization of Boundary Nodes: Root and Collapse

To address the formal completeness of the GDFG definition (Equation 1 in Main Text), we explicitly define the behavior of boundary nodes V_{root} and V_{col}. Unlike processing nodes (V_{proc}) that draw operations from the Method Pool $\mathcal{Q}$, boundary nodes serve as structural interfaces with distinct configuration mechanisms.

Root Nodes (V_{root}): Typed Input Distributors. In the AdaptoFlux implementation, the Root node functions as a *Typed Input Distributor*. It does not perform computational logic but acts as a semantic anchor that injects external features into the graph while strictly enforcing type signatures. Formally, let $\mathbf{x}_t \in \mathbb{R}^{D_{in}}$ be the input vector at time step t. The Root node $v_r \in V_{root}$ decomposes $\mathbf{x}_t$ into a set of typed tokens $\{x_t^{(i)}\}_{i=1}^{D_{in}}$, where each token carries a type signature $\sigma_{out}(v_r^{(i)}) = \text{type}(x_t^{(i)})$ derived from the `input_types_list` configuration. The influence of the Root node is defined as an identity mapping for data injection:

$$I_{v_r}(s_t, \mathbf{x}_t) = \mathbf{x}_t \tag{47}$$

This design decouples data ingestion from semantic processing, allowing the same graph topology to accept heterogeneous input schemas without structural modification.

Collapse Nodes (V_{col}): Independent Output Aggregation. Crucially, the Collapse node operates **independently of the Method Pool** $\mathcal{Q}$. While internal nodes (V_{proc}) are bound to methods $m \in \mathcal{Q}$, the Collapse node is configured via a separate `CollapseFunctionManager` that handles global output projection. Formally, let $\mathbf{H}_t = \{I_u(s_t, \mathbf{x}_t) \mid u \in \text{pred}(v_c)\}$ be the set of inputs from all predecessor nodes connected to the collapse layer. The Collapse node $v_c \in V_{col}$ computes:

$$I_{v_c}(s_t, \mathbf{x}_t) = \mathcal{A}(\mathbf{H}_t) \tag{48}$$

where $\mathcal{A}$ is an aggregation operator selected from a predefined set of structural reducers (e.g., `SUM`, `PROD`, `IDENTITY`, `ENERGY`), as defined in the `CollapseMethod` enum. This separation ensures that:

- **Structural Consistency**: The aggregation logic (e.g., summing all outputs) remains stable regardless of the evolutionary changes in the internal method pool.
- **Dimensional Flexibility**: Operators like `IDENTITY` allow the graph to output raw intermediate representations without forcing a scalar reduction, supporting multi-output tasks.
- **Decoupled Optimization**: The search for optimal internal semantics ($\mathcal{Q}$) is orthogonal to the selection of the output aggregation strategy ($\mathcal{A}$).

Engineering Implementation Details. In our Python prototype, this separation is realized through distinct classes:

- **Method Pool**: Managed by `@method_profile` decorators, governing V_{proc} nodes.
- **Collapse Manager**: Implemented in `CollapseFunctionManager`, which dispatches to static aggregation functions (e.g., `_sum`, `_product`) based on the user-configured `collapse_method`. This manager is initialized independently of the graph's evolutionary state.

Detailed code snippets for the `CollapseFunctionManager` are provided in the anonymous code repository under `ATF/CollapseManager/collapse_functions.py`.

P.6 Static Acyclicity and Runtime Loop Semantics

Equation 1 constrains the *stored* topology of a GDFG to be acyclic, whereas the behaviour a GDFG induces may be closed-loop. This section makes that distinction explicit, since it is the basis on which the framework accommodates loop-like and recursive computation without giving up static scheduling.

Setup. Fix a valid data graph G and its per-step execution contract $P = (G, \pi, E_c)$. Let the environment state $\mathbf{s}_t$ include all persistent resources needed by the program. For fixed inputs and random choices, denote one successful evaluation by Φ_P and the next-observation interface by Π. A trace of horizon T satisfies

$$\mathbf{s}_{t+1} = \Phi_P(\mathbf{s}_t, \mathbf{x}_t), \qquad \mathbf{x}_{t+1} = \Pi(\mathbf{s}_{t+1}). \tag{49}$$

The observation interface may read the environment directly; the collapse output need not itself contain the complete environment state.

Proposition 1 (loop–DAG correspondence). For a fixed successful per-step execution contract and a finite horizon T, the trace in Eq. 49 has an equivalent unrolled acyclic representation. The construction uses T copies of the augmented graph $G^+ = (V, E_d \cup E_c)$, carries the complete state and next observation from one copy to the next, and preserves the within-step order of conflicting accesses.

Proof. Within a step, the frozen order π orients all data and control dependencies forward. Between steps, state and observation flow from t to $t+1$. A lexicographic order by $(t, \text{position in } \pi)$ therefore orients every dependency forward, so the unrolled graph is acyclic. Induction on t gives execution equivalence: the first copy receives the same initial state and input, and each completed copy produces precisely the state and observation passed to the next one. Thus each copy executes the same transition Φ_P. □

Three routes by which a loop enters.

- **Time-step unrolling** (Eq. 49): the loop is temporal and lies outside G; unrolling is precisely the construction of Proposition 1. The per-step data-graph class is unchanged; the external controller chooses the horizon.
- **Self-invocation and sub-graph invocation**: a node binds an arbitrary callable impl, so a method may call itself or execute a further registered graph. The engine performs one opaque node evaluation, so type-checking and scheduling of the enclosing graph are unaffected; a nested graph carries its own execution contract, and its outer declaration summarizes the resources accessed inside it. Recursion depth and termination are properties of impl, which the framework neither analyses nor constrains.
- **Output or environment feedback**: the next step may consume the collapse output, reread the updated environment, or combine both through Π. The full state transition is retained independently of how the numerical outputs are aggregated.

Table 49: Where a loop lives, and what it changes.

Route	Formal position	Changes $\Omega_{\mathrm{Q},\Sigma}$?	Visible to the scheduler?
Time-step unrolling	outside G, in the trace	no	per-step contract reused
Self-invocation / sub-graph	inside impl (opaque)	no	no (one node evaluation)
Collapse feedback	boundary interface V_{col}	no	no (inter-step edge)

Preserved structure. The stored graph stays acyclic, while the augmented dependency graph determines which operations may run concurrently. Loop unrolling establishes a finite-trace representation; recursion and termination remain properties of the callable implementation. The original LayerGrow/-GraphEvo probes search the per-step graph and exhibit the trajectory-dependent optimization failures reported below. The structured CMA controller in Appendix E provides a separate learned control example with a supplied scaffold.

Q Challenges in Automated Synthesis of Stateful Policies

In the original study, alongside the hand-crafted Snake controller in Appendix N, we conducted preliminary experiments using the original navigation probes (LayerGrow/GraphEvo) to automatically synthesize policies with side effects on *Snake* and *Flappy Bird* tasks. The source code for these experiments can be found in the `experiments/embodied_snake` and `experiments/embodied_bird` directories of the anonymous repository.

Observations on Degenerate Local Optima. Under the tested method pools, rewards and original LayerGrow/GraphEvo configurations, the searches converged to **degenerate local optima** that maximized proxy rewards rather than the intended task objectives:

- **Snake Task (Survival Bias):** The agent almost immediately converges to a "continuous turning" strategy. Since the agent does not consume food, its length remains constant, allowing it to survive indefinitely by circling within a safe local area. Although this behavior yields a high survival score, task progress is zero, vividly demonstrating how greedy probes exploit reward proxies without achieving semantic goals.
- **Bird Task (Immediate Failure):** The agent tends to converge to a "continuous jumping until death" strategy or remains stationary, failing to develop obstacle-avoidance behaviors with long-term vision.

In contrast, when GDFG is used purely as a numerical discriminator (i.e., outputting action probabilities rather than directly executing side effects), its performance is significantly more stable. This phenomenon indicates that current greedy replacement-based probes face significant limitations in navigating **Trajectory-dependent Loss Landscapes** introduced by stateful operations.

Explanation of Experimental Code Structure. To clearly demonstrate AdaptoFlux's **Representational Completeness**, we made specific adjustments to the provided codebase:

- **Snake Task (Demo Only):** Since the primary goal of the Snake example in Appendix N is to serve as an *Existence Proof* of the framework's native ability to execute stateful operations, we have **removed the training scripts** from the `experiments/embodied_snake` directory. The provided `snake_main.py` strictly executes the pre-defined JSON configuration to reproduce the behavior of the hand-crafted controller. This distinguishes the fixed execution example from the original automatic-synthesis attempts. The minimal task in Section 4.3 and the later structured CMA experiments in Appendix E use separate protocols.

- **Bird Task (Training Reference):** For reviewers wishing to attempt automatic synthesis of stateful policies, we retained the complete training pipeline in `experiments/embodied_bird`.

  Code Version and Reproduction Note: Please note that the Bird experiment code was developed during an early iteration of the framework. Although the core logic of configuring side-effect method pools and running LayerGrow/GraphEvo probes remains valid and reference-worthy, some API interfaces in the main branch may have evolved. We recommend using this directory primarily as a structural reference for setting up embodied training loops, rather than expecting exact experimental logs to be reproduced with a one-click run without minor adaptations.

  Parts that may require modification for reproduction include: method pool file definitions, loss function interfaces, disabling graphical display functions, and modifying variable access paths. Additionally, this directory contains an early hand-crafted model JSON file; since some methods referenced therein have been removed from the current method pool, **direct loading of this JSON file is not recommended**; instead, refer to the training logic in its Python scripts.

Theoretical Significance and Future Directions. The difficulty in automating these tasks empirically validates the core argument of Main Text § 3.3: **There is a significant Trade-off between the expressiveness of stateful operations and their optimization difficulty.** On such long-horizon tasks, current search strategies are insufficient to effectively explore non-monotonic paths introduced by state transitions, often falling into semantically hollow local optima (e.g., infinite survival without growth); the minimal stateful task of Section 4.3 shows that the same probes do succeed when the required stateful graph is small.

We emphasize that this limitation is not a flaw in the framework design but an inherent challenge of *Credit Assignment* in discrete program synthesis. Potential future directions to break this bottleneck include:

- **Hierarchical Search Strategies:** Introducing macro-action abstractions to encapsulate low-level side-effect operations into high-level semantic units, thereby reducing the dimensionality of the search space.

- **Fine-Grained Type Systems:** Extending metadata $\mathcal{M}$ to include pre/post-conditions of state transitions (e.g., requiring "consuming food" as a prerequisite for positive rewards), thereby pruning invalid state paths during candidate generation.

- **Trajectory-Aware Regularization:** Designing loss function smoothing terms targeted at long-range dependencies to guide the optimizer out of short-sighted local optima like "continuous turning."

These observations locate concrete reward and search failures in the original LayerGrow/GraphEvo control configurations. The later CMA study uses an explicit action-scoring scaffold and reports learned controllers separately in Appendix E.

R Prospective Integration of Language-Model Candidate Generators

We propose exploring language models as candidate generators for AdaptoFlux. A model could receive a task description, the registered method signatures and documentation, and the graph schema, then suggest a complete graph or edits to an existing candidate. This would connect language-model-guided search to the representation's common execution and evaluation interface.

Proposed workflow. A proposed integration would resolve each candidate against the Method Pool and check method availability, arity, port compatibility, and acyclicity. Declared resource accesses and the selected program order would determine its control dependencies under the execution contract in Appendix A. After validation, actual execution would produce task scores and state/action trajectories. Validation diagnostics and measured outcomes could then guide the model's next proposal, forming a propose–validate–execute–feedback loop.

Future evaluation. The proposed integration could be evaluated by executable-candidate rate, task success, search budget, and generation and validation costs. Its implementation and empirical evaluation remain future work.

S Limitations

While AdaptoFlux establishes a structured foundation for discrete program synthesis, several limitations remain. First, the current navigation probes (LayerGrow and GraphEvo) rely on local search heuristics, which may struggle with long-horizon dependency tasks or complex state trajectories where credit assignment is difficult. The experiments identify this bottleneck for the tested optimization configurations. Second, the topology-based knowledge extraction mechanism is currently in a prototype stage. While it demonstrates feasibility in controlled settings, it requires further engineering refinement and debugging to ensure robustness in noisy, real-world environments. Finally, our current Python-based prototype faces engineering constraints. Specifically, it has limited support for vectorized operators (e.g., batched matrix operations), where fine-grained node-level scheduling overhead may outweigh computational benefits. Additionally, the Global Interpreter Lock (GIL) constrains fine-grained parallelism for CPU-bound tasks. These implementation bottlenecks suggest that migrating to system-level backends (e.g., C++ or Rust) and adopting hybrid execution models would be necessary for high-performance numerical tasks.

T Broader Impacts

This work contributes to the field of neuro-symbolic AI and program synthesis, with several potential societal implications:

Positive Impacts. AdaptoFlux enhances the **interpretability** and **transparency** of automated decision-making systems by generating explicit, human-readable symbolic structures. This can help mitigate the "black box" risks associated with deep learning, particularly in safety-critical domains where understanding the logic behind a decision is paramount. Furthermore, our framework's **computational efficiency** (CPU-only, low memory footprint) lowers the barrier for entry, promoting more accessible and energy-conscious AI research compared to resource-intensive large-scale models.

Negative Impacts and Risks. We acknowledge that automated program synthesis technologies carry potential **dual-use risks**. The ability to automatically generate executable code could theoretically be misused for creating malicious scripts, automating cyber-attacks, or bypassing security controls. Additionally, if the method pool or training data contains biases, the synthesized programs might perpetuate or amplify these biases in downstream applications.

Mitigation Strategies. To address these risks, we advocate for the responsible use of AdaptoFlux. We encourage researchers and practitioners to implement appropriate safety checks, such as sandboxed execution environments and code verification steps, when deploying synthesized programs. Future

work should also focus on developing robust filtering mechanisms for the method pool to prevent the inclusion of harmful or biased operators.

U Discussion and Future Work

This section discusses the theoretical implications of our current design choices regarding boundary nodes and outlines a concrete roadmap for systems engineering improvements, specifically focusing on parallel scalability.

U.1 Reinterpreting Boundary Nodes: From Aggregation Heuristics to Semantic Alignment

In the current implementation and experiments of AdaptoFlux, we predominantly assign specific collapse functions (e.g., `SUM`, `PROD`) to V_{col} nodes to align internal representations with target outputs. However, we posit that this design choice is largely a **compromise necessitated by the limitations of current optimization probes**, rather than an intrinsic theoretical requirement of the GDFG formalism.

The "Collapse" as a Proxy for Optimization Constraints. Ideally, a sufficiently powerful optimizer should achieve semantic alignment directly through feature control during graph generation. In such an ideal scenario, the internal topology and operator selection would naturally structure the data flow such that the final nodes inherently encode the target semantics without requiring post-hoc aggregation heuristics.

Currently, the explicit collapse function acts as a *semantic corrector* that compensates for the optimizer's inability to precisely shape the internal feature manifold. For instance, if the optimizer could perfectly control the semantic flow, the output node would naturally compute the required product or sum through its incoming edges and bound operators, rendering the external `PROD` or `SUM` collapse redundant. Thus, the current reliance on fixed collapse modes reflects the current ceiling of our navigation strategies in navigating the discrete combinatorial space.

Future Trajectory: Convergence to Pure Interfaces. As more advanced optimizers are developed—capable of finer-grained control over topological and semantic evolution—we anticipate a fundamental shift in the semantic role of boundary nodes:

- **From Aggregation to Identity:** The role of the V_{col} node will diminish from an active aggregator to a passive `IDENTITY` interface. It will serve merely as the "End" marker of the semantic flow, exposing the final computed state without altering it.
- **Symmetric Boundary Semantics:** Correspondingly, the V_{root} node serves as the "Start" marker, injecting raw data. In a fully optimized system, both V_{root} and V_{col} will converge to pure structural interfaces. The core logic of semantic transformation will reside entirely within the processing nodes (V_{proc}) and their topological connections, rather than being offloaded to boundary aggregation rules.

This perspective reinforces the decoupling principle of AdaptoFlux: the representation space (GDFG) is designed to be agnostic to the optimizer's sophistication. While current experiments require explicit collapse functions to bridge the gap between structural exploration and semantic precision, future work with stronger optimizers will likely render these functions obsolete, allowing the graph structure itself to bear the full burden of semantic alignment.

U.2 Systems Engineering: Migration to C++ for True Parallelism

A critical limitation of the current Python prototype is the Global Interpreter Lock (GIL), which constrains fine-grained parallelism for CPU-bound tasks. As noted in Appendix J.2.3 and K.3, while the augmented dependency graph identifies independent operations, the Python implementation incurs significant synchronization overhead and cannot achieve true parallel execution for pure computational nodes.

To unlock the full performance potential of AdaptoFlux, we plan to **migrate the core execution engine to C++**:

- **True Multi-Core Parallelism:** A C++ backend will eliminate the GIL bottleneck, enabling true parallel execution of independent nodes across multiple CPU cores. Its gains will depend on operator cost, available parallelism and memory traffic; the measurements in Appendix D provide a CPU-bound reference.
- **Reduced Scheduling Overhead:** By implementing a lock-free, topology-aware scheduler in C++, we can minimize the overhead of dependency tracking and data routing, which currently dominates runtime in fine-grained graphs.
- **Memory Efficiency:** Migrating to C++ will allow for manual memory management and the elimination of the current "full intermediate result cache" technical debt (Appendix J.2.1), significantly reducing memory footprint and improving cache locality for large-scale batch inference.

Such a backend is a path to reducing scheduling and interpreter costs in larger control and numerical workloads. Concurrently, we will focus on developing more robust optimization strategies to better exploit this enhanced computational substrate.

U.3 Intended Use and Enabling Scenarios

The experiments in this paper validate the substrate on a small set of tasks, so it is fair to ask what the representation enables in practice that a purely functional program space does not. This subsection answers that question at the level of design intent: the three scenarios below follow from the mechanisms already described, with structured action learning and library reuse now illustrated in Appendices E and G. Broader deployments require task-specific evaluation.

Registering heterogeneous agent primitives. Physical agents—different robots, or industrial controllers—expose callable operations that differ per embodiment and frequently carry side effects. Because the Method Pool accepts typed black-box callables (Section 3.1) and edge compatibility is checked statically, such primitives can be registered as they are and interleave with pure computation as *structured anchors*. Under this reading, imitation becomes fitting an action sequence inside a constrained pool rather than training a monolithic policy from scratch, so that learning can focus on composing existing primitives.

Reusing validated domain modules. In many deployments the building blocks already exist as validated artefacts: control logic, external APIs, or physical models. Side-effectful modules of this kind can be composed directly into the expression space instead of being relearned, so that the search operates over their composition while the space itself remains explorable—an alternative to blind data-driven exploration.

Symbolic skeleton, then continuous parameters. Because the stored topology is a typed DAG, macro structure and numeric parameters can be optimized in two stages: search the sequence of actions first, then fine-tune each method's parameters with the topology frozen. This differs from neural architecture search, which returns a static architecture: the object searched here is a dynamic action chain in which heterogeneous, stateful methods determine the execution flow, and the substrate supplies only the type-safe interface for the second stage rather than a joint end-to-end training algorithm.

Future work. Turning these scenarios into evaluated capabilities requires state-aware search strategies—hierarchical decomposition, or failure-driven constraint learning—that address the credit-assignment difficulty documented in Appendix Q, together with the systems work of Appendix U.2. The minimal stateful result and the structured controllers establish concrete starting points; broader long-horizon control remains an important direction.